

Конспект курса

Natural Language Processing and Reinforcement Learning

Подготовка к экзамену

Студент: _____

June 3, 2026

Contents

“`latex`

1 Лекция 1. Word Embeddings

1.1 Общая постановка задач NLP

Natural Language Processing (NLP) — область машинного обучения, связанная с обработкой текстов на естественном языке. Во многих прикладных задачах NLP текст нужно преобразовать в числовое представление, которое затем можно подать на вход модели машинного обучения.

Типичные задачи NLP:

- **Sentiment analysis** — определение тональности текста: положительная, отрицательная или нейтральная.
- **Spam filtering** — определение, является ли сообщение спамом.
- **Fake news detection** — обнаружение фейковых новостей.
- **Topic prediction** — определение темы текста.
- **Hashtag prediction** — предсказание подходящих хэштегов.

Многие из этих задач являются задачами классификации текста. В общем виде пайплайн выглядит так:

Text $\longrightarrow$ Feature Extraction / Representation Learning $\longrightarrow$ Classifier $\longrightarrow$ Label or Probability Distribution

Иными словами, исходный текст сначала переводится в вектор признаков фиксированной длины, а затем этот вектор передаётся в классификатор.

В качестве классификатора могут использоваться:

- линейный классификатор;
- логистическая регрессия;
- нейронная сеть;
- другие модели машинного обучения.

Главная проблема, обсуждаемая в этой лекции: **как представить текст в виде чисел так, чтобы модель могла эффективно с ним работать.**

1.2 Виды меток в задачах NLP

Метки в задачах NLP бывают дискретными и непрерывными.

1.2.1 Дискретные метки

Бинарная классификация:

$$y \in \{0, 1\}.$$

Примеры:

- спам / не спам;
- положительный / отрицательный отзыв.

Многоклассовая классификация:

$$y \in \{1, 2, \dots, C\}.$$

Пример: классификация товара по описанию в одну из нескольких категорий.

Многометочная классификация:

$$y \in \{0, 1\}^C.$$

В этом случае одному тексту может соответствовать сразу несколько меток. Пример — предсказание набора хэштегов.

1.2.2 Непрерывные метки

Иногда по тексту нужно предсказать вещественное число:

$$y \in \mathbb{R}.$$

Например, можно предсказывать цену товара по его текстовому описанию.

1.3 Предобработка текста

Перед построением признаков текст обычно нужно предобработать.

1.3.1 Токенизация

Токенизация — разбиение текста на отдельные элементы, называемые токенами.

Чаще всего токенами являются слова, но ими также могут быть:

- символы;

- подслова;
- знаки пунктуации;
- специальные токены.

Например:

“This is a sentence” $\longrightarrow$ [This, is, a, sentence].

Токенизация нужна, потому что модели машинного обучения обычно работают не с целой строкой, а с последовательностью элементарных единиц.

1.3.2 Нормализация токенов

Одна и та же лексическая единица может встречаться в разных формах:

dog, dogs $\longrightarrow$ dog,

bark, barks $\longrightarrow$ bark.

Идея нормализации состоит в том, чтобы привести разные формы слова к одной общей форме.

Основные подходы:

- stemming;
- lemmatization.

1.3.3 Stemming

Stemming — эвристическое отсечение или замена суффиксов слова для получения основы слова, называемой stem.

Пример:

consigned, consigning, consignment $\longrightarrow$ consign.

Стеммер обычно не пытается понять грамматику слова. Он применяет набор правил, например:

SSES $\rightarrow$ SS,

IES $\rightarrow$ I,

$$S \rightarrow \emptyset.$$

Примеры для Porter stemmer:

dresses $\rightarrow$ dress,

ponies $\rightarrow$ poni,

dogs $\rightarrow$ dog.

Популярные стеммеры:

- **Porter stemmer** — классический стеммер, опубликован в 1979 году.
- **Lancaster stemmer** — более агрессивный стеммер, опубликован в 1990 году.
- **Snowball stemmer** или Porter 2 — развитие Porter stemmer, сейчас часто используется на практике.

Недостатки stemming:

- **Overstemming** — разные по смыслу слова приводятся к одной и той же основе.
- **Understemming** — слова, которые следовало бы привести к одной основе, остаются разными.

Пример overstemming:

university, universal, universities, universe $\longrightarrow$ univers.

Здесь слова связаны морфологически, но не всегда должны считаться одним и тем же понятием.

Пример understemming:

data $\rightarrow$ dat,

datum $\rightarrow$ datu.

Хотя *data* и *datum* близки по смыслу, стеммер может привести их к разным основам.

1.3.4 Lemmatization

Лемматизация — приведение слова к его словарной форме, называемой леммой.

Пример:

playing, plays, played $\longrightarrow$ play.

В отличие от stemming, lemmatization обычно использует словари и грамматическую информацию.

Например, лемматизатор из NLTK:

- пытается найти словарную форму слова;
- основан на базе WordNet;
- работает лучше, если дополнительно передать part-of-speech tag, то есть часть речи.

WordNet — лексическая база данных, в которой слова организованы по смысловым связям: синонимия, гипонимия, гиперонимия и т.д.

1.3.5 Что ещё нужно учитывать при предобработке

Помимо токенизации и нормализации, часто нужно обрабатывать:

- заглавные буквы;
- пунктуацию;
- сокращения;
- числа, даты, идентификаторы, номера страниц;
- stop-words: *the*, *is*, *a* и т.д.;
- HTML/XML-теги.

Полезные инструменты:

- NLTK;
- `nltk.stem.SnowballStemmer`;
- `nltk.stem.PorterStemmer`;
- `nltk.stem.WordNetLemmatizer`;
- `nltk.corpus.stopwords`;

- BeautifulSoup для парсинга HTML;
- регулярные выражения через re;
- pymorphy2 для русского языка.

1.4 Bag-of-Words

Bag-of-Words, BoW — классический способ представить текст в виде вектора.

Пусть есть словарь размера V :

$$\mathcal{V} = \{w_1, w_2, \dots, w_V\}.$$

Тогда документ d можно представить вектором:

$$x_d = (x_1, x_2, \dots, x_V),$$

где

$$x_i = \text{число вхождений слова } w_i \text{ в документ } d.$$

Например, для предложения

“the dog is on the table”

вектор BoW содержит частоты слов:

$$\text{dog} : 1, \quad \text{is} : 1, \quad \text{on} : 1, \quad \text{table} : 1, \quad \text{the} : 2.$$

Проблемы Bag-of-Words:

- теряется порядок слов;
- векторы имеют огромную размерность;
- векторы очень разреженные;
- векторы не нормализованы;
- разные формы одного слова считаются разными признаками.

Главный недостаток: BoW не хранит информацию о структуре предложения. Например, фразы

“dog bites man”

и

“man bites dog”

могут иметь одинаковое BoW-представление, хотя смысл у них разный.

1.5 Bag-of-Ngrams

Чтобы частично сохранить информацию о порядке слов, можно использовать не отдельные слова, а n -граммы.

n -грамма — последовательность из n подряд идущих токенов.

Например, для предложения

“The brown dog plays with a little cat”

биграммы:

The brown, brown dog, dog plays, plays with, with a, a little, little cat.

Но не все биграммы одинаково полезны. Некоторые несут смысловую информацию:

brown dog, dog plays, little cat.

А некоторые могут быть почти бесполезны:

with a, a little.

Значимые n -граммы часто называются **collocations**.

1.6 Collocations

Collocation — устойчивое сочетание слов, которое часто встречается вместе и имеет смысл как единая единица.

Примеры:

New York, machine learning, credit card.

Первый простой способ отобрать полезные n -граммы:

- удалить слишком частые n -граммы;
- удалить слишком редкие n -граммы.

Слишком частые n -граммы часто состоят из:

- артиклей;
- предлогов;
- вспомогательных глаголов;
- слишком общей лексики.

Слишком редкие n -граммы часто являются:

- опечатками;
- случайными комбинациями;
- сочетаниями, встретившимися один-два раза.

1.7 PMI и pPMI

Для оценки силы связи между словами можно использовать совместные частоты встречаемости в окне фиксированного размера.

Пусть:

- n_{uv} — сколько раз слова u и v встретились вместе в одном контекстном окне;
- n_u — сколько раз встретилось слово u ;
- n_v — сколько раз встретилось слово v ;
- n — общее число наблюдений.

Pointwise Mutual Information, PMI измеряет, насколько совместное появление двух слов отличается от независимого появления:

$$PMI(u, v) = \log \frac{p(u, v)}{p(u)p(v)}.$$

Через частоты это можно записать так:

$$PMI(u, v) = \log \frac{n_{uv}n}{n_u n_v}.$$

Интерпретация:

- если $PMI(u, v) > 0$, слова встречаются вместе чаще, чем ожидалось бы при независимости;

- если $PMI(u, v) = 0$, совместная встречаемость примерно соответствует независимости;
- если $PMI(u, v) < 0$, слова встречаются вместе реже, чем ожидалось бы при независимости.

На практике часто используют **Positive PMI, pPMI**:

$$pPMI(u, v) = \max(0, PMI(u, v)).$$

То есть отрицательные значения PMI заменяются нулём.

1.8 TF-IDF

TF-IDF — классический способ взвешивания слов в документе. Он учитывает не только частоту слова в документе, но и то, насколько слово редко встречается во всём корпусе.

TF-IDF состоит из двух частей:

- **TF** — term frequency;
- **IDF** — inverse document frequency.

1.8.1 Term Frequency

Term Frequency показывает, как часто термин t встречается в документе d .

Одна из простых форм:

$$tf(t, d) = f_{t,d},$$

где $f_{t,d}$ — частота термина t в документе d .

Часто используют нормированную частоту:

$$tf(t, d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}}.$$

1.8.2 Inverse Document Frequency

Inverse Document Frequency показывает, насколько термин редок во всём корпусе.

Пусть:

$$D = \{d_1, d_2, \dots, d_N\}$$

— корпус документов, где

$$N = |D|.$$

Тогда

$$idf(t, D) = \log \frac{N}{|\{d \in D : t \in d\}|}.$$

Здесь

$$|\{d \in D : t \in d\}|$$

— число документов, в которых встречается термин t .

Если слово встречается почти во всех документах, его idf мал. Если слово встречается редко, его idf велик.

1.8.3 Итоговая формула TF-IDF

$$tfidf(t, d, D) = tf(t, d) \cdot idf(t, D).$$

Смысл:

- частые слова внутри документа получают больший вес;
- слова, часто встречающиеся во всём корпусе, получают меньший вес;
- редкие и специфичные слова получают больший вес.

Например, в корпусе из двух предложений:

$A = \text{“The car is driven on the road”},$

$B = \text{“The truck is driven on the highway”}.$

Слова *the*, *is*, *driven*, *on* встречаются в обоих документах, поэтому:

$$idf = \log \frac{2}{2} = 0.$$

Значит, их TF-IDF вес будет равен нулю.

А слова *car*, *truck*, *road*, *highway* встречаются только в одном документе, поэтому:

$$idf = \log \frac{2}{1}.$$

Если используется десятичный логарифм, то:

$$\log_{10} 2 \approx 0.3.$$

Если слово встречается один раз в документе из 7 слов, то:

$$tf = \frac{1}{7}.$$

Тогда:

$$tfidf = \frac{1}{7} \cdot 0.3 \approx 0.043.$$

В Python TF-IDF можно получить через:

```
sklearn.feature_extraction.text.TfidfVectorizer.
```

1.9 One-hot encoding

One-hot encoding — способ представить слово вектором длины V , где V — размер словаря.

Каждому слову соответствует свой индекс. Вектор содержит единицу на позиции этого слова и нули на всех остальных позициях.

Например:

$$\text{Rome} = [1, 0, 0, 0, \dots, 0],$$

$$\text{Paris} = [0, 1, 0, 0, \dots, 0],$$

$$\text{Italy} = [0, 0, 1, 0, \dots, 0].$$

Проблемы one-hot encoding:

- размерность равна размеру словаря;
- векторы очень разреженные;
- нет информации о семантической близости слов.

Например, слова *Paris* и *France* семантически связаны, но их one-hot векторы ортогональны:

$$\langle x_{\text{Paris}}, x_{\text{France}} \rangle = 0.$$

То есть с точки зрения one-hot представления они так же далеки друг от друга, как любые два разных слова.

1.10 Distributional Semantics

Идея **distributional semantics** выражается фразой:

“You shall know a word by the company it keeps.”

Смысл: значение слова можно понять по его контекстам.

Если два слова часто встречаются в похожих контекстах, вероятно, они имеют похожие значения.

Например, слова *dog* и *cat* могут встречаться в похожих контекстах:

the fluffy dog barked,

the fluffy cat slept.

Значит, модели можно попытаться выучить такие векторы слов, чтобы слова с похожими контекстами имели близкие векторные представления.

1.11 Word representations через матричную факторизацию

Один из подходов к построению векторных представлений слов — использовать матрицу совместных встречаемостей.

Пусть построена матрица X , где элемент X_{ij} отражает связь слова i и контекстного слова j . Например, это может быть:

- частота совместной встречаемости;
- PMI;
- pPMI.

Тогда можно применить понижение размерности, например SVD:

$$X \approx U \Sigma V^T.$$

После этого строки одной из полученных матриц можно использовать как плотные векторы слов.

Идея:

- вход: матрица PMI или co-occurrence matrix;
- метод: dimensionality reduction, например SVD;
- выход: векторы слов, по которым можно считать сходство.

Такой подход уже позволяет получать семантически осмысленные представления, но он может быть тяжёлым по памяти и вычислениям для больших словарей.

1.12 Word Embeddings

Word embedding — плотный вещественный вектор фиксированной размерности, представляющий слово.

Вместо огромного sparse-вектора длины V каждое слово получает компактный вектор:

$$w \mapsto \mathbf{v}_w \in \mathbb{R}^d,$$

где обычно:

$$d \ll V.$$

Например, $d = 100, 200, 300$.

Хорошие embeddings обладают важным свойством: семантически близкие слова имеют близкие векторы.

Близость часто измеряют косинусной мерой:

$$\cos(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u}^\top \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|}.$$

Чем ближе значение к 1, тем более похожи направления векторов.

1.13 Семантические аналогии

Один из известных эффектов word embeddings — возможность решать аналогии через векторную арифметику.

Классический пример:

$$\mathbf{v}_{king} - \mathbf{v}_{man} + \mathbf{v}_{woman} \approx \mathbf{v}_{queen}.$$

Смысл: направление

$$\mathbf{v}_{king} - \mathbf{v}_{man}$$

может кодировать переход от мужского человека к мужскому монарху, а добавление

$$\mathbf{v}_{woman}$$

даёт женский аналог.

Другие типичные аналогии:

$$\mathbf{v}_{Paris} - \mathbf{v}_{France} + \mathbf{v}_{Italy} \approx \mathbf{v}_{Rome}.$$

То есть embeddings могут кодировать отношения:

- мужчина — женщина;
- страна — столица;
- настоящее время — прошедшее время;
- единственное число — множественное число.

1.14 Word2Vec

Word2Vec — семейство моделей для обучения word embeddings, предложенное Mikolov et al. в 2013 году.

Главная идея Word2Vec: обучать векторы слов через задачу предсказания контекста. Есть две основные архитектуры:

- Continuous Bag-of-Words, CBOW;
- Skip-gram.

1.15 Skip-gram

В модели **Skip-gram** по центральному слову предсказываются слова контекста.

Пусть есть последовательность слов:

$$w_1, w_2, \dots, w_T.$$

Для позиции t центральное слово:

$$w_t.$$

Контекстное окно размера m :

$$w_{t-m}, \dots, w_{t-1}, w_{t+1}, \dots, w_{t+m}.$$

Skip-gram оценивает вероятности:

$$P(w_{t+j} \mid w_t), \quad j \in \{-m, \dots, -1, 1, \dots, m\}.$$

То есть модель учится отвечать на вопрос:

какие слова вероятно встретятся рядом с данным словом?

Например, для фразы:

“the quick brown fox jumps over the lazy dog”

при центральном слове *brown* и окне размера 2 обучающие пары будут:

(brown, the), (brown, quick), (brown, fox), (brown, jumps).

Архитектурно Skip-gram можно представить как простую нейронную сеть:

- вход — one-hot вектор центрального слова;
- скрытый слой — embedding центрального слова;
- выход — распределение вероятностей по словам словаря.

Если словарь имеет размер V , а размер embedding равен d , то матрица входных весов имеет размер:

$$V \times d.$$

Строка этой матрицы и является вектором соответствующего слова.

1.16 Softmax в Word2Vec

Для слова w_o из контекста вероятность при центральном слове w_c можно записать через softmax:

$$P(w_o \mid w_c) = \frac{\exp(\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c})}{\sum_{w \in \mathcal{V}} \exp(\mathbf{u}_w^\top \mathbf{v}_{w_c})}.$$

Здесь:

- $\mathbf{v}_{w_c}$ — embedding центрального слова;
- $\mathbf{u}_{w_o}$ — выходной embedding контекстного слова;
- $\mathcal{V}$ — словарь.

Проблема: сумма в знаменателе идёт по всему словарю. Если словарь большой, вычисление softmax становится дорогим.

Например, при:

$$V = 10\,000, \quad d = 300$$

матрица весов имеет:

$$10\,000 \times 300 = 3\,000\,000$$

параметров.

На больших корпусах обучение становится слишком долгим и вычислительно дорогим.

1.17 Как ускоряют Word2Vec

В Word2Vec используются несколько практических приёмов:

1. Частые словосочетания можно рассматривать как отдельные токены.
2. Частые слова можно subsample-ить, то есть случайно удалять из обучающих примеров.
3. Вместо полного softmax можно использовать negative sampling.

1.18 Subsampling frequent words

Очень частые слова, например *the*, *is*, *of*, встречаются почти везде и часто дают мало полезной семантической информации.

Поэтому их можно случайно удалять из обучающих данных.

Пусть:

$$w_i$$

— слово, а

$$z(w_i)$$

— доля этого слова в корпусе.

Вероятность сохранить слово задаётся формулой:

$$P(w_i) = \left(\sqrt{\frac{z(w_i)}{0.001}} + 1 \right) \frac{0.001}{z(w_i)}.$$

Смысл формулы:

- очень частые слова сохраняются с меньшей вероятностью;

- редкие слова почти не удаляются;
- уменьшается число обучающих примеров;
- модель меньше фокусируется на малоинформативных словах.

1.19 Negative Sampling

Negative Sampling — способ заменить дорогой softmax более дешёвой задачей бинарной классификации.

Идея:

- для настоящей пары слов (w_c, w_o) модель должна предсказать, что это положительный пример;
- дополнительно выбираются несколько случайных слов, которые считаются отрицательными примерами;
- обновляются веса только для небольшого числа слов, а не для всего словаря.

То есть вместо того, чтобы считать ошибку для всех слов словаря, мы считаем её только для:

- одного положительного слова;
- нескольких отрицательных слов.

Остальные слова получают нулевую ошибку и не обновляются на этом шаге.

Отрицательные примеры выбираются не равномерно, а с учётом частоты слов.

Вероятность выбрать слово w_i как negative sample:

$$P(w_i) = \frac{f(w_i)^{3/4}}{\sum_{j=1}^V f(w_j)^{3/4}}.$$

Здесь $f(w_i)$ — частота слова.

Степень $3/4$ сглаживает распределение:

- частые слова выбираются чаще;
- но не настолько часто, как при прямом использовании частот.

1.20 CBOW

Continuous Bag-of-Words, CBOW решает обратную задачу по сравнению со Skip-gram.

В CBOW по словам контекста предсказывается центральное слово.

Формально модель оценивает:

$$P(w_i \mid w_{i-h}, \dots, w_{i-1}, w_{i+1}, \dots, w_{i+h}).$$

Контекст рассматривается как “мешок слов”, то есть порядок слов в контексте обычно не учитывается.

Например, если центральное слово — *banking*, а контекст:

problems, turning, crises, as,

то CBOW пытается предсказать:

banking.

Особенности CBOW:

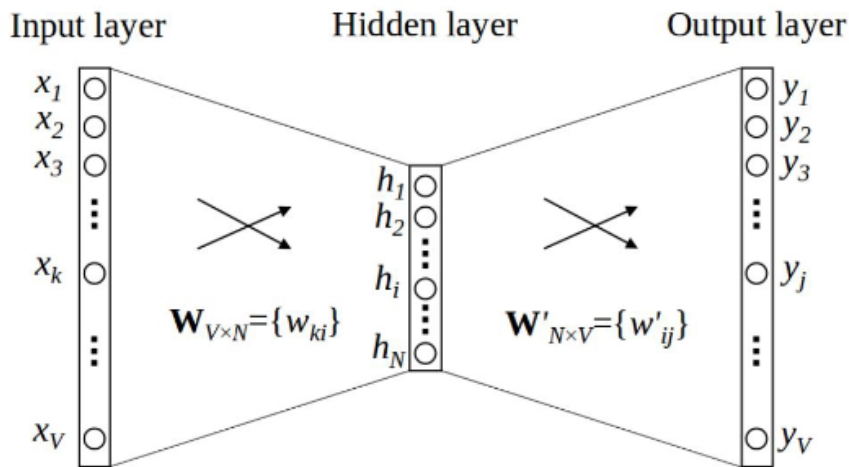
- предсказывает одно слово за раз;
- обычно обучается быстрее Skip-gram;
- хорошо работает для частых слов.

1.21 Сравнение CBOW и Skip-gram

CBOW	Skip-gram
Предсказывает центральное слово по контексту.	Предсказывает слова контекста по центральному слову.
$P(w_i \mid w_{i-h}, \dots, w_{i+h})$	$P(w_{i-h}, \dots, w_{i+h} \mid w_i)$
Быстрее обучается.	Обучается медленнее.
Лучше для частых слов.	Лучше работает с редкими словами.
Контекст агрегируется как bag-of-words.	Каждое контекстное слово предсказывается отдельно.

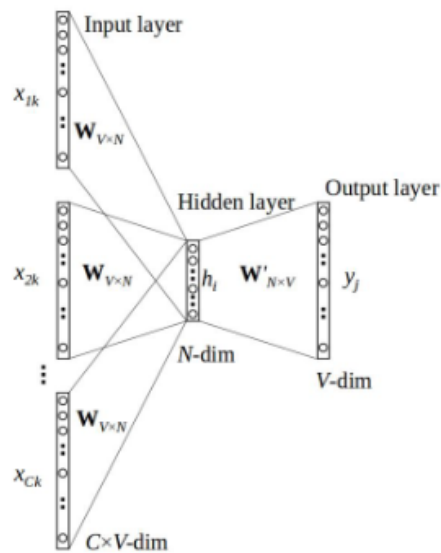
Table 1: Сравнение CBOW и Skip-gram.

Skip-gram



55

CBOW



56

Figure 1: Две основные архитектуры Word2Vec: CBOW и Skip-gram.

1.22 GloVe

GloVe расшифровывается как **Global Vectors for Word Representation**.

Идея GloVe находится между двумя подходами:

- count-based методами, которые используют статистику совместных встречаемостей;
- predictive методами, такими как Word2Vec.

Word2Vec обучается предсказывать близость слов через контекстные события.

GloVe обучается предсказывать статистики совместной встречаемости слов.

Пусть:

$$X_{ij}$$

— количество раз, когда слово j встретилось в контексте слова i .

GloVe минимизирует функцию потерь:

$$J = \sum_{i,j=1}^V f(X_{ij}) \left(\mathbf{w}_i^\top \tilde{\mathbf{w}}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2.$$

Здесь:

- $\mathbf{w}_i$ — обучаемый embedding слова i ;
- $\tilde{\mathbf{w}}_j$ — обучаемый embedding контекстного слова j ;
- $b_i, \tilde{b}_j$ — обучаемые bias-параметры;
- X_{ij} — co-occurrence count;
- $f(X_{ij})$ — весовая функция.

Смысл функции потерь:

$$\mathbf{w}_i^\top \tilde{\mathbf{w}}_j + b_i + \tilde{b}_j$$

должно приближать:

$$\log X_{ij}.$$

То есть скалярное произведение embeddings должно кодировать логарифм частоты совместной встречаемости.

Весовая функция $f(X_{ij})$ нужна, чтобы:

- уменьшать влияние слишком редких совместных встречаемостей;
- регуляризовать влияние слишком частых пар;
- сделать обучение устойчивее.

1.23 Сравнение подходов к представлению текста

Метод	Идея	Проблемы / особенности
One-hot	Каждое слово — отдельный sparse-вектор.	Нет семантической близости, огромная размерность.
Bag-of-Words	Документ — вектор частот слов.	Теряется порядок слов, sparse-векторы.
Bag-of-Ngrams	Используются последовательности из n слов.	Размерность быстро растёт.
TF-IDF	Слова взвешиваются по важности для документа и редкости в корпусе.	Хорошо для классических моделей, но не даёт плотной семантики.
PMI / pPMI	Используются статистики совместной встречаемости слов.	Можно получать семантическую близость, но матрицы большие.
Word2Vec	Векторы обучаются через предсказание контекста.	Эффективные dense embeddings, нужны оптимизации.
GloVe	Векторы обучаются предсказывать co-occurrence statistics.	Соединяет count-based и neural-подходы.

Table 2: Основные способы представления текста и слов.

1.24 Главные выводы

- Текст нельзя напрямую подать в классическую ML-модель: его нужно преобразовать в числовые признаки.
- Bag-of-Words — простой baseline, но он теряет порядок слов и даёт sparse-векторы.
- TF-IDF улучшает BoW, понижая вес слишком общих слов и повышая вес редких информативных слов.
- One-hot encoding представляет слова как независимые категории и не кодирует семантическое сходство.
- Distributional semantics говорит: значение слова определяется его контекстом.
- Word embeddings — плотные векторы слов, в которых похожие слова должны иметь близкие представления.

- Word2Vec обучает embeddings через задачу предсказания контекста.
- В Skip-gram центральное слово используется для предсказания контекста.
- В CBOW контекст используется для предсказания центрального слова.
- Negative Sampling ускоряет обучение Word2Vec, обновляя только малую часть весов.
- GloVe обучает embeddings так, чтобы их скалярные произведения приближали статистики совместной встречаемости.

1.25 Что важно помнить к экзамену

1. Уметь объяснить общий pipeline text classification:

text $\rightarrow$ features $\rightarrow$ classifier $\rightarrow$ label.

2. Понимать разницу между stemming и lemmatization.

3. Знать проблемы Bag-of-Words:

- нет порядка слов;
- огромная размерность;
- sparse-векторы;
- разные формы слов считаются разными словами.

4. Уметь записать формулу TF-IDF:

$$tfidf(t, d, D) = tf(t, d) \cdot \log \frac{N}{|\{d \in D : t \in d\}|}.$$

5. Понимать, почему one-hot vectors не кодируют смысл.

6. Уметь объяснить distributional hypothesis: слова с похожими контекстами имеют похожие значения.

7. Знать формулу PMI:

$$PMI(u, v) = \log \frac{p(u, v)}{p(u)p(v)}.$$

8. Знать, что такое pPMI:

$$pPMI(u, v) = \max(0, PMI(u, v)).$$

9. Уметь объяснить Word2Vec, CBOW и Skip-gram.
10. Понимать, зачем нужен Negative Sampling.
11. Знать основную идею GloVe: embeddings обучаются предсказывать логарифмы co-occurrence counts.

““

“‘latex

2 Лекция 2. CNN for Texts and Embeddings for Different Languages

2.1 План лекции

В этой лекции рассматриваются три основные темы:

- повторение Word2Vec и embeddings;
- embeddings для разных языков и задача перевода без параллельных данных;
- применение сверточных нейронных сетей к текстам.

Основная идея лекции: векторы слов можно использовать не только внутри одного языка, но и для сопоставления разных языков, а CNN можно применять к тексту как к последовательности локальных фрагментов, похожих на n -граммы.

2.2 Повторение Word2Vec

Word2Vec обучает плотные векторные представления слов через задачу предсказания контекста.

В модели Skip-gram центральное слово используется для предсказания соседних слов. Если есть предложение

The quick brown fox jumps over the lazy dog,

то для центрального слова *brown* и окна размера 2 можно получить обучающие пары:

(brown, the), (brown, quick), (brown, fox), (brown, jumps).

Идея состоит в том, что слова с похожими контекстами получают похожие векторные представления.

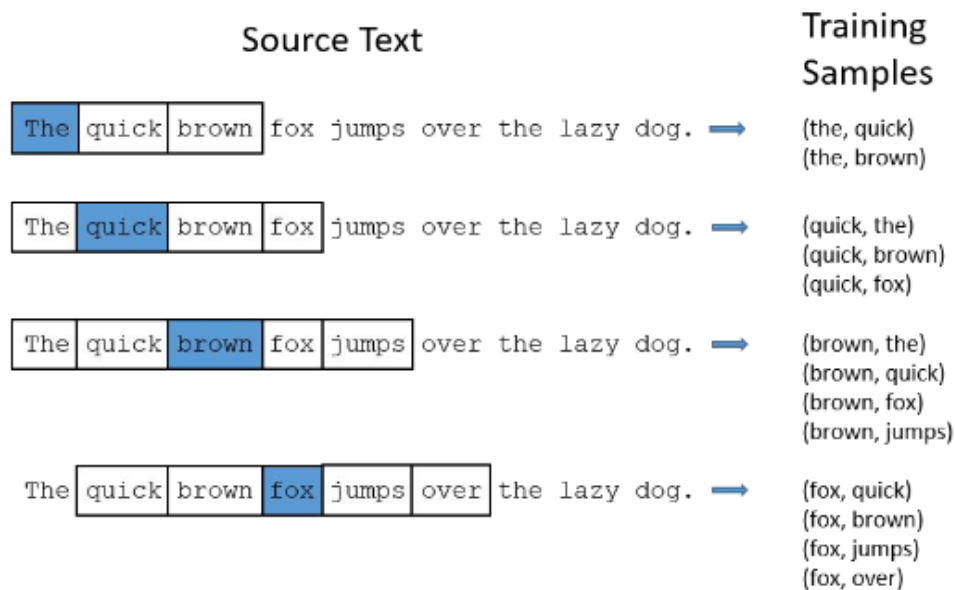


Figure 2: Формирование обучающих пар в Skip-gram. Слайд 3.

Архитектурно Word2Vec можно представить как нейронную сеть:

- вход — one-hot вектор слова;
- скрытый слой — плотное представление слова;
- выход — распределение вероятностей по словам словаря.

Если размер словаря равен V , а размерность embedding равна d , то матрица входных весов имеет размер

$$V \times d.$$

Строки этой матрицы после обучения становятся векторами слов.

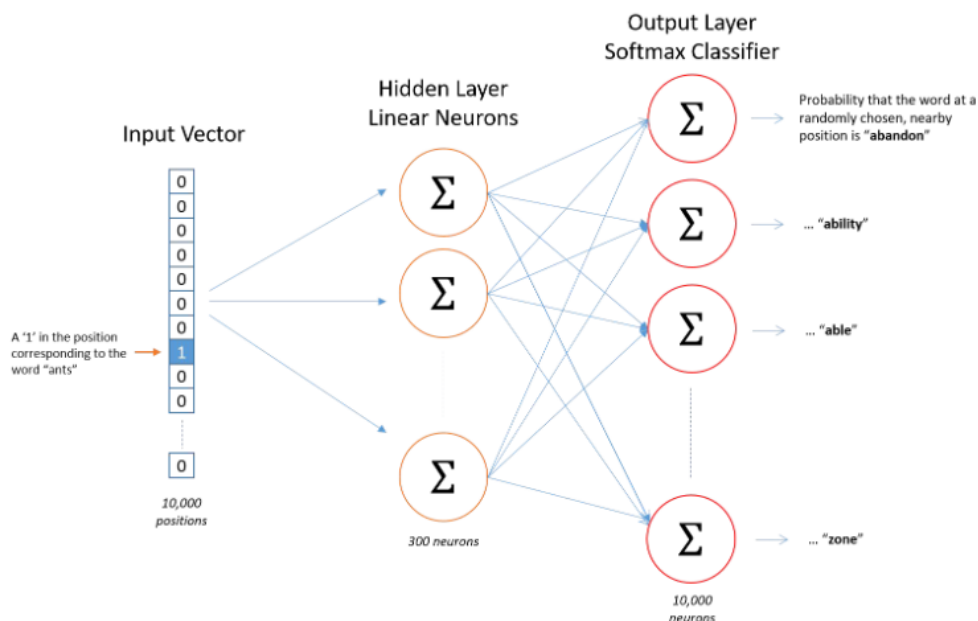


Figure 3: Word2Vec как нейронная сеть: one-hot вход, скрытый слой и softmax-выход. Слайд 4.

Матрицу весов скрытого слоя можно интерпретировать как lookup table:

$$w_i \mapsto \mathbf{v}_{w_i}.$$

То есть каждому слову сопоставляется строка матрицы — его embedding.

2.3 Embeddings для разных языков

Оказывается, что embedding-пространства разных языков часто имеют похожую геометрическую структуру.

Например, в одном языке могут быть слова:

one, two, three, four, five,

а в другом языке их переводы:

uno, dos, tres, cuatro, cinco.

Если обучить embeddings отдельно для каждого языка, относительное расположение таких слов может оказаться похожим. Аналогично могут сохраняться отношения между словами:

dog, cat, cow, horse

и их переводами в другом языке.

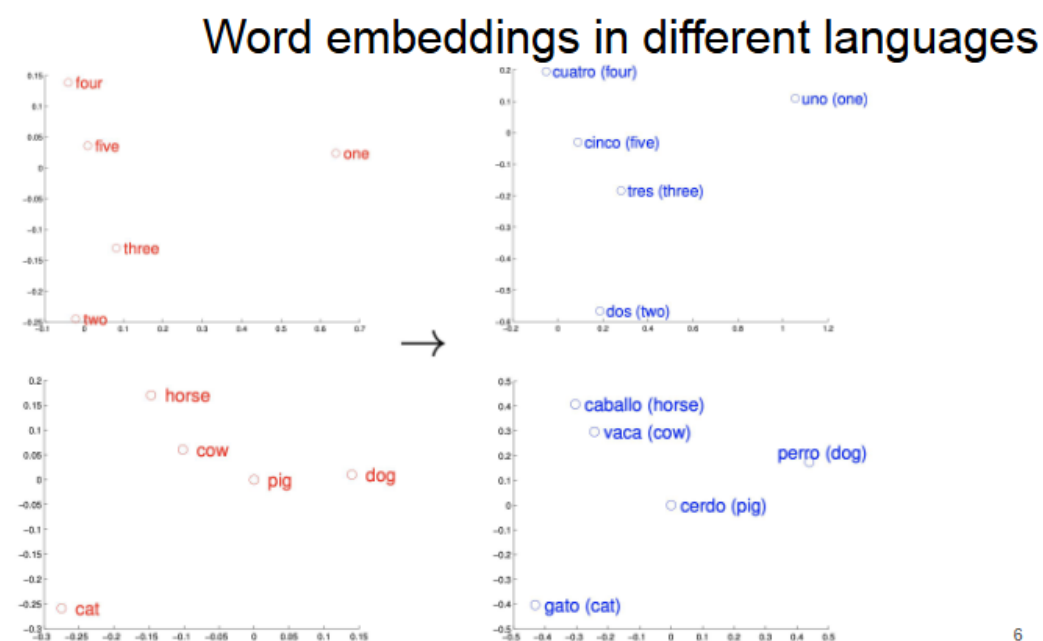


Figure 4: Похожая структура embedding-пространств для разных языков. Слайд 6.

Это позволяет решать задачу перевода слов через отображение одного embedding-пространства в другое.

2.4 Линейное отображение между языками

Пусть есть два языка:

- source language — исходный язык;
- target language — целевой язык.

Для некоторого набора слов известны пары переводов:

$$\{(x_i, y_i)\}_{i=1}^n,$$

где

$$x_i \in \mathbb{R}^d$$

— embedding слова из исходного языка, а

$$y_i \in \mathbb{R}^d$$

— embedding его перевода в целевом языке.

Например, можно взять

$$n = 5000$$

пар слов-переводов.

Идея: выучить линейное отображение

$$W$$

из source embedding space в target embedding space.

Если собрать векторы исходного языка в матрицу

$$X \in \mathbb{R}^{d \times n},$$

а векторы целевого языка в матрицу

$$Y \in \mathbb{R}^{d \times n},$$

то можно искать матрицу W , минимизирующую расстояние между отображёнными исходными embeddings и целевыми embeddings:

$$W^* = \arg \min_{W \in M_d(\mathbb{R})} \|WX - Y\|_F.$$

Здесь

$$\|\cdot\|_F$$

— норма Фробениуса.

После обучения матрицы W перевод слова s можно искать так:

$$t = \arg \max_t \cos(Wx_s, y_t).$$

То есть embedding исходного слова x_s сначала отображается в пространство целевого языка, а затем среди всех слов целевого языка выбирается ближайший вектор по cosine similarity.

2.5 Ортогональное отображение

На практике часто накладывают ограничение ортогональности на матрицу W :

$$W \in O_d(\mathbb{R}).$$

Это означает:

$$W^\top W = I.$$

Ортогональное преобразование сохраняет длины и углы между векторами. Это полезно, потому что при переводе между языками мы хотим не произвольно деформировать

пространство, а в основном повернуть или отразить его так, чтобы оно совпало с другим языком.

Задача принимает вид:

$$W^* = \arg \min_{W \in O_d(\mathbb{R})} \|WX - Y\|_F.$$

Решение можно выразить через SVD:

$$U\Sigma V^\top = \text{SVD}(YX^\top),$$

$$W^* = UV^\top.$$

После этого перевод снова ищется по ближайшему вектору:

$$t = \arg \max_t \cos(Wx_s, y_t).$$

2.6 Unsupervised mapping между языками

В презентации отмечается, что отображение между двумя языками может быть построено и полностью unsupervised способом, например с использованием GAN.

Идея в общих чертах:

- есть два набора embeddings X и Y ;
- нет известных пар переводов;
- модель пытается найти такое отображение W , чтобы распределение WX стало похоже на распределение Y ;
- после грубого выравнивания можно уточнять соответствия между словами.

Это важная идея для cross-lingual NLP: можно сопоставлять языки даже без параллельного корпуса и без большого словаря переводов.

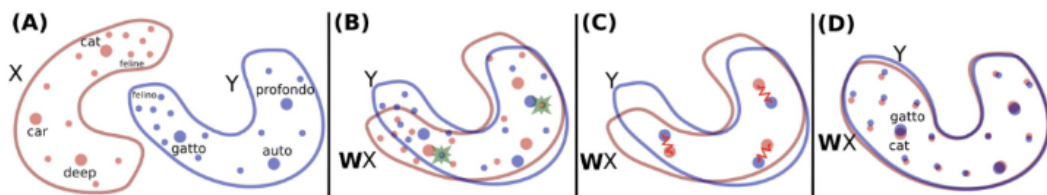


Figure 5: Интуиция unsupervised mapping между embedding-пространствами двух языков. Слайд 9.

2.7 Cosine similarity

Для сравнения векторов часто используют cosine similarity:

$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|}.$$

В координатной форме:

$$\cos(\theta) = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}.$$

Cosine similarity измеряет не абсолютное расстояние между концами векторов, а угол между ними.

Это полезно для текстовых векторов, потому что длина вектора может сильно зависеть от частоты слова или длины документа. Например, в count-based подходах длинные документы могут иметь большие нормы просто потому, что в них больше слов.

Для word embeddings cosine similarity также часто оказывается удобной, потому что направление embedding-вектора обычно важнее его нормы.

2.8 Норма embedding-вектора и частота слова

В презентации показано, что норма embedding-вектора может зависеть от частоты слова в корпусе.

Более частые слова могут получать embeddings с большей евклидовой нормой. Поэтому если сравнивать векторы обычным скалярным произведением, частотность слова может сильно влиять на результат.

Именно поэтому часто используют cosine similarity:

$$\cos(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u}^\top \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|}.$$

Нормировка убирает влияние длины векторов и оставляет в основном направление.

2.9 Как получить embedding для текста

Word embeddings дают представления отдельных слов. Но во многих задачах нужен embedding всего текста:

$$[w_1, w_2, \dots, w_n] \mapsto \mathbf{h}_{text}.$$

Один простой способ — усреднить embeddings слов:

$$\mathbf{h}_{text} = \frac{1}{n} \sum_{i=1}^n \mathbf{v}_{w_i}.$$

Но такой подход теряет порядок слов.

Другой способ — использовать модели последовательностей, например RNN, LSTM или GRU.

2.10 Повторение RNN

Recurrent Neural Network, RNN обрабатывает последовательность слева направо.

На каждом шаге она получает текущий вход x_t и предыдущее скрытое состояние h_{t-1} :

$$h_t = f(W_x x_t + W_h h_{t-1} + b).$$

Для vanilla RNN часто используется:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b).$$

Идея: скрытое состояние h_t должно содержать информацию о префиксе последовательности:

$$x_1, x_2, \dots, x_t.$$

Тогда финальное состояние h_n можно использовать как embedding всей последовательности.

Проблема RNN: финальный вектор может слишком сильно зависеть от последних слов и хуже хранить информацию о локальных фразах в середине текста.

Кроме того, обычные RNN сталкиваются с проблемой vanishing gradient. Поэтому на практике часто используют:

- LSTM;
- GRU;
- skip connections;
- gradient clipping.

2.11 Переход от RNN к CNN

RNN естественно обрабатывает текст как последовательность. Но у неё есть недостаток: чтобы получить вектор для фразы, модель учитывает префиксный контекст.

Например, в фразе

the country of my birth

RNN будет постепенно накапливать информацию:

the → the country → the country of → the country of my → the country of my birth.

Но если нам нужны локальные фразы, например

country of, my birth, country of my birth,

RNN не всегда даёт удобный независимый вектор для каждой такой фразы.

Идея CNN для текста: вычислять признаки для всех локальных фрагментов текста.

2.12 n -граммы и CNN

В прошлой лекции рассматривались n -граммы. Например, для предложения

The brown dog plays with a little cat

биграммы:

The brown, brown dog, dog plays, plays with, with a, a little, little cat.

CNN для текста можно понимать как обобщение идеи n -грамм.

Фильтр размера h смотрит на окно из h соседних слов:

$$x_{i:i+h-1}.$$

Например:

- фильтр размера 1 похож на unigram-признак;
- фильтр размера 2 похож на bigram-признак;
- фильтр размера 3 похож на trigram-признак.

Отличие от обычных n -грамм: CNN не просто считает частоты конкретных фраз, а обучает фильтры, которые могут находить полезные паттерны в embedding-пространстве.

2.13 Свёртка по тексту

Пусть текст представлен матрицей embeddings:

$$X \in \mathbb{R}^{n \times d},$$

где:

- n — длина текста;
- d — размерность embedding;
- строка X_i — embedding i -го слова.

Фильтр размера h применяется к окну:

$$X_{i:i+h-1}.$$

На каждом окне вычисляется признак:

$$c_i = f(W^\top X_{i:i+h-1} + b).$$

Здесь:

- W — параметры фильтра;
- b — bias;
- f — нелинейная функция активации;
- c_i — значение feature map в позиции i .

Если фильтр применяется ко всем возможным позициям, получается feature map:

$$\mathbf{c} = [c_1, c_2, \dots, c_{n-h+1}] \in \mathbb{R}^{n-h+1}.$$

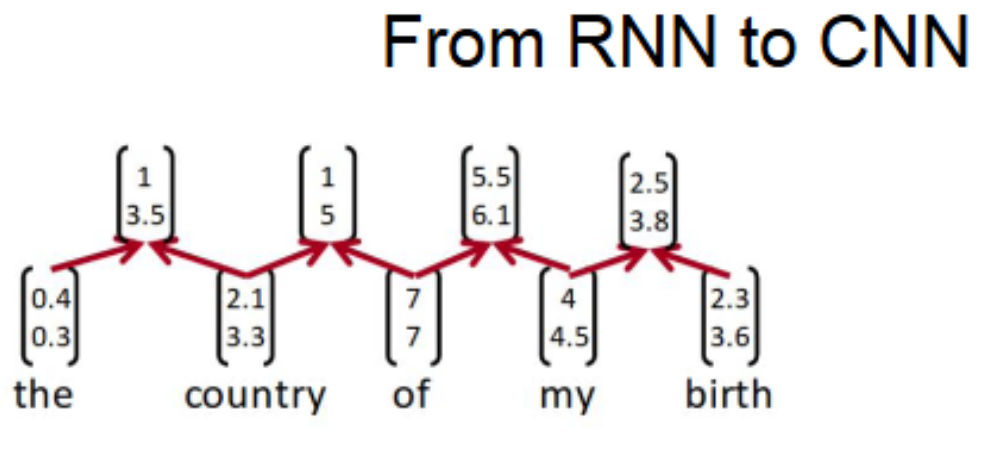


Figure 6: Операция над каждой парой соседних слов может интерпретироваться как свёртка по word vectors. Слайд 23.

2.14 One-layer CNN для текста

Один слой CNN для текста обычно состоит из:

1. embedding layer;
2. convolutional layer;
3. pooling layer;
4. fully connected layer;
5. output layer.

Свёртка строит feature map:

$$\mathbf{c} = [c_1, c_2, \dots, c_{n-h+1}].$$

Затем применяется pooling. Самый распространённый вариант для классификации текста — max-over-time pooling:

$$\hat{c} = \max \mathbf{c}.$$

Это означает, что из всей feature map берётся максимальное значение.

Интерпретация:

- фильтр ищет некоторый паттерн в тексте;
- max pooling отвечает на вопрос: встретился ли этот паттерн где-нибудь в тексте;
- позиция паттерна становится неважной.

После pooling длина текста уже не имеет значения. Поэтому можно использовать тексты разной длины и фильтры разных размеров:

$$h = 1, 2, 3, 4, \dots$$

То есть модель может одновременно искать unigram-, bigram-, trigram- и более длинные паттерны.

2.15 CNN architecture from Kim 2014

Классическая архитектура CNN для sentence classification была предложена Kim в 2014 году.

Общая схема:

- предложение представляется как матрица word embeddings;
- применяются несколько convolutional filters разных ширин;
- для каждого фильтра строится feature map;
- по каждой feature map применяется max-over-time pooling;
- полученные признаки объединяются;
- затем идут dropout, fully connected layer и softmax.

Если использовать фильтры разных размеров, модель может находить признаки на уровне разных n -грамм.

Например:

$$h = 2$$

ищет bigram-like паттерны,

$$h = 3$$

ищет trigram-like паттерны,

$$h = 4$$

ищет более длинные фразы.

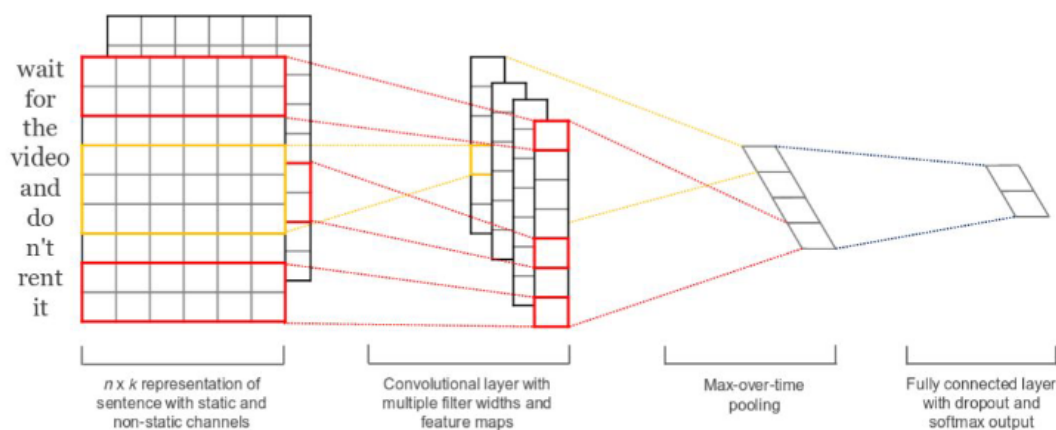


Figure 7: CNN для классификации предложений из статьи Kim, 2014. Слайд 26.

2.16 Static и non-static embeddings

В CNN для текстов embeddings могут использоваться по-разному.

Static embeddings:

- берутся заранее обученные embeddings;
- во время обучения модели они не обновляются;
- полезно, если мало обучающих данных.

Non-static embeddings:

- embeddings инициализируются предобученными векторами;
- затем дообучаются вместе с основной моделью;
- могут лучше адаптироваться к конкретной задаче.

Multichannel embeddings:

- используется несколько каналов embeddings;
- например, один канал static, другой non-static;
- аналогия с каналами изображения в CNN.

2.17 Narrow и wide convolution

В CNN для текста можно различать narrow и wide convolution.

Narrow convolution применяется только там, где фильтр полностью помещается внутрь последовательности. Для длины текста n и ширины фильтра h длина feature map равна:

$$n - h + 1.$$

Wide convolution использует zero-padding, то есть добавляет нулевые векторы по краям последовательности. Это позволяет фильтру применяться также к позициям около границ текста.

Padding полезен, потому что:

- крайние слова тоже получают полноценную обработку;
- можно управлять размером выходной feature map;
- можно строить более глубокие CNN без слишком быстрого уменьшения длины последовательности.

Также можно использовать stride:

$$s > 1.$$

Stride означает, что фильтр сдвигается не на одну позицию, а на несколько.

2.18 Применения CNN в NLP

CNN использовались в разных NLP-задачах:

- классификация предложений;
- sentiment analysis;
- question classification;
- neural machine translation;
- построение sentence embeddings.

В ранних моделях neural machine translation CNN могли использоваться как encoder, а RNN — как decoder.

То есть CNN сжимает исходное предложение в представление, а RNN затем генерирует перевод.

2.19 Сравнение подходов

В презентации приведена таблица результатов разных моделей на NLP-бенчмарках. Основная идея таблицы: CNN-based модели оказались конкурентоспособными с другими подходами для классификации текста.

Сравнивались варианты:

- CNN-rand;
- CNN-static;
- CNN-non-static;
- CNN-multichannel;
- RAE;
- MV-RNN;
- RNTN;
- DCNN;
- Paragraph-Vec;
- SVM и другие baseline-модели.

Важный практический вывод: даже относительно простая CNN-архитектура с предобученными embeddings может давать сильные результаты в задачах классификации текста.

2.20 Outro and tips

Главные практические замечания из лекции:

- Vanishing gradient встречается не только в RNN, но и в глубоких нейронных сетях вообще.
- Для борьбы с vanishing gradient используют memory mechanisms, skip connections и другие архитектурные приёмы.
- LSTM и GRU хорошо подходят для последовательностей.
- GRU обычно быстрее, а LSTM может лучше ловить сложные зависимости.
- Полезно использовать gradient clipping.
- CNN для текстов похожи на обучаемый n -gram trick.
- CNN эффективны, когда можно параллельно выполнять много вычислений.
- RNN и CNN можно комбинировать.

2.21 Attention как следующая тема

В конце лекции упоминается attention.

В машинном переводе attention позволяет модели явно сопоставлять слова исходного предложения со словами целевого предложения.

Визуально attention часто изображают как матрицу alignment:

$$A_{ij},$$

где A_{ij} показывает, насколько сильно i -е слово перевода связано с j -м словом исходного предложения.

В примере из Bahdanau et al. attention показывает почти диагональную структуру соответствий между английским и французским предложениями, но также позволяет учитывать перестановки слов.

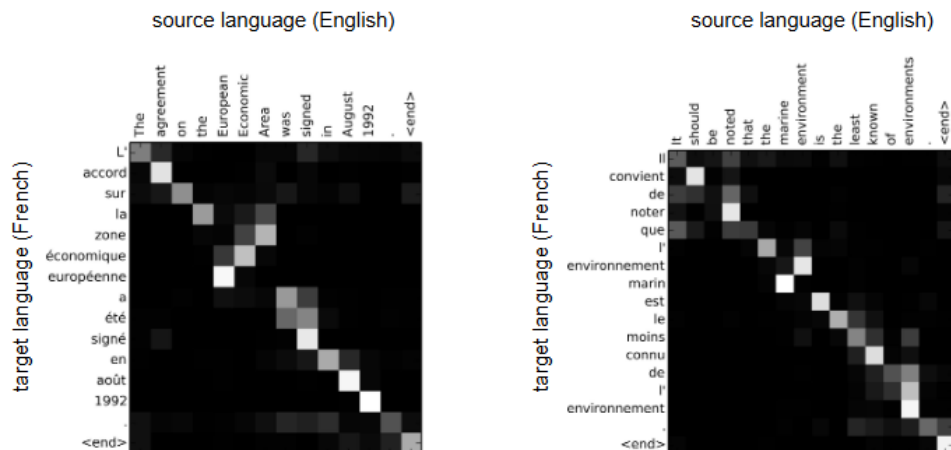


Figure 8: Attention alignment в neural machine translation. Слайд 31.

Эта тема важна, потому что attention затем станет основой архитектуры Transformer.

2.22 Проблема Word2Vec

В финале презентации подчёркивается ограничение Word2Vec:

Word2Vec embeddings capture only local context.

То есть Word2Vec обучается на локальных окнах вокруг слова. Поэтому он хорошо отражает локальную совместную встречаемость, но не всегда достаточно хорошо учитывает:

- дальние зависимости;
- структуру всего предложения;
- разные значения одного слова в разных контекстах;
- глобальный смысл текста.

Это мотивирует переход к более сложным моделям контекстуальных представлений: RNN, CNN, attention и Transformer.

2.23 Главные выводы

- Word2Vec строит embeddings слов через задачу предсказания контекста.
- Матрица весов скрытого слоя Word2Vec фактически является таблицей embeddings.

- Embedding-пространства разных языков часто имеют похожую структуру.
- Перевод слов можно делать через линейное отображение между embedding-пространствами.
- Ортогональное отображение сохраняет углы и длины, поэтому хорошо подходит для cross-lingual mapping.
- Cosine similarity удобна для сравнения embeddings, потому что фокусируется на направлении векторов, а не на их длине.
- RNN строит представление текста последовательно, но может плохо выделять локальные фразы.
- CNN для текста можно понимать как обучаемые n -gram detectors.
- Свёрточный фильтр проходит по окнам слов и строит feature map.
- Max-over-time pooling делает длину текста несущественной и фиксирует наличие паттерна в тексте.
- CNN с предобученными embeddings является сильным baseline для классификации текста.
- Attention решает задачу явного сопоставления частей входной и выходной последовательности.

2.24 Что важно помнить к экзамену

1. Уметь объяснить, почему embedding-пространства разных языков можно сопоставлять.
2. Знать задачу линейного mapping:

$$W^* = \arg \min_W \|WX - Y\|_F.$$

3. Знать ортогональный вариант:

$$W^* = \arg \min_{W \in O_d(\mathbb{R})} \|WX - Y\|_F.$$

4. Помнить решение через SVD:

$$U\Sigma V^\top = SVD(YX^\top), \quad W^* = UV^\top.$$

5. Уметь записать cosine similarity:

$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|}.$$

6. Понимать, почему cosine similarity лучше обычного dot product для embeddings.
7. Уметь объяснить, как CNN применяется к тексту:

$$c_i = f(W^\top X_{i:i+h-1} + b).$$

8. Понимать, что фильтр размера h похож на обучаемый h -gram detector.
9. Знать max-over-time pooling:

$$\hat{c} = \max \mathbf{c}.$$

10. Уметь сравнить RNN и CNN для текста:

- RNN хорошо моделирует последовательность;
- CNN хорошо ловит локальные паттерны;
- CNN легче параллелить.

11. Помнить ограничение Word2Vec: он в основном учитывает локальный контекст.

“

“\texttt{latex}

3 Лекция 3. Machine Translation and Attention Mechanism

3.1 План лекции

В этой лекции рассматриваются:

- исторический обзор машинного перевода;
- statistical machine translation;
- word alignment;
- phrase-based machine translation;
- neural machine translation;
- архитектура seq2seq;
- greedy decoding и beam search;
- оценка качества машинного перевода;

- attention mechanism.

Главная идея лекции: машинный перевод прошёл путь от rule-based и statistical machine translation к neural machine translation. В нейронном машинном переводе важнейшей архитектурой стала encoder-decoder схема, а attention был введён как способ устранить bottleneck фиксированного вектора, в который encoder должен был сжать всё исходное предложение.

3.2 Исторический обзор машинного перевода

Машинный перевод — одна из старейших задач NLP. Уже в 1950-е годы появились первые эксперименты по автоматическому переводу.

Один из первых известных экспериментов — Georgetown experiment, проведённый 7 января 1954 года. В нём выполнялся автоматический перевод с русского на английский:

- переводилось 60 предложений;
- использовался словарь примерно из 250 словарных статей;
- использовалось 6 грамматических правил;
- вычисления выполнялись на IBM 701.

В СССР в 1954 году также проводились эксперименты по машинному переводу на БЭСМ. Эти ранние системы были rule-based: они опирались на словари и вручную заданные правила.

Дальше развитие машинного перевода можно грубо разделить на несколько этапов:

- rule-based machine translation;
- expert systems;
- statistical machine translation;
- neural machine translation.

3.3 Обучающие данные для машинного перевода

Для обучения систем машинного перевода обычно используются **parallel corpora**.

Parallel corpus — корпус, состоящий из пар:

$\langle \text{source}, \text{translation} \rangle$.

Обычно это пары предложений:

$$(x, y),$$

где:

- x — предложение на исходном языке;
- y — его перевод на целевой язык.

Иногда параллельные данные могут быть выровнены на уровне абзацев или документов, но чаще используется sentence-level alignment.

Параллельные корпуса могут быть:

- вручную подготовлены переводчиками;
- собраны из веба;
- сгенерированы синтетически.

3.4 Statistical Machine Translation

Statistical Machine Translation, SMT — подход к машинному переводу, доминировавший примерно в 1990–2010 годы.

Пусть есть предложение x на исходном языке. Нужно найти наиболее вероятный перевод y на целевом языке:

$$\hat{y} = \arg \max_y P(y \mid x).$$

С помощью правила Байеса:

$$P(y \mid x) = \frac{P(x \mid y)P(y)}{P(x)}.$$

Так как x фиксировано, знаменатель $P(x)$ не влияет на выбор максимума:

$$\hat{y} = \arg \max_y P(x \mid y)P(y).$$

Здесь появляются две основные компоненты SMT:

- **translation model** $P(x \mid y)$ — оценивает, насколько хорошо предложение y переводится в x ;
- **language model** $P(y)$ — оценивает, насколько предложение y является естественным предложением на целевом языке.

Иными словами:

translation model отвечает за adequacy,

language model отвечает за fluency.

Adequacy означает, что перевод сохраняет смысл исходного предложения. **Fluency** означает, что перевод звучит естественно на целевом языке.

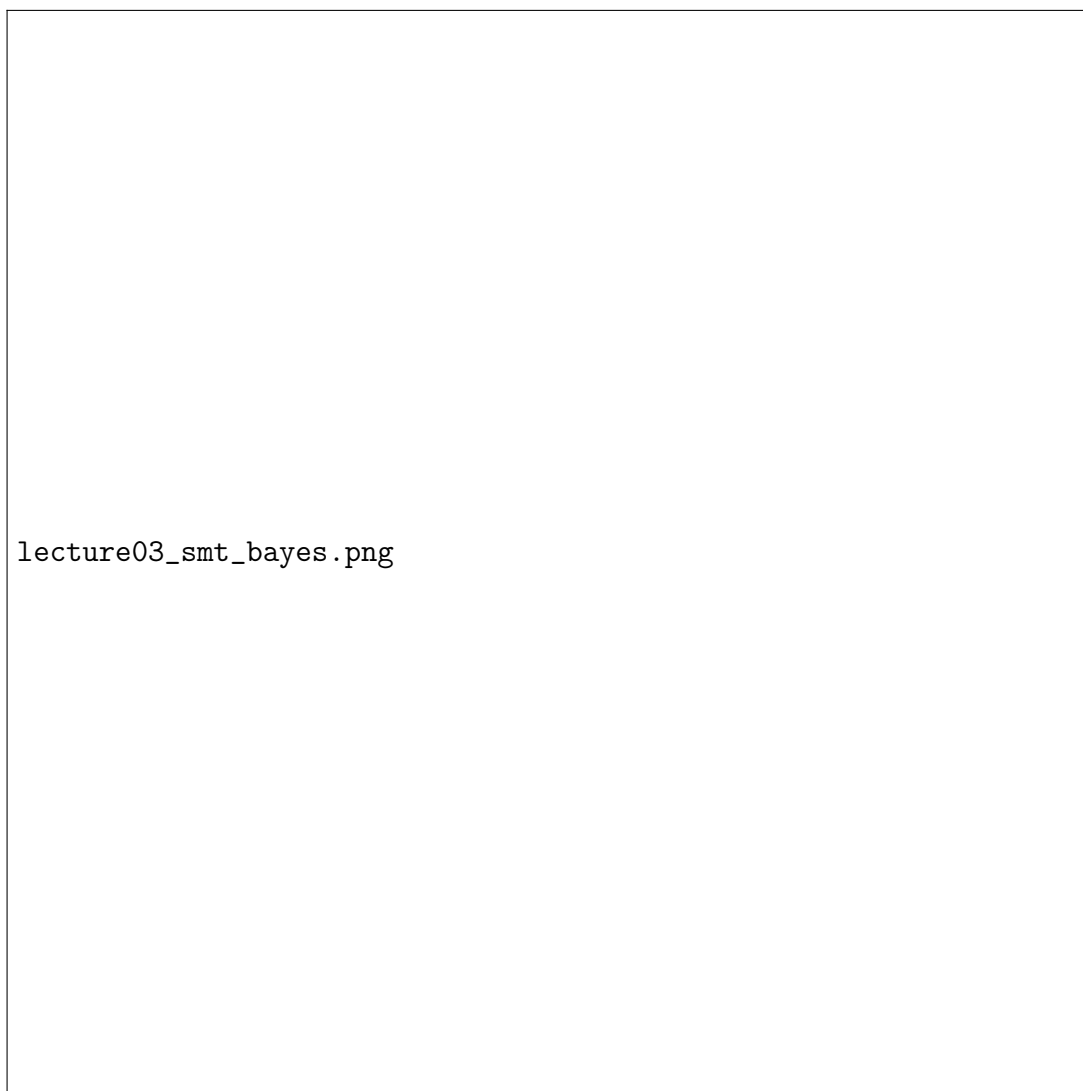


Figure 9: Statistical Machine Translation: разложение задачи перевода на translation model и language model.

3.5 Word Alignment

Чтобы обучить translation model из параллельного корпуса, нужно понять, какие слова исходного предложения соответствуют каким словам перевода.

Alignment — соответствие между словами исходного предложения и словами целевого предложения.

Пусть:

$$x = (x_1, x_2, \dots, x_m)$$

— предложение на исходном языке, а

$$y = (y_1, y_2, \dots, y_n)$$

— предложение на целевом языке.

Alignment задаёт связи между словами x_i и y_j .

В простейшем случае слово соответствует одному слову:

$$x_i \leftrightarrow y_j.$$

Но в реальных языках всё сложнее.

3.5.1 Many-to-one alignment

Несколько слов одного языка могут соответствовать одному слову другого языка.

Например, английское словосочетание может переводиться одним словом во французском или наоборот.

Это называется:

many-to-one alignment.

3.5.2 One-to-many alignment

Одно слово одного языка может соответствовать нескольким словам другого языка.

В презентации это связывается с понятием **fertility**.

Fertility слова — количество слов в переводе, которым соответствует данное слово.

Если одно слово порождает много слов в переводе, оно называется highly fertile.

3.5.3 Many-to-many alignment

На практике часто возникает соответствие не слово-к-слову, а фраза-к-фразе.

Например:

do not have any money

может переводиться как цельное выражение, и отдельные слова могут не иметь независимых прямых соответствий.

Такое соответствие называется:

many-to-many alignment.

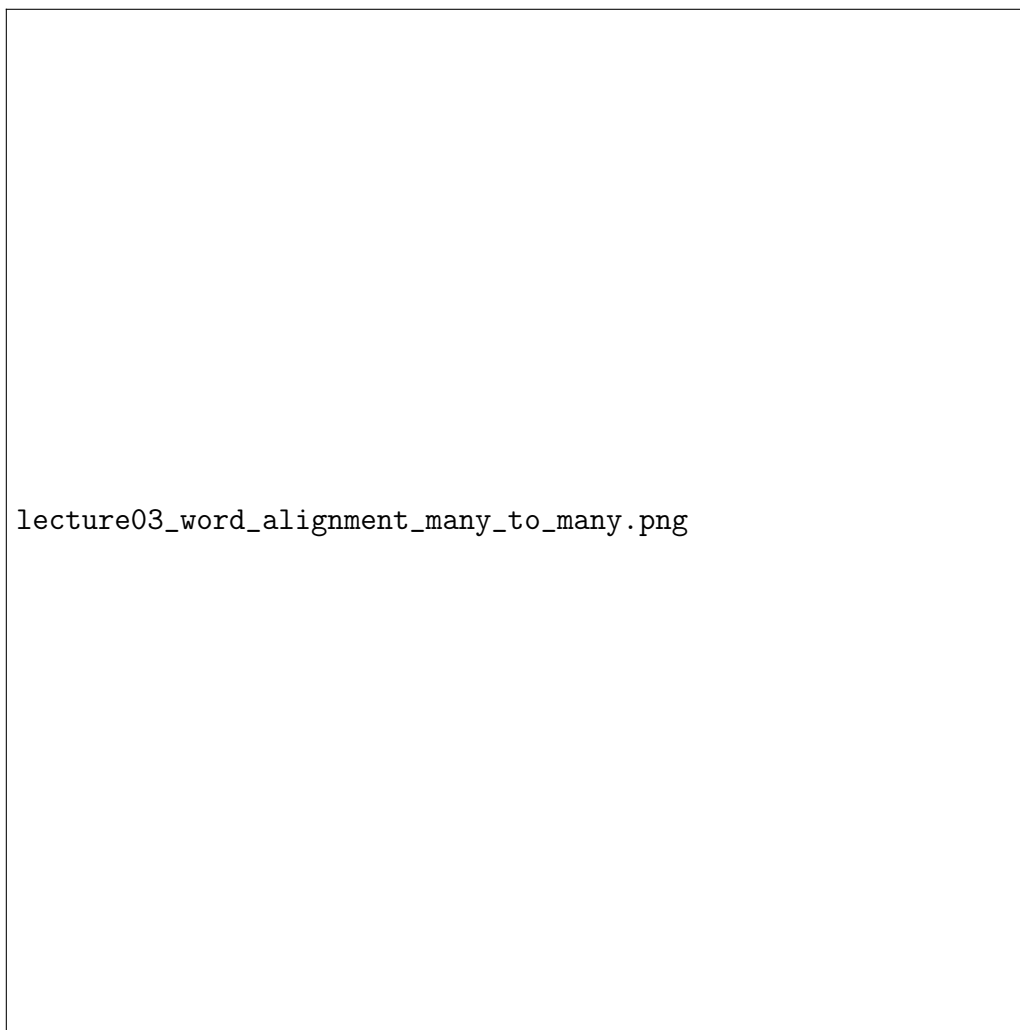


Figure 10: Пример many-to-many alignment: соответствие между фразами, а не отдельными словами.

3.6 Почему полный перебор переводов невозможен

В SMT нужно найти:

$$\hat{y} = \arg \max_y P(x | y)P(y).$$

Но множество возможных переводов y огромно. Нельзя перебрать все возможные предложения на целевом языке и посчитать вероятность каждого.

Поэтому используются эвристические алгоритмы поиска, которые:

- строят перевод постепенно;
- хранят только наиболее вероятные гипотезы;
- отбрасывают низковероятные варианты.

Эта идея позже появится и в neural machine translation в виде beam search.

3.7 Phrase-Based Machine Translation

Phrase-Based Machine Translation, PBMT — развитие SMT, где перевод выполняется не отдельными словами, а фразами.

Здесь **phrase** не обязательно означает грамматическую фразу. Это может быть просто последовательность из нескольких слов, то есть n -грамма.

PBMT использует:

- **phrase table**;
- **n -gram language model**.

3.7.1 Phrase table

Phrase table хранит возможные переводы фраз и их вероятности или стоимости.

Например, для source phrase:

I saw a cat

могут быть варианты перевода:

Я видел кошку,

Я увидел кошку,

Я видел кошечку.

Каждому варианту сопоставляется score, например:

$$-\log P.$$

Чем меньше значение $-\log P$, тем выше вероятность.

3.7.2 n -gram language model

Language model оценивает вероятность следующего слова по предыдущему контексту:

$$P(w_t \mid w_{t-n+1}, \dots, w_{t-1}).$$

Она помогает выбрать перевод, который звучит естественно на целевом языке.

Например, если модель видела много предложений вида:

I saw a cat,

I saw a dog,

то она понимает, что после *I saw a* вероятно существительное.

3.7.3 Недостатки RBMT

RBMT-системы имели много отдельно проектируемых компонентов:

- phrase table;
- language model;
- distortion model;
- reordering model;
- feature weights;
- decoding algorithm.

Это приводило к проблемам:

- много feature engineering;
- нужно вручную проектировать признаки под языковые явления;
- нужно поддерживать большие таблицы фраз;
- много человеческих усилий;
- для каждой языковой пары работу приходилось повторять.

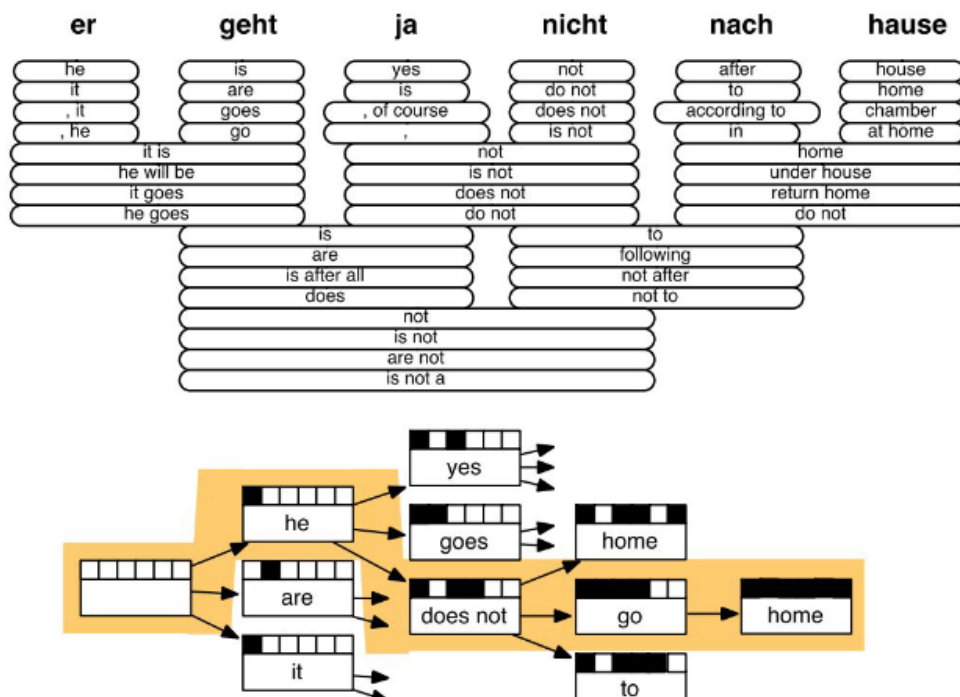


Figure 11: Phrase-Based Machine Translation: перевод строится из фразовых соответствий и оценивается языковой моделью.

3.8 Neural Machine Translation

Neural Machine Translation, NMT — подход, в котором машинный перевод выполняется одной нейронной сетью.

Вместо множества вручную спроектированных компонентов используется end-to-end обучение:

$$x \mapsto y.$$

NMT напрямую моделирует:

$$P(y \mid x),$$

где:

- x — source sentence;
- y — target sentence.

Классическая архитектура NMT называется:

sequence-to-sequence

или сокращённо:

seq2seq.

3.9 Seq2Seq

Seq2Seq состоит из двух основных частей:

- encoder;
- decoder.

3.9.1 Encoder

Encoder читает исходное предложение:

$$x = (x_1, x_2, \dots, x_m)$$

и преобразует его в скрытое представление.

Если encoder является RNN, то на каждом шаге:

$$h_i = f(h_{i-1}, x_i).$$

Финальное состояние encoder-a:

$$h_m$$

считается вектором, который кодирует всё исходное предложение.

3.9.2 Decoder

Decoder генерирует перевод по одному токenu.

Он начинает со специального токена:

$$\langle SOS \rangle,$$

а затем на каждом шаге предсказывает следующий токен:

$$y_t.$$

Генерация продолжается до специального токена:

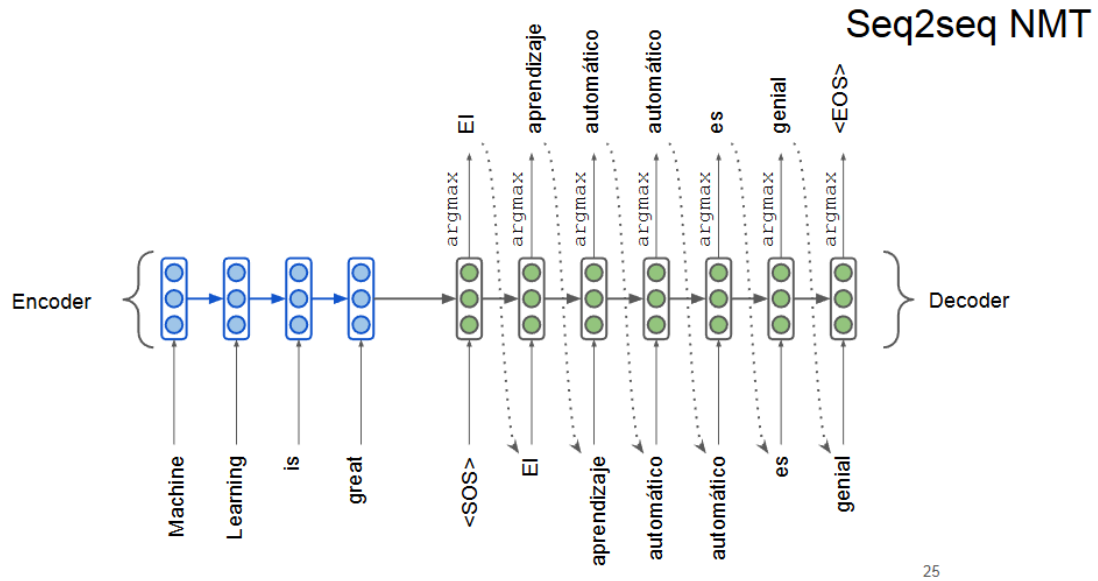
$$\langle EOS \rangle.$$

Формально:

$$P(y | x) = \prod_{t=1}^T P(y_t | y_1, \dots, y_{t-1}, x).$$

Или короче:

$$P(y | x) = \prod_{t=1}^T P(y_t | y_{<t}, x).$$



25

Figure 12: Seq2seq NMT: encoder кодирует исходное предложение, decoder генерирует перевод token за tokenом.

3.10 Обучение Seq2Seq

Seq2seq обучается end-to-end на параллельном корпусе.

Для каждой пары:

$$(x, y)$$

модель максимизирует вероятность правильного перевода:

$$P(y | x).$$

Обычно минимизируется negative log-likelihood:

$$\mathcal{L} = -\log P(y | x).$$

С учётом разложения по токенам:

$$\mathcal{L} = - \sum_{t=1}^T \log P(y_t \mid y_{<t}, x).$$

Интуитивно модель штрафуются каждый раз, когда она даёт низкую вероятность правильному следующему токenu перевода.

Во время обучения часто используется teacher forcing: на вход decoder-а на следующем шаге подаётся правильный предыдущий токен, а не обязательно токен, предсказанный самой моделью.

3.11 Greedy Decoding

После обучения нужно сгенерировать перевод.

Самый простой способ — **greedy decoding**.

На каждом шаге decoder выбирает наиболее вероятный токен:

$$\hat{y}_t = \arg \max_w P(w \mid \hat{y}_{<t}, x).$$

Потом выбранный токен подаётся на следующий шаг.

Проблема greedy decoding:

- локально лучший токен не обязательно ведёт к глобально лучшему предложению;
- если модель ошиблась на раннем шаге, эта ошибка становится частью входа на следующих шагах;
- ошибки могут накапливаться.

То есть greedy decoding делает жадный выбор без рассмотрения альтернатив.

3.12 Exhaustive Search

Теоретически хотелось бы найти перевод:

$$\hat{y} = \arg \max_y P(y \mid x).$$

Но:

$$P(y \mid x) = P(y_1 \mid x) \prod_{t=2}^T P(y_t \mid y_1, \dots, y_{t-1}, x).$$

Если размер словаря равен V , а длина перевода равна T , то число возможных последовательностей порядка:

$$V^T.$$

Это экспоненциальная сложность, поэтому полный перебор невозможен.

3.13 Beam Search

Beam Search — компромисс между greedy decoding и полным перебором.

Идея:

- на каждом шаге хранить не одну лучшую гипотезу, а k лучших частичных переводов;
- k называется beam size;
- на практике часто используют $k \approx 5-10$;
- на каждом шаге все текущие гипотезы расширяются возможными следующими словами;
- затем снова оставляются только top- k гипотез.

Гипотеза:

$$(y_1, \dots, y_t)$$

имеет score, равный логарифму вероятности:

$$score(y_1, \dots, y_t) = \log P_{LM}(y_1, \dots, y_t \mid x).$$

С учётом авторегрессионного разложения:

$$score(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{LM}(y_i \mid y_1, \dots, y_{i-1}, x).$$

Beam search ищет high-scoring hypotheses, но не гарантирует нахождение глобально оптимального перевода.

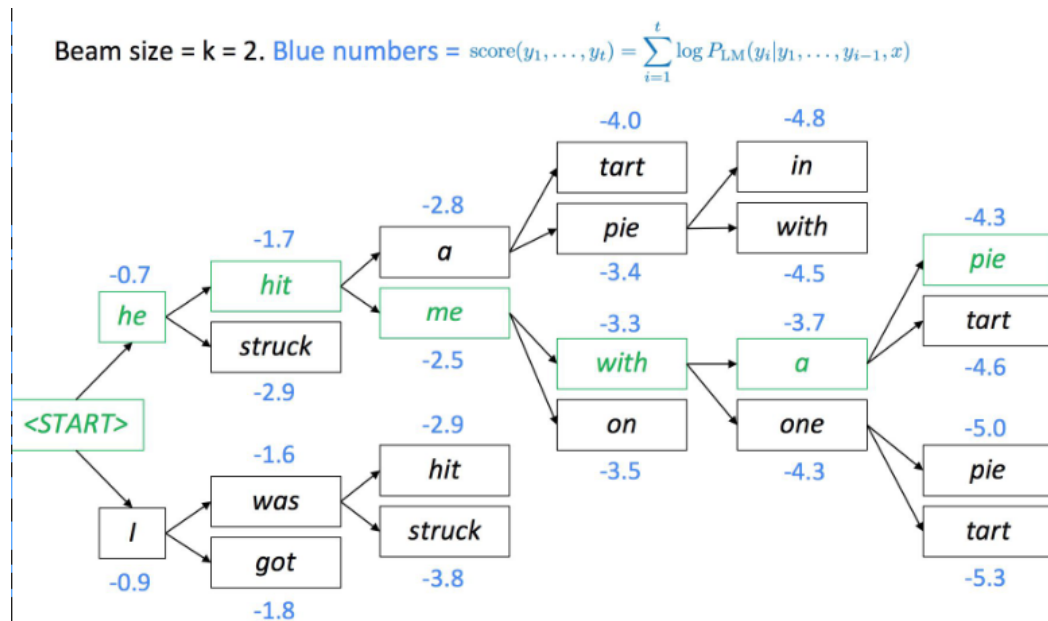


Figure 13: Пример beam search decoding: на каждом шаге сохраняются top- k частичных гипотез.

3.14 Критерий остановки Beam Search

В greedy decoding обычно генерация продолжается до тех пор, пока модель не выдаст:

$\langle EOS \rangle$.

В beam search разные гипотезы могут завершиться на разных шагах.

Если гипотеза породила:

$\langle EOS \rangle$,

то она считается завершённой. Её можно отложить в список completed hypotheses, а beam search продолжает развивать остальные гипотезы.

Обычно beam search продолжают до одного из условий:

- достигнут заранее заданный максимальный timestep T ;
- накоплено достаточно завершённых гипотез;
- все гипотезы завершились.

3.15 Length Normalization

У beam search есть проблема: длинные гипотезы обычно имеют меньший score.

Причина: score является суммой логарифмов вероятностей:

$$score(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{LM}(y_i \mid y_1, \dots, y_{i-1}, x).$$

Так как вероятности меньше или равны 1, их логарифмы обычно неположительны:

$$\log P \leq 0.$$

Чем больше токенов, тем больше отрицательных слагаемых. Поэтому длинные переводы штрафуются просто за длину.

Простая коррекция — нормировать score на длину:

$$score_{norm}(y_1, \dots, y_t) = \frac{1}{t} \sum_{i=1}^t \log P_{LM}(y_i \mid y_1, \dots, y_{i-1}, x).$$

Это позволяет сравнивать гипотезы разной длины более честно.

3.16 Оценка качества машинного перевода

Для оценки качества машинного перевода используются автоматические и человеческие метрики.

3.17 BLEU

BLEU расшифровывается как:

Bilingual Evaluation Understudy.

BLEU сравнивает машинный перевод с человеческим reference translation.

Метрика основана на двух идеях:

- n -gram precision;
- brevity penalty.

3.17.1 n -gram precision

n -gram precision измеряет, какая доля n -грамм машинного перевода встречается в reference translation.

Например, для unigram precision:

$$precision_1 = \frac{\text{число совпавших unigram}}{\text{число unigram в машинном переводе}}.$$

Аналогично можно считать:

$$precision_2, \quad precision_3, \quad precision_4.$$

3.17.2 Brevity penalty

Если система выдаёт слишком короткий перевод, она может получить искусственно высокую precision.

Например, если reference:

the cat sits on the mat,

а система выдала только:

the cat,

то precision может быть высокой, но перевод неполный.

Поэтому BLEU использует brevity penalty.

В упрощённой форме из лекции:

$$brevity\ penalty = \min \left(1, \frac{output\ length}{reference\ length} \right).$$

3.17.3 Формула BLEU

Для n -грамм до порядка n :

$$BLEU = brevity\ penalty \cdot \left(\prod_{i=1}^n precision_i \right)^{1/n} \cdot 100\%.$$

Смысл:

- высокая BLEU означает сильное n -граммное совпадение с reference translation;
- brevity penalty штрафует слишком короткие переводы;
- геометрическое среднее делает метрику чувствительной к плохим precision на больших n -граммах.

3.18 Недостатки BLEU

BLEU несовершенна.

Главная проблема: одно и то же предложение можно корректно перевести разными способами.

Если хороший перевод использует другие слова или другой порядок слов, он может получить низкий BLEU из-за малого n -gram overlap с reference.

То есть BLEU хорошо подходит для быстрой автоматической оценки на корпусном уровне, но плохо отражает качество отдельных переводов.

3.19 Другие метрики качества МТ

Кроме BLEU, используются:

- **ROUGE** — Recall-Oriented Understudy for Gisting Evaluation;
- **METEOR** — Metric for Evaluation of Translation with Explicit ORdering;
- human evaluation;
- ML-based quality estimation.

METEOR может использовать синонимы из WordNet, поэтому она менее жёстко зависит от буквального совпадения слов.

3.20 Human Evaluation

Человеческая оценка качества перевода может быть устроена по-разному.

3.20.1 Side-by-side human evaluation

Человеку показывают исходное предложение и несколько переводов.

Задача:

выбрать лучший перевод.

Например:

- System 1: плохой или неестественный перевод;
- System 2: более корректный перевод.

3.20.2 Direct Assessment

В Direct Assessment человеку показывают один перевод и просят оценить его по шкале, например от 1 до 5.

Человеческая оценка обычно качественнее автоматических метрик, но она дорогая.

3.21 ML-based quality estimation

Так как human evaluation дорогая, появляются модели, обученные приближать человеческую оценку.

Примеры:

- **BLEURT** — модель Google, BERT fine-tuned на Direct Assessment ratings;
- **XCOMET** — модель Unbabel, XLM fine-tuned на DA и MQM ratings;
- **LLM-based estimators** — использование больших языковых моделей для оценки качества перевода.

3.22 Преимущества NMT

Преимущества neural machine translation:

- лучшее качество перевода;
- более fluent output;
- лучшее использование контекста;
- лучшее использование похожести фраз;
- одна нейронная сеть оптимизируется end-to-end;
- нет необходимости вручную оптимизировать множество отдельных компонентов;
- меньше feature engineering;
- один и тот же общий подход можно применять к разным языковым парам.

3.23 Недостатки NMT

Недостатки neural machine translation:

- модель менее интерпретируема;
- её труднее отлаживать;
- поведение модели сложнее контролировать;
- трудно явно задать правила или guidelines для перевода;
- возможны safety concerns.

Нейронные модели могут давать странные переводы на необычных входах, опечатках, повторяющихся символах или фразах вне распределения обучающих данных.

3.24 Решён ли машинный перевод?

Машинный перевод нельзя считать полностью решённой задачей.

Остаются проблемы:

- использование большого контекста;
- low-resource language pairs;
- отсутствие больших параллельных корпусов для многих языков;
- сленг;
- узкие домены;
- устойчивость к ошибкам и опечаткам;
- контроль качества и безопасности перевода.

3.25 Bottleneck в Seq2Seq

В классическом seq2seq вся информация об исходном предложении должна быть сжата в один вектор — финальное hidden state encoder-a.

$$h_m$$

используется как начальное состояние decoder-a.

Проблема: если исходное предложение длинное или сложное, один вектор может стать узким местом.

Это называется:

bottleneck.

Decoder должен генерировать весь перевод, опираясь на одно фиксированное представление исходного предложения. Поэтому модель может терять детали, особенно для длинных предложений.

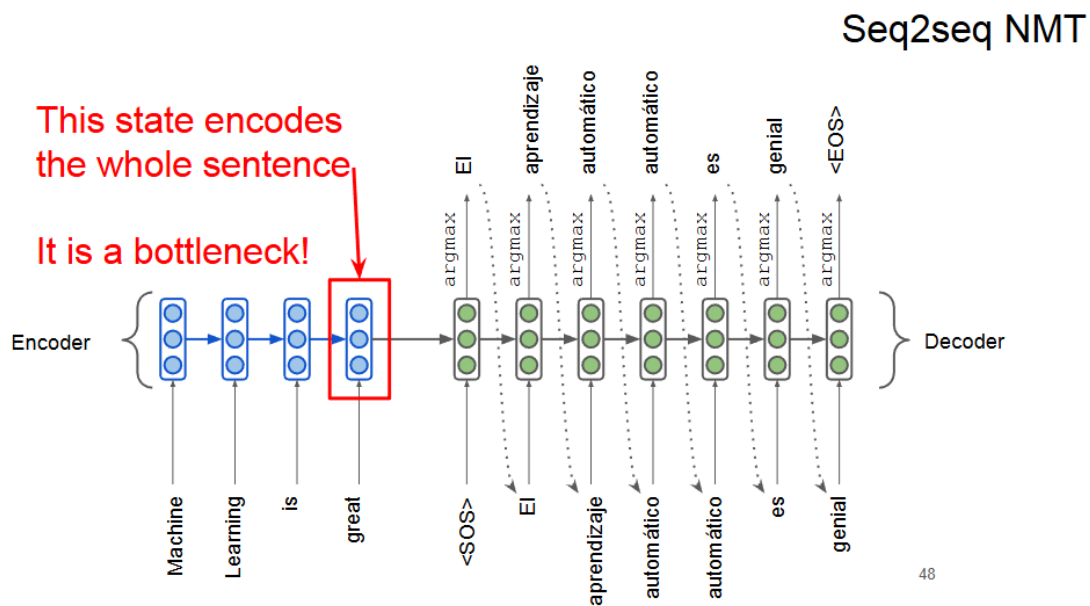


Figure 14: Bottleneck классического seq2seq: всё исходное предложение кодируется одним состоянием encoder-a.

3.26 Attention: основная идея

Attention решает проблему bottleneck.

Основная идея:

На каждом шаге decoder-a использовать прямое соединение с encoder-ом, чтобы фокусироваться на конкретной части исходной последовательности.

То есть decoder больше не обязан пользоваться только одним финальным вектором encoder-a. Вместо этого на каждом шаге генерации он может смотреть на все hidden states encoder-a:

$$h_1, h_2, \dots, h_m.$$

Для каждого шага decoder-a attention вычисляет:

- насколько важен каждый encoder state;
- distribution внимания по source tokens;
- weighted sum encoder states;
- context vector для текущего шага decoder-a.

3.27 Seq2Seq with Attention

Пусть encoder породил hidden states:

$$h_1, h_2, \dots, h_m.$$

Пусть decoder на шаге t имеет состояние:

$$s_t.$$

Attention вычисляет score для каждой позиции исходного предложения:

$$e_{t,i} = \text{score}(s_t, h_i).$$

Score показывает, насколько encoder state h_i важен для decoder-а на шаге t .

Затем scores нормируются softmax-ом:

$$\alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_{j=1}^m \exp(e_{t,j})}.$$

Коэффициенты:

$$\alpha_{t,1}, \dots, \alpha_{t,m}$$

образуют attention distribution:

$$\sum_{i=1}^m \alpha_{t,i} = 1.$$

После этого строится attention output, или context vector:

$$c_t = \sum_{i=1}^m \alpha_{t,i} h_i.$$

То есть context vector — это взвешенная сумма всех encoder states.

Затем c_t объединяется с состоянием decoder-а s_t и используется для предсказания следующего токена:

$$P(y_t \mid y_{<t}, x).$$



lecture03_attention_weighted_sum.png

Figure 15: Seq2seq with attention: scores преобразуются softmax-ом в attention distribution, затем строится weighted sum encoder states.

3.28 Attention в формулах

Обозначим:

$$h_i$$

— hidden state encoder-а для i -го source token.

$$s_t$$

— hidden state decoder-а на шаге t .

3.28.1 Dot-product attention

Самый простой вариант score:

$$e_{t,i} = s_t^\top h_i.$$

Чем больше скалярное произведение, тем больше похожи состояния decoder-а и encoder-а, тем больше внимания получает позиция i .

Attention distribution:

$$\alpha_t = \text{softmax}(e_t).$$

Компонентно:

$$\alpha_{t,i} = \frac{\exp(s_t^\top h_i)}{\sum_{j=1}^m \exp(s_t^\top h_j)}.$$

Context vector:

$$c_t = \sum_{i=1}^m \alpha_{t,i} h_i.$$

3.28.2 Multiplicative attention

В multiplicative attention добавляется обучаемая матрица W :

$$e_{t,i} = s_t^\top W h_i.$$

Здесь W позволяет модели обучить более гибкую меру совместимости между decoder state и encoder state.

3.28.3 Additive attention

В additive attention score вычисляется через маленькую нейронную сеть:

$$e_{t,i} = v^\top \tanh(W_1 h_i + W_2 s_t).$$

Здесь:

- W_1, W_2 — обучаемые матрицы;
- v — обучаемый вектор;
- $\tanh$ добавляет нелинейность.

Additive attention часто связывают с работой Bahdanau et al. “Neural Machine Translation by Jointly Learning to Align and Translate”, 2014.

3.29 Интерпретируемость Attention

Attention даёт частичную интерпретируемость.

На каждом шаге decoder-а можно посмотреть attention distribution:

$$\alpha_{t,1}, \dots, \alpha_{t,m}.$$

Она показывает, на какие слова source sentence модель обращала внимание при генерации текущего target token.

Это похоже на word alignment:

source tokens $\leftrightarrow$ target tokens.

Поэтому говорят, что attention даёт word alignment “for free”.

Важно понимать: attention weights не всегда являются идеальным объяснением работы модели, но для машинного перевода они часто дают полезную визуализацию соответствий между словами.

3.30 Почему Attention важен

Attention стал ключевым механизмом, потому что:

- снимает bottleneck фиксированного вектора encoder-а;
- позволяет decoder-у обращаться к разным частям source sentence на разных шагах;
- улучшает перевод длинных предложений;
- даёт полезные alignment-визуализации;
- стал основой дальнейшего развития NLP-архитектур.

Позже идея attention приведёт к Transformer, где attention станет главным механизмом обработки последовательностей.

3.31 Сравнение SMT, PBMT и NMT

Подход	Идея	Проблемы
SMT	Использует вероятностную модель $P(x y)P(y)$.	Много независимых компонентов, сложный feature engineering.
PBMT	Переводит фразами, использует phrase table и n -gram LM.	Нужно хранить и поддерживать phrase tables, плохо масштабируется на новые языковые пары.
NMT	Одна нейронная сеть напрямую моделирует $P(y x)$.	Менее интерпретируема, сложнее контролировать, требует большие данные.
Seq2seq without attention	Encoder сжимает всё предложение в один вектор, decoder генерирует перевод.	Bottleneck для длинных и сложных предложений.
Seq2seq with attention	Decoder на каждом шаге смотрит на разные encoder states.	Лучше работает с длинным контекстом, но всё ещё требует аккуратного обучения и декодирования.

Table 3: Сравнение подходов к машинному переводу.

3.32 Главные выводы

- Statistical Machine Translation ищет перевод через:

$$\arg \max_y P(x | y)P(y).$$

- В SMT есть две главные части:

- translation model;
- language model.

- Word alignment описывает соответствия между словами исходного и целевого предложений.

- Alignment бывает:

- one-to-one;
- many-to-one;
- one-to-many;

– many-to-many.

- PBMT переводит не отдельными словами, а фразами.
- Neural Machine Translation моделирует $P(y \mid x)$ одной нейронной сетью.
- Seq2seq состоит из encoder-a и decoder-a.
- Decoder генерирует перевод авторегрессионно:

$$P(y \mid x) = \prod_{t=1}^T P(y_t \mid y_{<t}, x).$$

- Greedy decoding выбирает самый вероятный токен на каждом шаге, но может ошибаться глобально.
- Beam search хранит top- k гипотез на каждом шаге и является практическим компромиссом.
- BLEU оценивает перевод через n -gram precision и brevity penalty.
- Классический seq2seq имеет bottleneck: всё исходное предложение сжимается в один вектор.
- Attention позволяет decoder-у на каждом шаге обращаться ко всем hidden states encoder-a.
- Attention context vector:

$$c_t = \sum_{i=1}^m \alpha_{t,i} h_i.$$

- Attention улучшает перевод длинных предложений и даёт интерпретируемое alignment-подобное поведение.

3.33 Что важно помнить к экзамену

1. Формулу SMT через Байеса:

$$\hat{y} = \arg \max_y P(y \mid x) = \arg \max_y P(x \mid y)P(y).$$

2. Что такое translation model и language model.
3. Что такое word alignment и какие бывают типы alignment.
4. Что PBMT использует phrase tables и n -gram language model.

5. Почему SMT/PBMT требуют много feature engineering.

6. Что NMT напрямую моделирует:

$$P(y \mid x).$$

7. Разложение вероятности перевода в seq2seq:

$$P(y \mid x) = \prod_{t=1}^T P(y_t \mid y_{<t}, x).$$

8. Loss для обучения:

$$\mathcal{L} = - \sum_{t=1}^T \log P(y_t \mid y_{<t}, x).$$

9. Чем greedy decoding отличается от beam search.

10. Score гипотезы в beam search:

$$score(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{LM}(y_i \mid y_1, \dots, y_{i-1}, x).$$

11. Зачем нужна length normalization.

12. Формулу BLEU:

$$BLEU = brevity\ penalty \cdot \left(\prod_{i=1}^n precision_i \right)^{1/n} \cdot 100\%.$$

13. Почему BLEU несовершенна.

14. Что bottleneck в seq2seq — это необходимость сжать всё source sentence в один вектор.

15. Основную идею attention: decoder на каждом шаге фокусируется на разных hidden states encoder-a.

16. Формулы attention:

$$\begin{aligned} e_{t,i} &= score(s_t, h_i), \\ \alpha_{t,i} &= \frac{\exp(e_{t,i})}{\sum_{j=1}^m \exp(e_{t,j})}, \\ c_t &= \sum_{i=1}^m \alpha_{t,i} h_i. \end{aligned}$$

17. Три варианта attention score:

- dot-product:

$$e_{t,i} = s_t^\top h_i;$$

- multiplicative:

$$e_{t,i} = s_t^\top W h_i;$$

- additive:

$$e_{t,i} = v^\top \tanh(W_1 h_i + W_2 s_t).$$

““

““latex

4 Лекция 4. Self-Attention and Transformer

4.1 План лекции

В этой лекции рассматриваются:

- краткое повторение CNN для текстов;
- повторение attention в seq2seq;
- архитектура Transformer;
- self-attention;
- multi-head attention;
- positional encoding;
- layer normalization;
- decoder side и masked self-attention.

Главная идея лекции: Transformer заменяет рекуррентность и свёртки механизмом self-attention. Благодаря этому модель может напрямую связывать любые позиции последовательности, хорошо параллелится и эффективно работает с длинными зависимостями.

4.2 Повторение: CNN для текстов

CNN для текстов можно понимать как обучаемый аналог n -граммных признаков.

Пусть текст представлен матрицей embeddings:

$$X \in \mathbb{R}^{n \times d},$$

где:

- n — длина последовательности;
- d — размерность embedding;
- строка X_i — embedding i -го токена.

Свёрточный фильтр размера h применяется к окну:

$$X_{i:i+h-1}.$$

Значение признака в позиции i :

$$c_i = f(W^\top X_{i:i+h-1} + b).$$

После применения фильтра ко всем позициям получается feature map:

$$\mathbf{c} = [c_1, c_2, \dots, c_{n-h+1}].$$

Для классификации текста часто используется max-over-time pooling:

$$\hat{c} = \max_i c_i.$$

Смысл: фильтр ищет некоторый паттерн в тексте, а max pooling проверяет, встретился ли этот паттерн где-нибудь.

CNN для текста хорошо ловит локальные паттерны, но для моделирования дальних зависимостей нужны либо глубокие сети, либо другие механизмы.

4.3 Повторение: attention в seq2seq

В seq2seq без attention encoder сжимает всё исходное предложение в один финальный вектор. Это создаёт bottleneck: decoder должен генерировать перевод, опираясь на одно фиксированное представление всего входа.

Attention решает эту проблему: на каждом шаге decoder может обращаться ко всем hidden states encoder-a.

Пусть encoder породил состояния:

$$h_1, h_2, \dots, h_N, \quad h_i \in \mathbb{R}^h.$$

Пусть на шаге t decoder имеет состояние:

$$s_t \in \mathbb{R}^h.$$

Для каждого encoder state вычисляется attention score:

$$e_{t,i} = s_t^\top h_i.$$

В векторной форме:

$$e^t = [s_t^\top h_1, \dots, s_t^\top h_N] \in \mathbb{R}^N.$$

Затем применяется softmax:

$$\alpha^t = \text{softmax}(e^t).$$

Вектор α^t является распределением внимания:

$$\sum_{i=1}^N \alpha_i^t = 1.$$

Attention output:

$$a_t = \sum_{i=1}^N \alpha_i^t h_i.$$

Затем a_t конкатенируется с decoder state:

$$[a_t; s_t],$$

и используется для предсказания следующего токена.

4.4 Варианты attention score

В лекции повторяются три основных варианта attention.

4.4.1 Dot-product attention

$$e_i = s^\top h_i.$$

Это самый простой вариант: совместимость decoder state и encoder state считается скалярным произведением.

4.4.2 Multiplicative attention

$$e_i = s^\top W h_i.$$

Здесь W — обучаемая матрица. Она позволяет модели выучить более гибкую меру совместимости.

4.4.3 Additive attention

$$e_i = v^\top \tanh(W_1 h_i + W_2 s).$$

Здесь:

- W_1, W_2 — обучаемые матрицы;
- v — обучаемый вектор;
- $\tanh$ добавляет нелинейность.

4.5 Преимущества attention

Attention даёт два важных преимущества:

- **Free word alignment:** можно посмотреть, на какие слова source sentence модель обращает внимание при генерации каждого target token.
- **Better results on long sequences:** модель лучше работает с длинными последовательностями потому что decoder не ограничен одним финальным состоянием encoder-a.



lecture04_attention_advantages.png

Figure 16: Преимущества attention: лучшее качество на длинных последовательностях и интерпретируемое выравнивание слов.

4.6 Transformer: общая идея

Transformer — архитектура, предложенная в статье *Attention is All You Need*.

Ключевая идея:

никаких RNN и CNN, только attention.

Transformer был предложен для машинного перевода и показал сильные результаты на WMT 2014 English-to-German translation task. В лекции указано значение:

28.4 BLEU.

Главные свойства Transformer:

- использует self-attention;
- не использует рекуррентные связи;

- хорошо параллелится;
- имеет короткий путь между любыми двумя позициями последовательности;
- стал базовой архитектурой для современных NLP-моделей.

4.7 Encoder-decoder структура Transformer

Transformer в классическом виде состоит из двух частей:

- encoder stack;
- decoder stack.

Encoder читает входную последовательность и строит контекстуальные представления токенов.

Decoder генерирует выходную последовательность авторегрессионно, используя:

- уже сгенерированные target tokens;
- выходы encoder-a;
- masked self-attention;
- encoder-decoder attention.



lecture04_transformer_stack.png

Figure 17: Transformer состоит из стека encoder-блоков и decoder-блоков.

4.8 Encoder block

Один encoder block состоит из двух основных подслоёв:

1. self-attention layer;
2. feed-forward neural network.

Схематически:

$$X \longrightarrow \text{Self-Attention} \longrightarrow Z \longrightarrow \text{Feed Forward} \longrightarrow R.$$

Self-attention позволяет каждому токeну учитывать другие токeны входной последовательности

Feed-forward network применяется к каждой позиции независимо, но с общими параметрами для всех позиций.

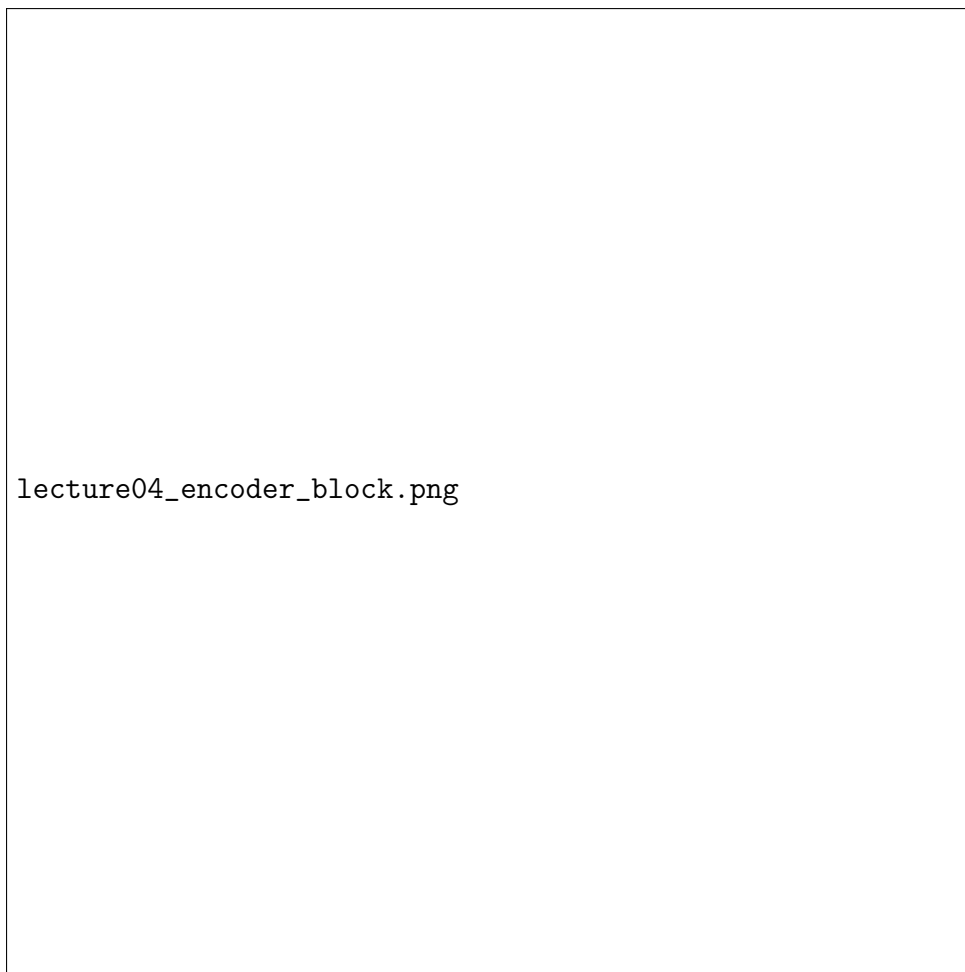


Figure 18: Один encoder block: self-attention layer и feed-forward layer.

Важное отличие от RNN: позиции последовательности не обрабатываются строго последовательно. Поэтому вычисления внутри слоя можно эффективно параллелить.

4.9 Self-Attention: интуиция

Self-attention нужен, чтобы при обработке каждого токена учитывать другие релевантные токены той же последовательности.

Пример из лекции:

The animal didn't cross the street because it was too tired.

Вопрос:

к чему относится *it*?

Правильная связь:

$it \rightarrow animal.$

Self-attention должен позволить представлению слова *it* включить информацию о слове *animal*.

То есть новое представление токена становится контекстуальным: оно зависит не только от самого токена, но и от других слов предложения.



Figure 19: Self-attention позволяет токеноу учитывать другие релевантные токены той же последовательности.

4.10 Query, Key, Value

В self-attention из каждого входного embedding строятся три вектора:

$$q_i, \quad k_i, \quad v_i.$$

Они называются:

- **Query;**
- **Key;**
- **Value.**

Пусть входной embedding токена:

$$x_i \in \mathbb{R}^{d_{model}}.$$

Тогда:

$$q_i = W^Q x_i,$$

$$k_i = W^K x_i,$$

$$v_i = W^V x_i.$$

Здесь W^Q , W^K , W^V — обучаемые матрицы.

Интуиция:

- **Query** — что текущий токен ищет;
- **Key** — что токен предлагает для сопоставления;
- **Value** — какая информация будет передана, если на этот токен обратят внимание.

4.11 Self-Attention для одного токена

Рассмотрим токен в позиции i .

Сначала считаем score между query текущего токена и key каждого токена:

$$score(i, j) = q_i^\top k_j.$$

Это показывает, насколько токен i должен обратить внимание на токен j .

Затем scores масштабируются:

$$\frac{q_i^\top k_j}{\sqrt{d_k}}.$$

Деление на $\sqrt{d_k}$ нужно для численной стабильности. Если размерность d_k большая, скалярные произведения могут иметь большую дисперсию, softmax становится слишком резким, и градиенты ухудшаются.

Затем применяется softmax:

$$\alpha_{ij} = \frac{\exp\left(\frac{q_i^\top k_j}{\sqrt{d_k}}\right)}{\sum_{\ell=1}^n \exp\left(\frac{q_i^\top k_\ell}{\sqrt{d_k}}\right)}.$$

После этого выход self-attention для позиции i :

$$z_i = \sum_{j=1}^n \alpha_{ij} v_j.$$

То есть новый вектор токена — это взвешенная сумма value-векторов всех токенов.



Figure 20: Scaled dot-product attention: scores масштабируются и нормируются softmax-ом.

4.12 Self-Attention в матричной форме

Для всей последовательности можно собрать матрицы:

$$X \in \mathbb{R}^{n \times d_{model}}.$$

Тогда:

$$Q = XW^Q,$$

$$K = XW^K,$$

$$V = XW^V.$$

Матрица attention scores:

$$QK^\top.$$

После масштабирования:

$$\frac{QK^\top}{\sqrt{d_k}}.$$

Attention weights:

$$A = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right).$$

Здесь softmax применяется построчно.

Итоговый выход:

$$Z = AV.$$

Полная формула:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right) V.$$

Это одна из главных формул Transformer.

4.13 Почему self-attention полезен

Self-attention имеет несколько важных свойств.

4.13.1 Короткий путь между позициями

В RNN информация от первого токена до последнего должна пройти через много recurrent steps.

В self-attention любая позиция может напрямую обратиться к любой другой позиции за один слой.

Поэтому path length между любыми двумя позициями:

$$O(1)$$

внутри одного attention layer.

4.13.2 Параллелизация

RNN обрабатывает последовательность последовательно:

$$h_t = f(h_{t-1}, x_t).$$

Следовательно, состояние h_t нельзя посчитать до h_{t-1} .

Transformer не имеет такой зависимости внутри encoder self-attention: все позиции слоя можно обрабатывать параллельно.

4.13.3 Контекстуальные представления

После self-attention представление каждого токена зависит от всей последовательности. Поэтому одно и то же слово может иметь разные векторы в разных контекстах.

4.14 Multi-Head Attention

Обычный attention даёт одну взвешенную сумму value-векторов.

Но одной attention-головы может быть недостаточно: разные типы связей в предложении могут требовать разных attention patterns.

Multi-head attention запускает несколько attention-механизмов параллельно.

Для каждой головы r :

$$Q_r = XW_r^Q,$$

$$K_r = XW_r^K,$$

$$V_r = XW_r^V.$$

Затем:

$$head_r = \text{Attention}(Q_r, K_r, V_r).$$

Если голов H , получаем:

$$head_1, head_2, \dots, head_H.$$

Они конкатенируются:

$$\text{Concat}(head_1, \dots, head_H),$$

а затем применяется выходная линейная проекция:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_H)W^O.$$

4.15 Зачем нужен Multi-Head Attention

Multi-head attention полезен потому, что разные головы могут смотреть на разные типы отношений:

- синтаксические зависимости;
- согласование местоимений;
- локальный контекст;
- дальний контекст;
- связи между объектом и действием;
- связи между частями фразы.

Иными словами, одна голова может фокусироваться на одном типе паттернов, другая — на другом.

В лекции multi-head attention описан как:

parallel attention layers with different linear transformations on input and output.

То есть это несколько attention layers, работающих параллельно с разными обучаемыми проекциями.

4.16 Attention vs Multi-Head Attention

Attention	Multi-Head Attention
Одна weighted average по value-векторам.	Несколько attention-механизмов параллельно.
Одна мера сходства между токенами.	Несколько разных обучаемых пространств сходства.
Может ловить один доминирующий тип связей.	Может одновременно ловить разные типы зависимостей.
Выход сразу используется дальше.	Выходы голов конкатенируются и проецируются через W^O .

Table 4: Сравнение обычного attention и multi-head attention.

4.17 Positional Encoding

Self-attention сам по себе не знает порядок токенов.

Если переставить токены в последовательности, то механизм attention без дополнительной информации не имеет встроенного способа понять, какая позиция была первой, второй и т.д.

Поэтому в Transformer добавляется **positional encoding**.

Идея:

$$\text{input representation} = \text{token embedding} + \text{positional encoding}.$$

То есть для токена x_i :

$$\tilde{x}_i = x_i + p_i,$$

где p_i — positional encoding позиции i .

4.18 Требования к positional encoding

В лекции перечислены требования:

- positional encoding должен быть уникальным для каждой позиции;
- расстояние между одинаковыми позициями должно сохраняться для последовательностей разной длины;
- желательно, чтобы encoding был детерминированным;
- желательно, чтобы он работал с последовательностями длиннее тех, что встречались при обучении.

4.19 Sinusoidal positional encoding

В классическом Transformer используется синусоидальный positional encoding.

Для позиции pos и измерения i :

$$PE_{(pos, 2i)} = \sin \left(\frac{pos}{10000^{2i/d_{model}}} \right),$$

$$PE_{(pos, 2i+1)} = \cos \left(\frac{pos}{10000^{2i/d_{model}}} \right).$$

Каждая размерность positional encoding соответствует синусоиде определённой частоты.

Длины волн образуют геометрическую прогрессию от:

$$2\pi$$

до примерно:

$$2\pi \cdot 10000.$$

Почему используются $\sin$ и $\cos$:

- encoding детерминированный;
- можно вычислять позиции, не встречавшиеся при обучении;
- относительные сдвиги между позициями имеют регулярную структуру;
- разные частоты позволяют кодировать как локальные, так и дальние позиционные отношения.



Figure 21: Sinusoidal positional encoding: разные измерения используют синусы и косинусы разных частот.

4.20 Layer Normalization

Layer Normalization — нормализация активаций внутри одного объекта по его признакам.

Если есть вектор признаков:

$$x = (x_1, x_2, \dots, x_d),$$

то считаются среднее и дисперсия по координатам этого вектора:

$$\mu = \frac{1}{d} \sum_{i=1}^d x_i,$$

$$\sigma^2 = \frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2.$$

Нормированный вектор:

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}.$$

Затем применяются обучаемые параметры масштаба и сдвига:

$$y_i = \gamma_i \hat{x}_i + \beta_i.$$

LayerNorm помогает стабилизировать обучение глубоких сетей.

4.21 LayerNorm vs BatchNorm

BatchNorm нормализует активации по batch dimension. Это хорошо работает во многих computer vision задачах, но может быть менее удобно для последовательностей переменной длины и autoregressive generation.

LayerNorm нормализует признаки внутри одного примера. Поэтому он лучше подходит для Transformer и NLP-задач.

BatchNorm	LayerNorm
Нормализация по batch.	Нормализация по признакам одного объекта.
Зависит от статистик батча.	Не зависит от других объектов в батче.
Часто используется в CNN.	Часто используется в Transformer.
Может быть неудобен для autoregressive моделей.	Удобен для последовательностей и генерации.

Table 5: Сравнение BatchNorm и LayerNorm.

4.22 Residual connections в Transformer

Хотя в презентации основной акцент сделан на attention и normalization, в стандартном Transformer каждый подслой используется вместе с residual connection.

Типичная структура:

$$x \rightarrow \text{Sublayer}(x),$$

$$y = \text{LayerNorm}(x + \text{Sublayer}(x)).$$

Residual connection помогает градиентам проходить через глубокую сеть и делает обучение стабильнее.

Для encoder block это обычно:

$$z = \text{LayerNorm}(x + \text{SelfAttention}(x)),$$

$$r = \text{LayerNorm}(z + \text{FeedForward}(z)).$$

4.23 Feed-Forward Network

В каждом encoder и decoder block есть позиционная feed-forward сеть.

Она применяется независимо к каждой позиции:

$$FFN(x) = W_2 \sigma(W_1 x + b_1) + b_2.$$

Обычно:

$$d_{ff} > d_{model}.$$

То есть сначала размерность расширяется, затем снова уменьшается до d_{model} .

Self-attention смешивает информацию между позициями, а feed-forward network преобразует представление каждой позиции отдельно.

4.24 Decoder Side

Decoder Transformer также состоит из нескольких блоков.

Каждый decoder block обычно содержит:

1. masked self-attention;
2. encoder-decoder attention;
3. feed-forward network.

4.25 Masked Self-Attention

В decoder-е нельзя позволять позиции t смотреть на будущие target tokens:

$$y_{t+1}, y_{t+2}, \dots$$

Иначе модель во время обучения увидит правильное будущее и задача генерации станет некорректной.

Поэтому используется **mask**.

Masked self-attention разрешает токenu t смотреть только на:

$$y_1, y_2, \dots, y_t.$$

То есть запрещены связи:

$$t \rightarrow j, \quad j > t.$$

В матрице attention это реализуется добавлением $-\infty$ к запрещённым позициям перед softmax:

$$\text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} + M \right),$$

где:

$$M_{ij} = \begin{cases} 0, & j \leq i, \\ -\infty, & j > i. \end{cases}$$

После softmax будущие позиции получают нулевой вес.



lecture04_decoder_mask.png

Figure 22: Masked self-attention в decoder-е запрещает смотреть на будущие токены.

4.26 Encoder-Decoder Attention

После masked self-attention decoder использует encoder-decoder attention.

В этом attention:

- queries берутся из decoder-а;
- keys и values берутся из выходов encoder-а.

То есть decoder на каждом шаге спрашивает:

какая часть source sequence сейчас важна для генерации target token?

Формально:

$$Q = H_{dec}W^Q,$$

$$K = H_{enc}W^K,$$

$$V = H_{enc}W^V.$$

Затем:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V.$$

Это аналог attention из seq2seq, но встроенный в Transformer.

4.27 Final Linear and Softmax Layer

На выходе decoder-а получаем вектор для каждой позиции target sequence.

Чтобы получить распределение по словарю, применяется linear layer:

$$o_t = W_{vocab}h_t + b_{vocab}.$$

Затем softmax:

$$P(y_t = w \mid y_{<t}, x) = \frac{\exp(o_{t,w})}{\sum_{w' \in \mathcal{V}} \exp(o_{t,w'})}.$$

Модель обучается предсказывать следующий токен в target sequence.

4.28 Сравнение RNN, CNN и Transformer

Архитектура	Плюсы	Минусы
RNN	Естественно моделирует последовательность, учитывает порядок.	Плохо параллелится, длинный путь между дальними токенами, vanishing gradients.
CNN	Хорошо ловит локальные паттерны, параллелится лучше RNN.	Для дальних зависимостей нужны большие окна или много слоёв.
Transformer	Короткий путь между любыми позициями, хорошо параллелится, моделирует дальние зависимости.	Self-attention имеет квадратичную сложность по длине последовательности.

Table 6: Сравнение RNN, CNN и Transformer для обработки последовательностей.

4.29 Сложность self-attention

Для последовательности длины n attention scores считаются для всех пар токенов:

$$QK^T \in \mathbb{R}^{n \times n}.$$

Поэтому сложность по времени и памяти относительно длины последовательности:

$$O(n^2).$$

Это важное ограничение Transformer для очень длинных последовательностей.

Однако для умеренных длин Transformer очень эффективен, потому что вычисления хорошо параллелятся на GPU.

4.30 Research challenges

В лекции упоминаются свойства и исследовательские вызовы attention-подходов:

- constant path length between any two positions;
- unbounded memory;
- trivial parallelization per layer;
- modeling self-similarity;
- relative attention;
- расширение attention-подходов на графы.

Ключевая идея: attention является более общим механизмом связывания элементов структуры. Его можно применять не только к текстам, но и к другим данным, где важны отношения между элементами.

4.31 Главные выводы

- Transformer основан на self-attention и не использует RNN или CNN как основной механизм обработки последовательности.
- Self-attention позволяет каждому токenu учитывать все остальные токены последовательности.
- Для каждого токена строятся Query, Key и Value векторы.
- Главная формула scaled dot-product attention:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V.$$

- Multi-head attention запускает несколько attention-механизмов параллельно.
- Positional encoding нужен, потому что self-attention сам по себе не знает порядок токенов.
- Sinusoidal positional encoding использует синусы и косинусы разных частот.
- LayerNorm стабилизирует обучение Transformer.
- Decoder использует masked self-attention, чтобы не смотреть в будущие target tokens.
- Transformer хорошо параллелится и имеет короткий путь между любыми позициями.

4.32 Что важно помнить к экзамену

1. Attention в seq2seq:

$$e_{t,i} = s_t^\top h_i,$$

$$\alpha^t = \text{softmax}(e^t),$$

$$a_t = \sum_{i=1}^N \alpha_i^t h_i.$$

2. Transformer предложен в статье *Attention is All You Need*.
3. В Transformer нет рекуррентности: основная операция — self-attention.
4. Encoder block состоит из:
 - self-attention;
 - feed-forward network;
 - residual connections;
 - layer normalization.
5. Query, Key, Value:

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V.$$

6. Scaled dot-product attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

7. Зачем делить на $\sqrt{d_k}$: чтобы стабилизировать значения перед softmax.

8. Multi-head attention:

$$head_r = \text{Attention}(Q_r, K_r, V_r),$$

$$\text{MultiHead} = \text{Concat}(head_1, \dots, head_H)W^O.$$

9. Positional encoding добавляется к token embeddings:

$$\tilde{x}_i = x_i + p_i.$$

10. Sinusoidal positional encoding:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right),$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right).$$

11. LayerNorm:

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}},$$

$$y_i = \gamma_i \hat{x}_i + \beta_i.$$

12. Masked self-attention запрещает decoder-у смотреть на будущие токены.

13. Главный плюс Transformer относительно RNN:

короткий путь между любыми двумя позициями и хорошая параллелизация.

14. Главный минус классического self-attention:

$$O(n^2)$$

по длине последовательности.

““

“latex

5 Лекция 5. Transfer Learning, BERT, GPT

5.1 План лекции

В этой лекции рассматриваются:

- повторение self-attention и Transformer;

- multi-head attention;
- positional encoding;
- layer normalization;
- transfer learning в NLP;
- OpenAI Transformer / GPT-подход;
- ELMo;
- BERT;
- pre-training и fine-tuning;
- BERT tokenization.

Главная идея лекции: современные NLP-модели обычно сначала обучаются на большой self-supervised задаче, например language modeling, а затем дообучаются на конкретную downstream-задачу. Это называется transfer learning.

5.2 Повторение: self-attention

Self-attention позволяет каждому токenu последовательности учитывать остальные токены той же последовательности.

Для входной матрицы embeddings:

$$X \in \mathbb{R}^{n \times d_{model}}$$

строятся три матрицы:

$$Q = XW^Q,$$

$$K = XW^K,$$

$$V = XW^V.$$

Здесь:

- Q — queries;
- K — keys;
- V — values;

- W^Q, W^K, W^V — обучаемые матрицы.

Для каждой пары токенов считается score:

$$QK^\top.$$

Затем scores масштабируются:

$$\frac{QK^\top}{\sqrt{d_k}}.$$

Деление на $\sqrt{d_k}$ нужно, чтобы значения перед softmax не становились слишком большими при большой размерности key-векторов.

После этого применяется softmax:

$$A = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right).$$

Итоговый выход self-attention:

$$Z = AV.$$

Главная формула:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right) V.$$



lecture05_self_attention_matrix.png

Figure 23: Матричная форма scaled dot-product attention.

5.3 Multi-Head Attention

В multi-head attention несколько attention-механизмов запускаются параллельно.

Для каждой головы r :

$$Q_r = XW_r^Q,$$

$$K_r = XW_r^K,$$

$$V_r = XW_r^V.$$

Затем считается:

$$head_r = \text{Attention}(Q_r, K_r, V_r).$$

Если всего H голов, то:

$$head_1, head_2, \dots, head_H$$

конкатенируются:

$$\text{Concat}(head_1, \dots, head_H),$$

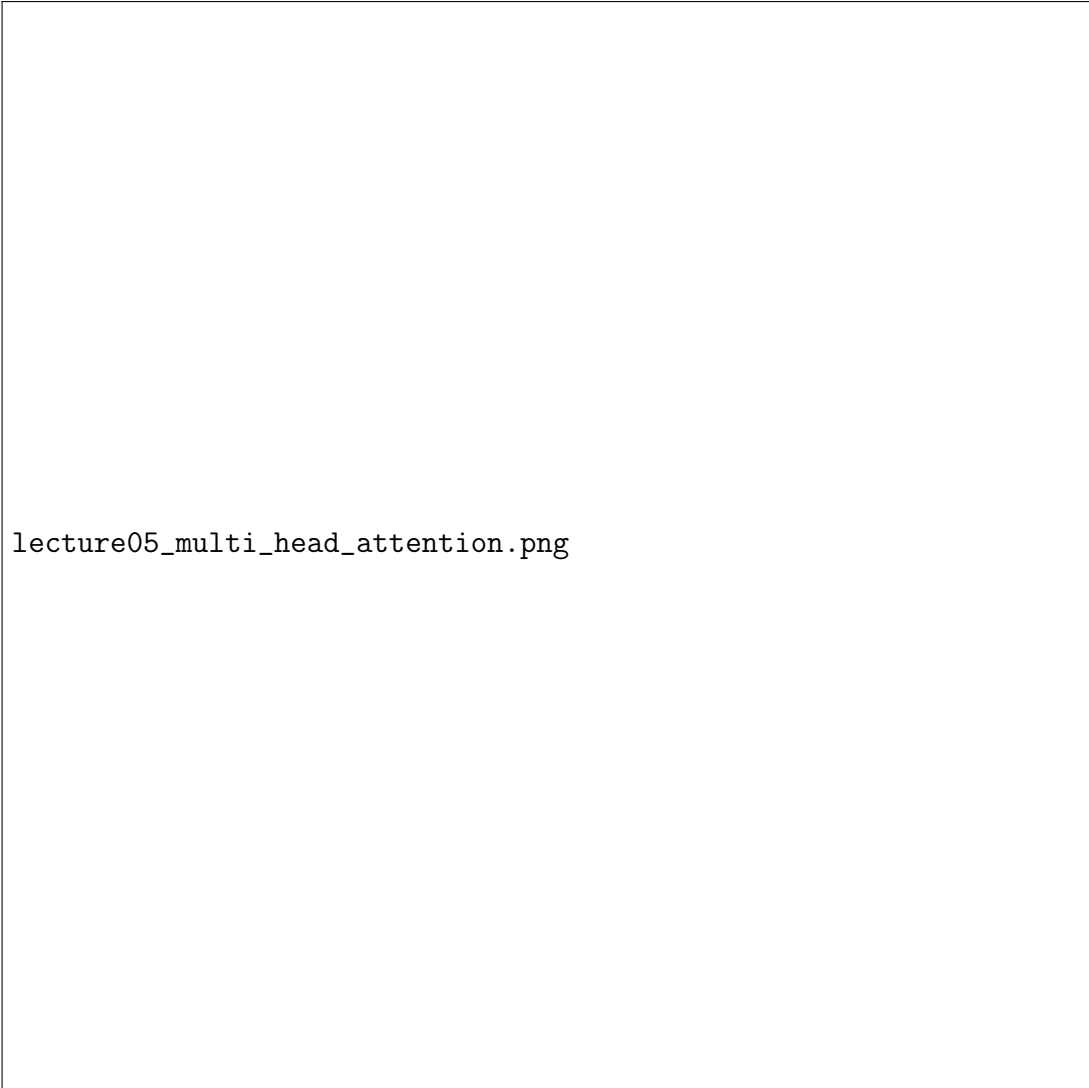
а затем проецируются обучаемой матрицей W^O :

$$\text{MultiHead}(Q, K, V) = \text{Concat}(head_1, \dots, head_H)W^O.$$

Смысл multi-head attention: разные головы могут смотреть на разные типы зависимостей в тексте.

Например:

- одна голова может отслеживать локальный контекст;
- другая — синтаксические связи;
- третья — связи между местоимением и объектом;
- четвёртая — дальние зависимости.



lecture05_multi_head_attention.png

Figure 24: Multi-head attention: несколько attention-голов работают параллельно, затем их выходы объединяются.

5.4 Positional Encoding

Self-attention сам по себе не содержит информации о порядке токенов. Поэтому Transformer добавляет к token embeddings positional encoding.

Для токена в позиции i :

$$\tilde{x}_i = x_i + p_i,$$

где:

- x_i — embedding токена;
- p_i — positional encoding;
- $\tilde{x}_i$ — итоговое входное представление.

Positional encoding нужен, чтобы модель могла различать предложения с одинаковыми словами, но разным порядком.

Например:

dog bites man

и

man bites dog

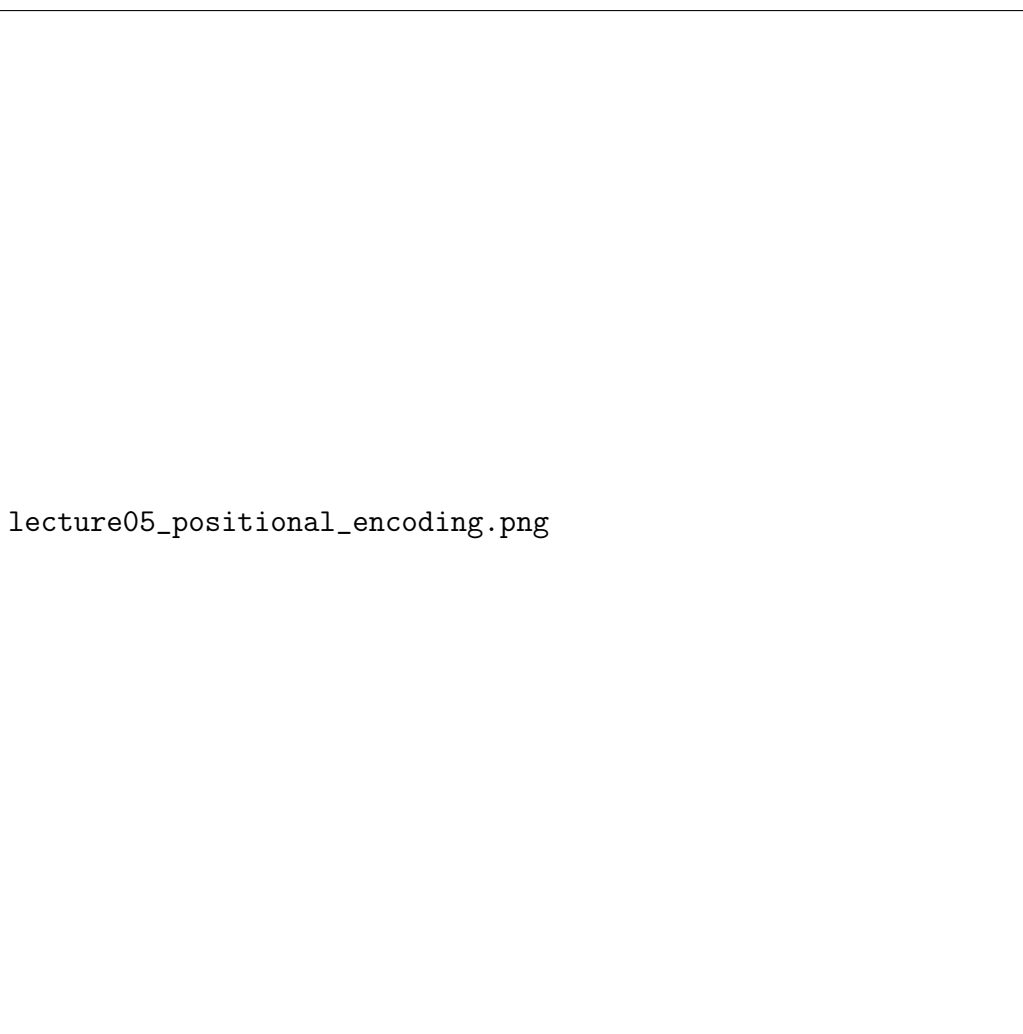
содержат одинаковые слова, но имеют разный смысл.

В классическом Transformer используется sinusoidal positional encoding:

$$PE_{(pos, 2i)} = \sin \left(\frac{pos}{10000^{2i/d_{model}}} \right),$$

$$PE_{(pos, 2i+1)} = \cos \left(\frac{pos}{10000^{2i/d_{model}}} \right).$$

Каждая размерность positional encoding соответствует синусоиде своей частоты.



lecture05_positional_encoding.png

Figure 25: Positional encoding добавляет информацию о позиции токена к его embedding.

5.5 Layer Normalization

В Transformer активно используется связка:

Add & Normalize.

Она означает residual connection и layer normalization.

Если есть вход x и некоторый подслой $\text{Sublayer}(x)$, то:

$$y = \text{LayerNorm}(x + \text{Sublayer}(x)).$$

Residual connection помогает передавать градиенты через глубокую сеть:

$$x + \text{Sublayer}(x).$$

Layer normalization стабилизирует распределение активаций.

Для вектора:

$$x = (x_1, x_2, \dots, x_d)$$

среднее:

$$\mu = \frac{1}{d} \sum_{i=1}^d x_i,$$

дисперсия:

$$\sigma^2 = \frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2.$$

Нормировка:

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}.$$

После этого применяются обучаемые scale и shift:

$$y_i = \gamma_i \hat{x}_i + \beta_i.$$

5.6 Transformer recap

Классический Transformer состоит из encoder stack и decoder stack.

Encoder block содержит:

- multi-head self-attention;
- add & norm;
- feed-forward network;
- add & norm.

Decoder block содержит:

- masked multi-head self-attention;
- add & norm;
- encoder-decoder attention;
- add & norm;
- feed-forward network;

- add & norm.

На выходе decoder-а применяются:

Linear $\rightarrow$ Softmax.

Это даёт распределение вероятностей по словарю.



Figure 26: Общая схема Transformer с encoder и decoder stack.

5.7 Transfer Learning в NLP

Transfer learning — подход, при котором модель сначала обучается на одной большой общей задаче, а затем используется или дообучается для других задач.

В NLP типичный сценарий:

1. pre-training на огромном корпусе без ручных меток;
2. fine-tuning на конкретной downstream-задаче.

Downstream-задачи:

- классификация текста;

- sentiment analysis;
- question answering;
- textual entailment;
- similarity;
- named entity recognition;
- machine translation;
- multiple choice.

Главное преимущество: для pre-training можно использовать огромные неразмеченные корпуса, а для конкретной задачи нужно намного меньше размеченных данных.

5.8 OpenAI Transformer и GPT-подход

Encoder-decoder Transformer хорошо подходит для машинного перевода. Но для многих задач, например sentence classification, можно использовать только часть Transformer.

OpenAI Transformer использует decoder-only подход.

Отличия от vanilla Transformer:

- нет encoder-a;
- decoder layers не имеют encoder-decoder attention sublayer;
- модель обучается предсказывать следующий токен;
- pre-training проводится на больших неразмеченных корпусах.

То есть модель обучается как language model:

$$P(x_1, x_2, \dots, x_T) = \prod_{t=1}^T P(x_t \mid x_{<t}).$$

Или в задаче next-token prediction:

$$P(x_t \mid x_1, \dots, x_{t-1}).$$

Так как модель обучается предсказывать следующий токен, она должна научиться представлять язык, синтаксис, семантику и типичные зависимости в тексте.

После pre-training модель можно использовать для downstream-задач.

5.9 Fine-tuning OpenAI Transformer

После pre-training слои модели уже умеют разумно обрабатывать язык. Для downstream-задачи можно добавить небольшую task-specific голову.

Например, для классификации текста:

text $\rightarrow$ OpenAI Transformer $\rightarrow$ FFNN + Softmax $\rightarrow$ class probabilities.

Для spam classification выход может быть:

$$P(\text{Spam} \mid x), \quad P(\text{Not Spam} \mid x).$$

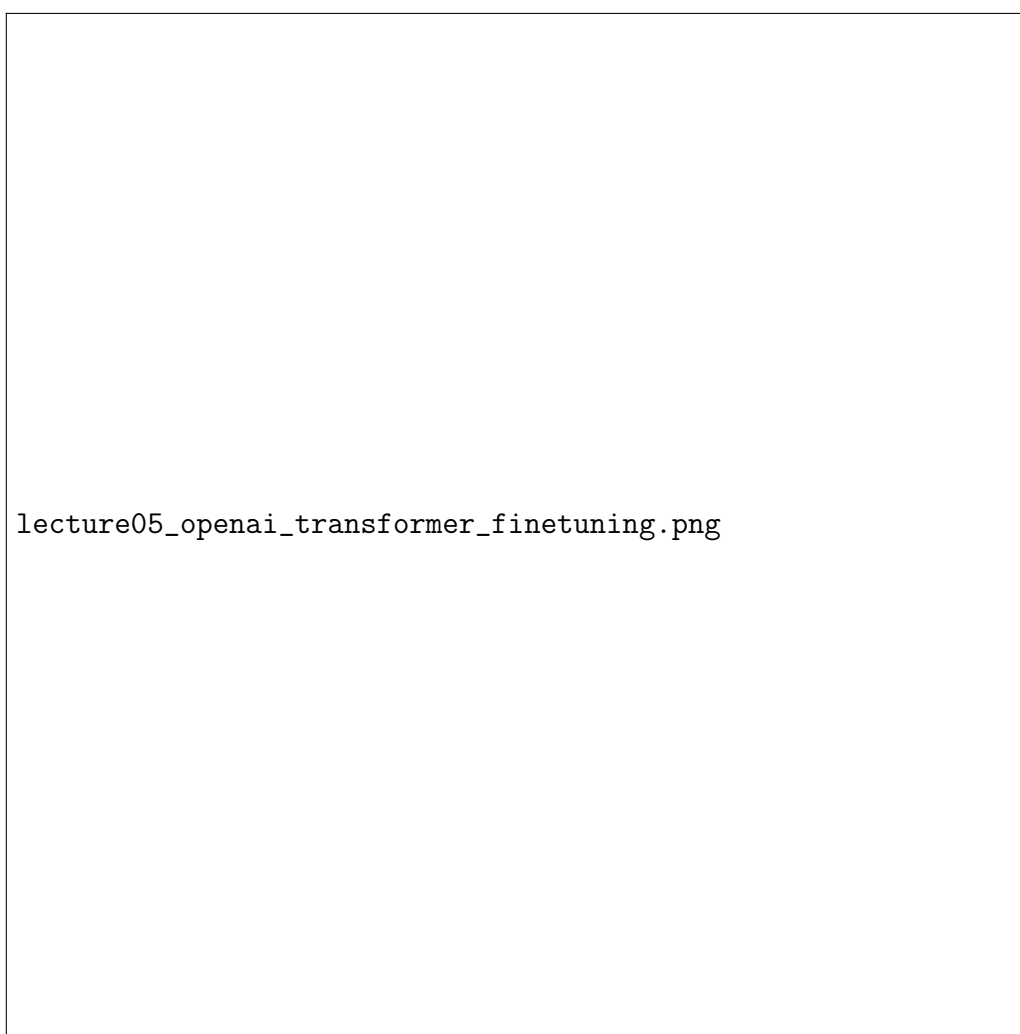


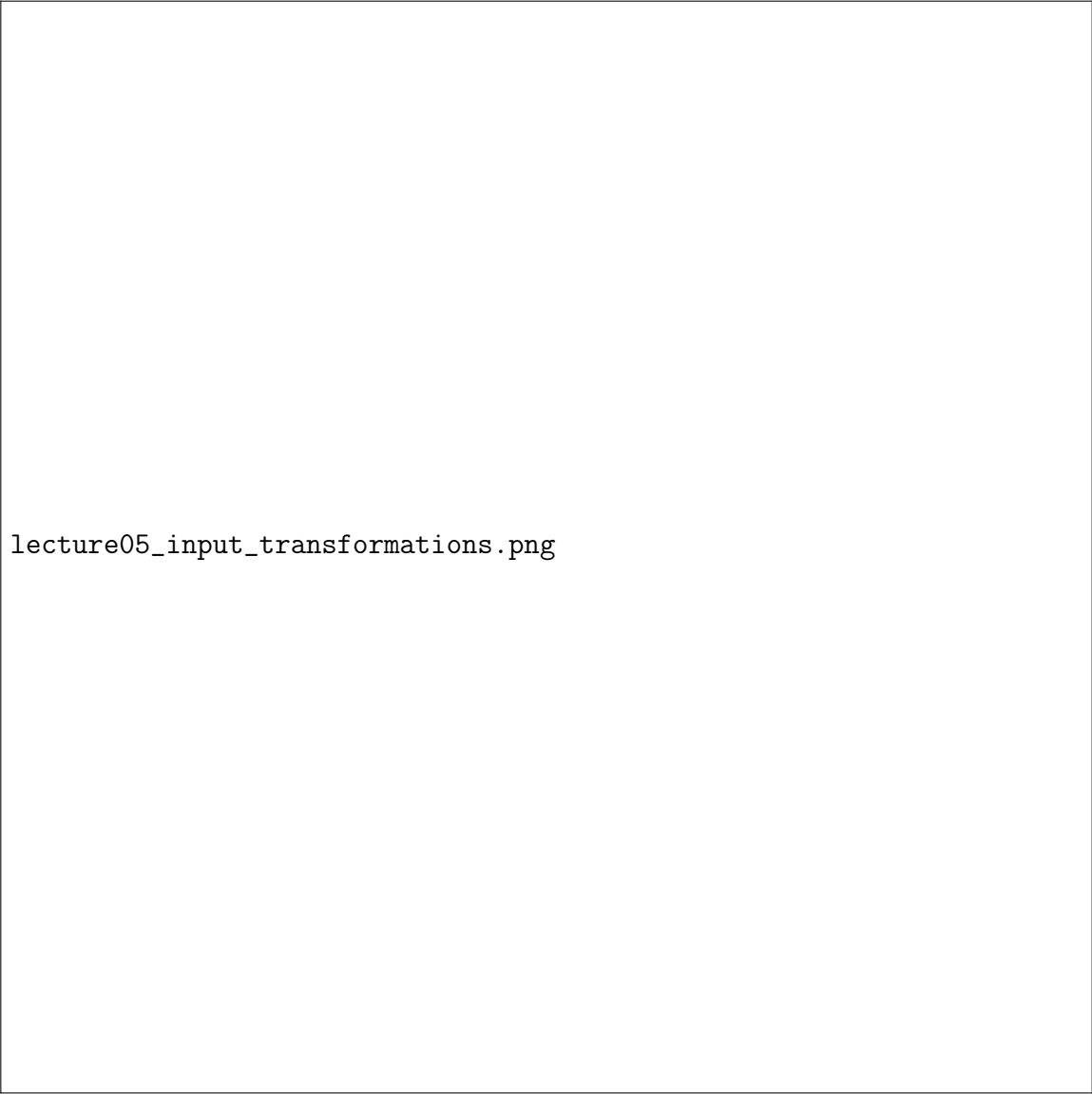
Figure 27: Fine-tuning OpenAI Transformer для downstream-задачи классификации.

5.10 Input transformations для разных задач

Одна и та же pretrained Transformer-модель может использоваться для разных задач, если правильно преобразовать вход.

Примеры:

- **Classification:** один текст подаётся в модель, затем используется выход для классификации.
- **Entailment:** подаются premise и hypothesis, разделённые специальным delimiter token.
- **Similarity:** подаются две версии пары текстов, затем их представления сравниваются.
- **Multiple choice:** для каждого варианта ответа формируется отдельная последовательность context + answer, затем выбирается лучший вариант.



lecture05_input_transformations.png

Figure 28: Input transformations позволяют применять одну pretrained модель к разным NLP-задачам.

5.11 ELMo: contextualized word embeddings

До контекстуальных embeddings одно слово обычно имело один фиксированный вектор.

Например, слово:

stick

имело бы один embedding независимо от контекста.

Но слово может иметь разные значения:

stick to the plan,

wooden stick,

stick in the mud.

ELMo решает эту проблему.

ELMo означает:

Embeddings from Language Models.

Главная идея:

Давать слову embedding, зависящий от контекста, в котором оно используется.

То есть:

$\text{embedding}(\text{word})$

становится функцией всего предложения:

$\text{embedding}(w_i \mid w_1, \dots, w_n).$

5.12 Bidirectional Language Models

ELMo использует bidirectional LSTM, обученную на language modeling task.

Bidirectional language model состоит из:

- forward language model;
- backward language model.

Forward LM моделирует вероятность последовательности слева направо:

$$P(t_1, t_2, \dots, t_N) = \prod_{k=1}^N P(t_k \mid t_1, \dots, t_{k-1}).$$

Backward LM моделирует вероятность справа налево:

$$P(t_1, t_2, \dots, t_N) = \prod_{k=1}^N P(t_k \mid t_{k+1}, \dots, t_N).$$

Идея: слово получает информацию и из левого, и из правого контекста.

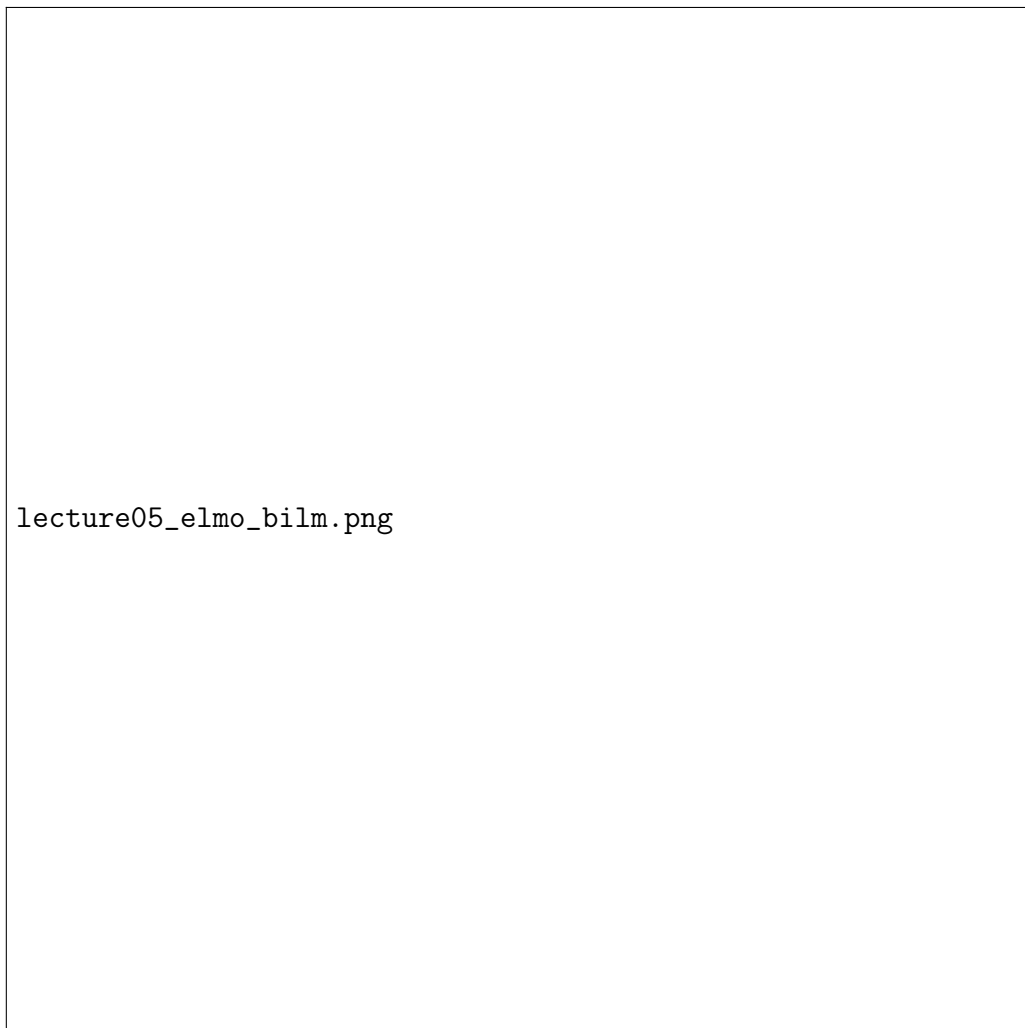


Figure 29: ELMo использует bidirectional language model: forward и backward LSTM.

5.13 ELMo pipeline

ELMo представляет слово как комбинацию hidden states разных слоёв biLSTM.

Пусть для токена k есть представления на разных слоях:

$$h_{k,0}, h_{k,1}, \dots, h_{k,L}.$$

Тогда ELMo embedding можно представить как взвешенную сумму:

$$\text{ELMo}_k = \gamma \sum_{j=0}^L s_j h_{k,j}.$$

Здесь:

- s_j — обучаемые task-specific веса слоёв;
- γ — обучаемый scale parameter;
- $h_{k,j}$ — представление токена k на слое j .

Разные слои могут кодировать разные типы информации:

- нижние слои чаще ловят синтаксис;
- верхние слои чаще ловят семантику.

ELMo можно добавить почти в любую нейронную NLP-модель через конкатенацию с обычным embedding layer:

$$x_i^{new} = [x_i; \text{ELMo}_i].$$

5.14 BERT

BERT расшифровывается как:

Bidirectional Encoder Representations from Transformers.

BERT использует encoder-only Transformer.

В отличие от GPT-like decoder-only моделей, BERT смотрит на контекст с обеих сторон. Поэтому он хорошо подходит для задач понимания текста:

- sentence classification;
- question answering;
- named entity recognition;
- entailment;
- similarity.

Для классификации текста схема выглядит так:

text $\rightarrow$ BERT $\rightarrow$ classifier $\rightarrow$ prediction.



Figure 30: BERT как encoder для downstream-классификации.

5.15 BERT Base и BERT Large

В лекции сравниваются варианты BERT Base и BERT Large.

Модель	Encoders	Units in FFN	Attention Heads
Transformer из лекции	6	512	8
BERT Base	12	768	12
BERT Large	24	1024	16

Table 7: Сравнение размеров Transformer, BERT Base и BERT Large.

BERT Large глубже и шире, поэтому обычно мощнее, но требует больше памяти и вычислений.

5.16 BERT inputs

BERT получает последовательность токенов фиксированной максимальной длины, например 512.

Для классификации в начало последовательности добавляется специальный токен:

[CLS].

Например:

[CLS], Help, Prince, Mayuko, . . .

Каждая позиция на выходе BERT выдаёт вектор.

Для sentence classification обычно берётся вектор первой позиции, соответствующий:

[CLS].

Этот вектор используется как представление всего текста:

h_{CLS} .

Затем:

$$\hat{y} = \text{softmax}(Wh_{CLS} + b).$$

То есть h_{CLS} подаётся в классификатор.

5.17 Почему BERT особенный

Главная особенность BERT — bidirectional pre-training.

Обычная autoregressive language model предсказывает следующий токен по левому контексту:

$$P(x_t \mid x_1, \dots, x_{t-1}).$$

BERT хочет использовать контекст с обеих сторон:

$$x_1, \dots, x_{t-1} \quad \text{и} \quad x_{t+1}, \dots, x_n.$$

Но если просто дать модели весь текст и попросить предсказывать токен, она увидит сам правильный токен.

Поэтому BERT использует masked language modeling.

5.18 BERT pre-training

BERT обучается на двух задачах:

1. Masked Language Modeling;
2. Next Sentence Prediction.

5.18.1 Masked Language Modeling

В Masked Language Modeling часть токенов во входе заменяется на специальный токен:

[MASK].

Модель должна восстановить скрытые токены по контексту.

Пример:

The animal did not cross the street because it was too tired

может быть превращён в:

The animal did not cross the [MASK] because it was too tired.

Модель должна предсказать:

street.

Преимущество MLM: модель может использовать и левый, и правый контекст.

5.18.2 Next Sentence Prediction

Чтобы BERT лучше понимал отношения между предложениями, добавляется задача:

Даны два предложения A и B . Является ли B вероятным следующим предложением после A ?

Это бинарная классификация:

$$y \in \{\text{IsNext}, \text{NotNext}\}.$$

Эта задача полезна для:

- question answering;
- natural language inference;
- задач на отношения между двумя предложениями.

5.19 BERT fine-tuning

После pre-training BERT можно дообучать для конкретных задач.

Общая схема:

pretrained BERT + small task-specific head $\rightarrow$ fine-tuning.

Примеры:

5.19.1 Text classification

Используется выход токена [CLS]:

$$h_{CLS} \rightarrow \text{Linear} \rightarrow \text{Softmax}.$$

5.19.2 Token classification

Для задач вроде named entity recognition используется выход каждой позиции:

$$h_1, h_2, \dots, h_n.$$

Для каждого токена предсказывается метка:

$$y_i = \text{softmax}(Wh_i + b).$$

5.19.3 Question answering

Для extractive QA модель предсказывает:

$$p_{start}$$

и

$$p_{end},$$

то есть начало и конец ответа в тексте.

5.20 BERT как feature extractor

BERT можно использовать не только для fine-tuning, но и как feature extractor.

В этом режиме:

- BERT не дообучается;
- через него прогоняют текст;

- берут hidden states одного или нескольких слоёв;
- эти векторы используют как признаки для другой модели.

Например, для каждого токена можно взять:

$$h_i^{(L)}$$

из последнего слоя или конкатенацию нескольких последних слоёв.

Fine-tuning обычно даёт лучшее качество, но feature extraction может быть полезен, если мало ресурсов или нужно быстро получить признаки.

5.21 BERT tokenization

BERT использует WordPiece tokenization.

Идея: слова разбиваются на подсловные единицы.

Пример из лекции:

Unaffable $\rightarrow$ un, ##aff, ##able.

Префикс:

##

означает, что фрагмент не является началом слова.

Преимущества WordPiece:

- можно обрабатывать редкие и неизвестные слова;
- словарь остаётся ограниченного размера;
- модель может работать с морфологически сложными словами;
- один и тот же словарь можно использовать для многих языков.

В лекции указано, что multilingual BERT использует один shared vocabulary для 104 языков:

119 547

WordPiece units.

Также отмечается:

- первые 106 символов — специальные константы вроде PAD и UNK;

- 36.5% словаря — non-initial word pieces;
- алфавит включает 9997 уникальных символов;
- символы представлены как word-initial и continuation symbols;
- остальные элементы словаря — многосимвольные word pieces разной длины.

5.22 Сравнение ELMo, GPT-like моделей и BERT

Модель	Архитектура	Основная идея
ELMo	BiLSTM language model.	Контекстуальные embeddings из forward и backward LM.
OpenAI Transformer / GPT-like	Decoder-only Transformer.	Pre-training через next-token prediction, затем fine-tuning.
BERT	Encoder-only Transformer.	Bidirectional representations через masked language modeling.

Table 8: Сравнение основных transfer learning подходов в NLP.

5.23 Главные выводы

- Transfer learning стал ключевой идеей современного NLP.
- Модель сначала обучается на большой self-supervised задаче, затем дообучается на downstream-задаче.
- OpenAI Transformer использует decoder-only Transformer и next-token prediction.
- ELMo даёт контекстуальные word embeddings на основе bidirectional LSTM.
- BERT использует encoder-only Transformer.
- BERT обучается через Masked Language Modeling и Next Sentence Prediction.
- Для классификации в BERT обычно используется выход токена [CLS].
- BERT можно использовать как fine-tuned модель или как feature extractor.
- WordPiece tokenization позволяет работать с редкими словами через подсловные единицы.

5.24 Что важно помнить к экзамену

1. Формулу self-attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

2. Что multi-head attention — это несколько attention heads с разными линейными проекциями.
3. Зачем нужен positional encoding: self-attention сам по себе не знает порядок токенов.
4. Что такое transfer learning:

pre-training $\rightarrow$ fine-tuning.

5. OpenAI Transformer / GPT-like подход:

$$P(x_1, \dots, x_T) = \prod_{t=1}^T P(x_t \mid x_{<t}).$$

6. ELMo:

$$\text{ELMo}_k = \gamma \sum_{j=0}^L s_j h_{k,j}.$$

7. Forward LM:

$$P(t_1, \dots, t_N) = \prod_{k=1}^N P(t_k \mid t_1, \dots, t_{k-1}).$$

8. Backward LM:

$$P(t_1, \dots, t_N) = \prod_{k=1}^N P(t_k \mid t_{k+1}, \dots, t_N).$$

9. BERT расшифровывается как:

Bidirectional Encoder Representations from Transformers.

10. BERT использует encoder-only Transformer.

11. BERT pre-training состоит из:

- Masked Language Modeling;
- Next Sentence Prediction.

12. Для sentence classification используется вектор токена:

[CLS].

13. WordPiece tokenization:

Unaffable $\rightarrow$ un, ##aff, ##able.

“ “`latex`

6 Лекция 6. Contrastive Learning

6.1 План лекции

В этой лекции рассматриваются:

- self-supervised learning;
- pretext tasks;
- отличие generative и self-supervised learning;
- оценка self-supervised методов;
- pretext tasks на изображениях;
- contrastive representation learning;
- InfoNCE loss;
- SimCLR;
- MoCo;
- CPC.

Главная идея лекции: можно обучать полезные представления без ручной разметки. Для этого модель решает автоматически сгенерированную задачу, а затем полученные признаки используются в downstream-задачах.

6.2 Self-supervised learning

Self-supervised learning — подход, при котором модель обучается без ручной разметки, но с supervised objective.

Ключевая идея:

- настоящих human labels нет;
- labels генерируются автоматически из самих данных;
- модель решает вспомогательную задачу;
- цель не сама вспомогательная задача, а хорошие признаки для downstream-задач.

Такая вспомогательная задача называется **pretext task**.

Например, для изображения можно автоматически создать задачу:

- предсказать угол поворота;
- восстановить скрытую часть изображения;
- собрать “jigsaw puzzle”;
- раскрасить grayscale image.

Для каждой такой задачи labels можно получить автоматически. Например, если мы сами повернули изображение на 90° , то label равен 90° .

6.3 Pretext task

Pretext task — искусственная задача, которая создаётся не ради итогового качества на этой задаче, а ради обучения полезного feature extractor.

Пусть есть много неразмеченных данных:

$$x_1, x_2, \dots, x_N.$$

Мы автоматически строим пары:

$$(\tilde{x}_i, \tilde{y}_i),$$

где:

- $\tilde{x}_i$ — изменённый объект;
- $\tilde{y}_i$ — автоматически известная метка.

После этого обучаем модель:

$$f_{\theta}(\tilde{x}) \rightarrow \tilde{y}.$$

Но на самом деле нас интересует не предсказание $\tilde{y}$, а внутреннее представление:

$$h = f_{\theta}(x).$$

Это представление затем используется в downstream-задачах.

6.4 Generative vs self-supervised learning

В generative learning модель часто пытается восстановить данные на низком уровне, например пиксели изображения.

Но для downstream-задач не всегда нужно уметь генерировать точные pixel-level details. Человеку часто достаточно высокоуровневого понимания объекта.

Поэтому self-supervised learning часто пытается обучить не генератор деталей, а encoder, который извлекает semantic features.

Пример: чтобы понять, что изображён доллар, необязательно помнить все мелкие детали купюры. Важнее выучить структуру и смысловые признаки.

6.5 Как оценивать self-supervised learning

Обычно качество self-supervised pretext task не является главным критерием.

Например, нам не так важно, насколько идеально модель предсказывает поворот изображения. Важно, насколько хорошие признаки она выучила.

Типичный pipeline оценки:

1. взять много неразмеченных данных;
2. обучить feature extractor на pretext task;
3. заморозить или частично заморозить feature extractor;
4. добавить простую supervised модель;
5. обучить её на небольшом количестве размеченных данных;
6. оценить на target task.

Target tasks:

- classification;
- detection;

- segmentation;
- tracking;
- retrieval.



Figure 31: Оценка self-supervised learning: признаки обучаются на pretext task, затем используются в downstream-задаче.

6.6 Pretext tasks from image transformations

Ранние self-supervised методы часто строились вокруг image transformations.

Идея: если модель умеет решать задачу, требующую понимания структуры изображения, то она вынуждена выучить полезные visual features.

Примеры:

- predict rotations;

- predict relative patch locations;
- solve jigsaw puzzles;
- inpainting;
- colorization;
- video colorization.

6.7 Pretext task: predict rotations

В задаче предсказания поворота изображение поворачивается на один из нескольких углов:

$$0^\circ, 90^\circ, 180^\circ, 270^\circ.$$

Модель получает повернутое изображение и должна предсказать угол:

$$y \in \{0, 1, 2, 3\}.$$

Это обычная задача классификации на 4 класса.

Гипотеза: чтобы определить правильную ориентацию объекта, модель должна иметь некоторый visual commonsense, то есть понимать, как объекты обычно выглядят.

Например, чтобы понять, что птица повернута на 90° , модель должна распознавать форму птицы, положение головы, лап, фона и т.д.



lecture06_rotation_prediction.png

Figure 32: Self-supervised pretext task: предсказание угла поворота изображения.

6.8 Transfer learned features

После pretraining на rotation prediction можно проверить, насколько хорошие признаки выучила модель.

Пример evaluation protocol:

- self-supervised learning на ImageNet без labels;
- затем fine-tuning на Pascal VOC 2007;
- оценка classification, detection, segmentation.

Такие эксперименты показывают, что даже простая pretext task может дать признаки лучше случайной инициализации.

Но такие признаки всё ещё часто уступают supervised pretraining на ImageNet.

6.9 Pretext task: predict relative patch locations

В этой задаче из изображения выбираются два patch-а. Один patch считается центральным, а второй находится в одной из соседних позиций.

Модель должна предсказать относительное положение второго patch-а относительно первого.

Например, label может быть:

$$y \in \{1, 2, \dots, 8\},$$

где 8 классов соответствуют восьми соседним областям вокруг центрального patch-а.

Чтобы решить такую задачу, модель должна понимать spatial structure изображения.

Например, если один patch содержит глаз кота, а другой — ухо, модель может понять, где ухо обычно расположено относительно глаза.

6.10 Pretext task: solving jigsaw puzzles

В jigsaw pretext task изображение разбивается на несколько фрагментов, затем фрагменты перемешиваются.

Модель должна восстановить правильный порядок или предсказать перестановку.

Пусть изображение разбито на 3×3 patch-и. Тогда существует много возможных перестановок:

$$9!$$

На практике обычно выбирается подмножество перестановок, чтобы задача была вычислительно разумной.

Смысл задачи: модель должна понимать, какие части изображения должны находиться рядом.

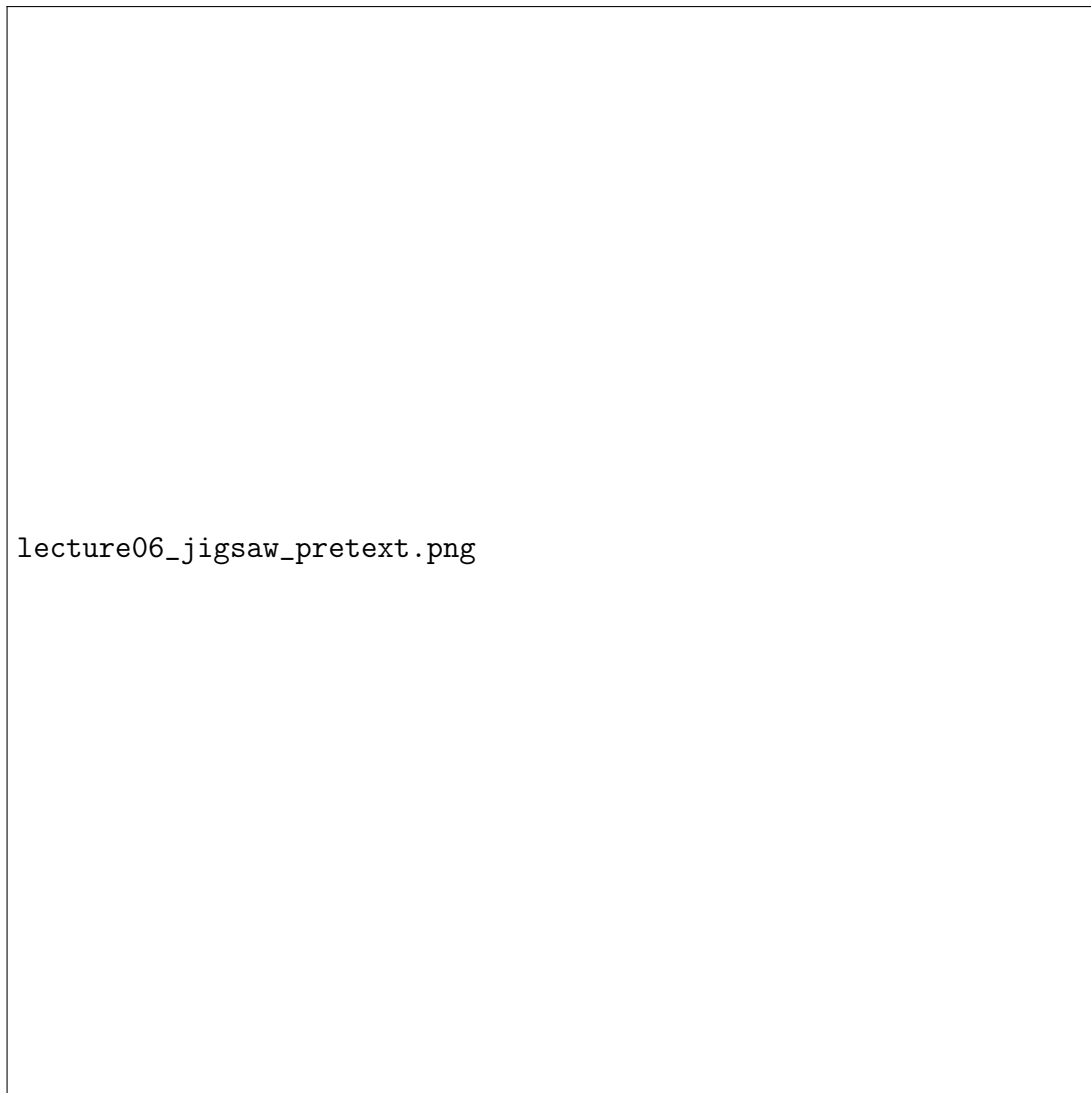


Figure 33: Self-supervised pretext task: решение jigsaw puzzle по patch-ам изображения.

6.11 Pretext task: inpainting

Inpainting — задача восстановления отсутствующей части изображения.

Вход:

$$x_{masked},$$

где часть изображения закрыта mask-ой.

Модель должна предсказать недостающие пиксели:

$$\hat{x}_{missing}.$$

Смысл: чтобы восстановить скрытый фрагмент, модель должна понимать контекст изображения.

Например, если закрыта часть здания, модель должна использовать окна, двери, стены и перспективу вокруг.

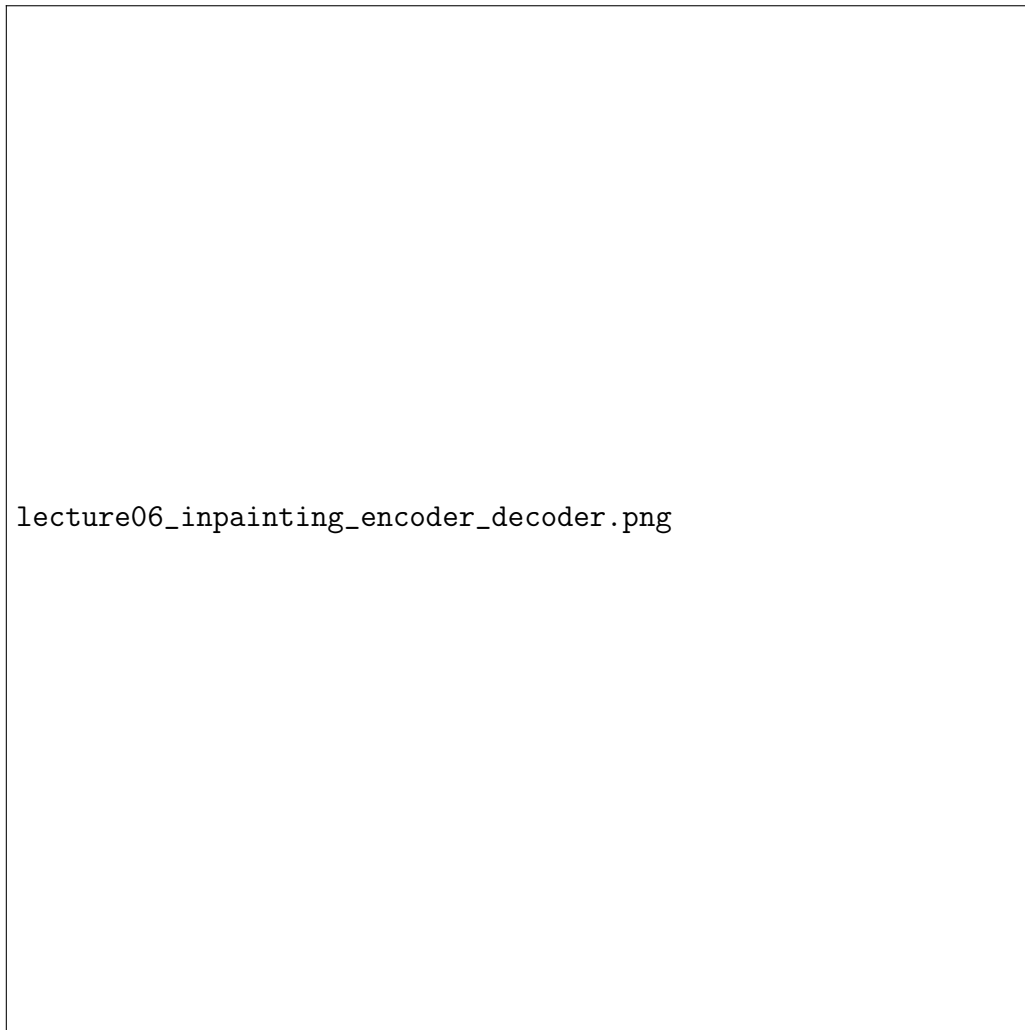


Figure 34: Inpainting: модель восстанавливает скрытую область изображения по контексту.

6.12 Loss для inpainting

В лекции приводится loss, состоящий из двух частей:

$$L(x) = L_{recon}(x) + L_{adv}(x).$$

Здесь:

- L_{recon} — reconstruction loss;
- L_{adv} — adversarial loss.

Reconstruction loss измеряет близость восстановленного изображения к ground truth:

$$L_{recon}(x) = \|M \odot (x - F_{\theta}((1 - M) \odot x))\|_2^2.$$

Здесь:

- M — mask скрытой области;
- $1 - M$ — видимая часть изображения;
- F_{θ} — модель восстановления;
- $\odot$ — поэлементное умножение.

Adversarial loss нужен, чтобы восстановленные изображения выглядели более реалистично.

Идея:

$$L_{adv}$$

учит модель не просто минимизировать pixel-wise ошибку, а генерировать визуально правдоподобные детали.

6.13 Pretext task: image coloring

В colorization исходное изображение переводится в grayscale, а модель должна восстановить цвет.

В цветовом пространстве LAB:

- L channel содержит яркость;
- a, b channels содержат цветовую информацию.

Вход:

$$X \in \mathbb{R}^{H \times W \times 1}$$

— grayscale L -channel.

Выход:

$$\hat{Y} \in \mathbb{R}^{H \times W \times 2}$$

— предсказанные a, b -channels.

Модель должна восстановить цветовую информацию по структуре изображения.

Это заставляет её понимать объекты: небо обычно голубое, трава зелёная, банан жёлтый и т.д.

6.14 Split-brain autoencoder

Split-brain autoencoder строится на идее cross-channel prediction.

Входные каналы изображения делятся на две части:

$$X_1, X_2.$$

Одна часть используется для предсказания другой:

$$F_1(X_1) \rightarrow \hat{X}_2,$$

$$F_2(X_2) \rightarrow \hat{X}_1.$$

Затем признаки из двух ветвей можно объединить.

Идея: модель должна выучить представления, которые связывают разные типы информации об одном и том же объекте.

6.15 Pretext task: video coloring

В video colorization используется temporal coherence.

Идея:

Если объект движется в видео, его цвет должен быть согласован во времени.

Модель получает reference frame с цветом и target frame, для которого нужно предсказать цвет.

Она учится устанавливать соответствия между пикселями или регионами reference и target frames в learned feature space.

Предсказанный цвет target pixel можно получить как weighted sum цветов reference frame:

$$\hat{c}_i = \sum_j \alpha_{ij} c_j^{ref}.$$

Здесь:

- α_{ij} — attention-like вес между target pixel i и reference pixel j ;
- c_j^{ref} — цвет reference pixel.

Интересный эффект: tracking может возникать без явной разметки. Если модель научилась переносить цвета между кадрами, она фактически научилась сопоставлять объекты во времени.

6.16 Проблемы handcrafted pretext tasks

Pretext tasks на трансформациях изображений полезны, но имеют ограничения:

- каждую задачу нужно придумывать вручную;
- задача может быть слишком специфичной;
- learned representations могут быть привязаны к конкретной pretext task;
- признаки не всегда хорошо переносятся на разные downstream-задачи.

Это мотивирует более общий подход:

contrastive representation learning.

6.17 Contrastive Representation Learning

Contrastive learning обучает encoder так, чтобы похожие объекты были близко в representation space, а непохожие — далеко.

Есть:

- reference sample x ;
- positive sample x^+ ;
- negative samples $x_1^-, \dots, x_{N-1}^-$.

Цель:

$$\text{score}(f(x), f(x^+))$$

должен быть большим, а

$$\text{score}(f(x), f(x^-))$$

должен быть маленьким.

То есть encoder f должен давать представления, в которых positive pair близка, а negative pairs далеки.

6.18 Positive и negative samples

Positive pair — два объекта, которые должны считаться похожими.

В vision contrastive learning positive pair часто создаётся через data augmentation одного и того же изображения:

$$x^+ = t_2(x), \quad x = t_1(x),$$

где t_1, t_2 — разные случайные augmentations.

Примеры augmentations:

- random cropping;
- color distortion;
- random blur;
- horizontal flip.

Negative samples — другие изображения в batch-е или в memory queue.

6.19 InfoNCE loss

Пусть есть один positive sample x^+ и $N - 1$ negative samples.

Обозначим score:

$$s(x, x') = \text{score}(f(x), f(x')).$$

Тогда contrastive loss:

$$\mathcal{L} = -\log \frac{\exp(s(x, x^+))}{\exp(s(x, x^+)) + \sum_{i=1}^{N-1} \exp(s(x, x_i^-))}.$$

Эта функция потерь называется **InfoNCE loss**.

Её можно понимать как cross entropy loss для N -way softmax classifier:

из N кандидатов нужно выбрать один positive sample.

То есть contrastive learning превращается в задачу классификации: найти правильную пару среди неправильных.

InfoNCE также интерпретируется как нижняя оценка mutual information между $f(x)$ и $f(x^+)$. Чем больше negative sample size N , тем tighter bound.

6.20 Cosine similarity

Часто score function задаётся через cosine similarity:

$$\text{sim}(u, v) = \frac{u^\top v}{\|u\| \|v\|}.$$

Тогда loss можно записать как:

$$\mathcal{L} = -\log \frac{\exp(\text{sim}(z, z^+)/\tau)}{\exp(\text{sim}(z, z^+)/\tau) + \sum_{i=1}^{N-1} \exp(\text{sim}(z, z_i^-)/\tau)}.$$

Здесь:

τ

— temperature parameter.

Temperature управляет sharpness softmax-a:

- маленькая τ делает распределение более резким;
- большая τ делает распределение более сглаженным.

6.21 SimCLR

SimCLR — простая и эффективная framework для contrastive learning.

Основные шаги:

1. взять изображение x ;
2. применить две случайные augmentations;
3. получить две views:

$$\tilde{x}_i, \tilde{x}_j;$$

4. пропустить их через encoder f ;
5. получить representations:

$$h_i = f(\tilde{x}_i), \quad h_j = f(\tilde{x}_j);$$

6. пропустить через projection head g :

$$z_i = g(h_i), \quad z_j = g(h_j);$$

7. применить contrastive loss в z -пространстве.



lecture06_simclr_positive_pairs.png

Figure 35: SimCLR: positive samples создаются как две разные augmentations одного изображения.

6.22 SimCLR loss в mini-batch

В mini-batch берётся N исходных изображений. Для каждого создаются две augmentations, поэтому всего получается:

$$2N$$

views.

Для каждой view остальные $2N - 1$ views являются кандидатами:

- одна из них — positive pair;
- остальные $2N - 2$ — negatives.

Loss считается для каждой из $2N$ views как reference и затем усредняется.

Особенность SimCLR: все non-positive samples в batch-е используются как negatives.

6.23 Projection head в SimCLR

SimCLR использует projection network:

$$g(\cdot).$$

Encoder выдаёт:

$$h = f(x),$$

projection head выдаёт:

$$z = g(h).$$

Contrastive loss применяется к z , но для downstream-задач обычно используется h .

Это важно, потому что contrastive objective может выбрасывать часть информации, полезной для downstream-задач, ради инвариантности к augmentations.

Projection head позволяет:

- обучать contrastive пространство z ;
- сохранить более богатое representation h ;
- улучшить downstream performance.

6.24 Large batch size в SimCLR

SimCLR сильно зависит от большого batch size.

Причина: negatives берутся из batch-а. Чем больше batch, тем больше negative samples:

$$\text{large batch} \Rightarrow \text{many negatives}.$$

Много negatives делает contrastive task информативнее.

Но большой batch size требует много памяти, особенно при backpropagation. Поэтому в ImageNet экспериментах SimCLR часто требует distributed training на TPU.

6.25 MoCo

MoCo расшифровывается как **Momentum Contrastive Learning**.

Главная проблема SimCLR: чтобы иметь много negatives, нужен большой batch.

MoCo решает это через очередь negative samples.

Основные идеи MoCo:

- хранить running queue of keys;
- использовать query encoder для текущих примеров;
- использовать key encoder для negative samples;
- обновлять key encoder медленно через momentum update;
- отделить размер mini-batch от числа negatives.



Figure 36: MoCo: очередь negative samples позволяет использовать много negatives без огромного batch size.

6.26 Momentum update в MoCo

В MoCo есть два encoder-а:

- query encoder с параметрами θ_q ;
- key encoder с параметрами θ_k .

Query encoder обновляется через backpropagation.

Key encoder обновляется momentum rule:

$$\theta_k \leftarrow m\theta_k + (1 - m)\theta_q.$$

Здесь:

$$m \in [0, 1)$$

— momentum coefficient, обычно близкий к 1.

Такой update делает key encoder стабильным, что важно, потому что queue содержит representations, вычисленные на предыдущих шагах.

6.27 MoCo vs SimCLR

SimCLR	MoCo
Negatives берутся из mini-batch.	Negatives хранятся в queue.
Для большого числа negatives нужен огромный batch.	Negative sample size отделён от batch size.
Требуется большой памяти.	Меньше memory footprint.
Простая end-to-end схема.	Есть momentum key encoder.
Сильно выигрывает от projection head и strong augmentation.	MoCo v2 также использует projection head и strong augmentation.

Table 9: Сравнение SimCLR и MoCo.

6.28 MoCo v2

MoCo v2 объединяет идеи SimCLR и MoCo:

- от SimCLR:
 - non-linear projection head;
 - strong data augmentation;
- от MoCo:

- momentum-updated queue;
- большое число negatives без огромного batch size.

Главный вывод из сравнения:

- non-linear projection head важен;
- strong augmentation важна;
- decoupling mini-batch size и negative sample size позволяет MoCo v2 быть эффективным при меньшем batch size.

6.29 Instance vs sequence contrastive learning

SimCLR и MoCo — примеры **instance contrastive learning**.

В них positive pair обычно создаётся из одного объекта через augmentation.

Но contrastive learning можно применять и к последовательностям.

Пример:

Contrastive Predictive Coding, CPC.

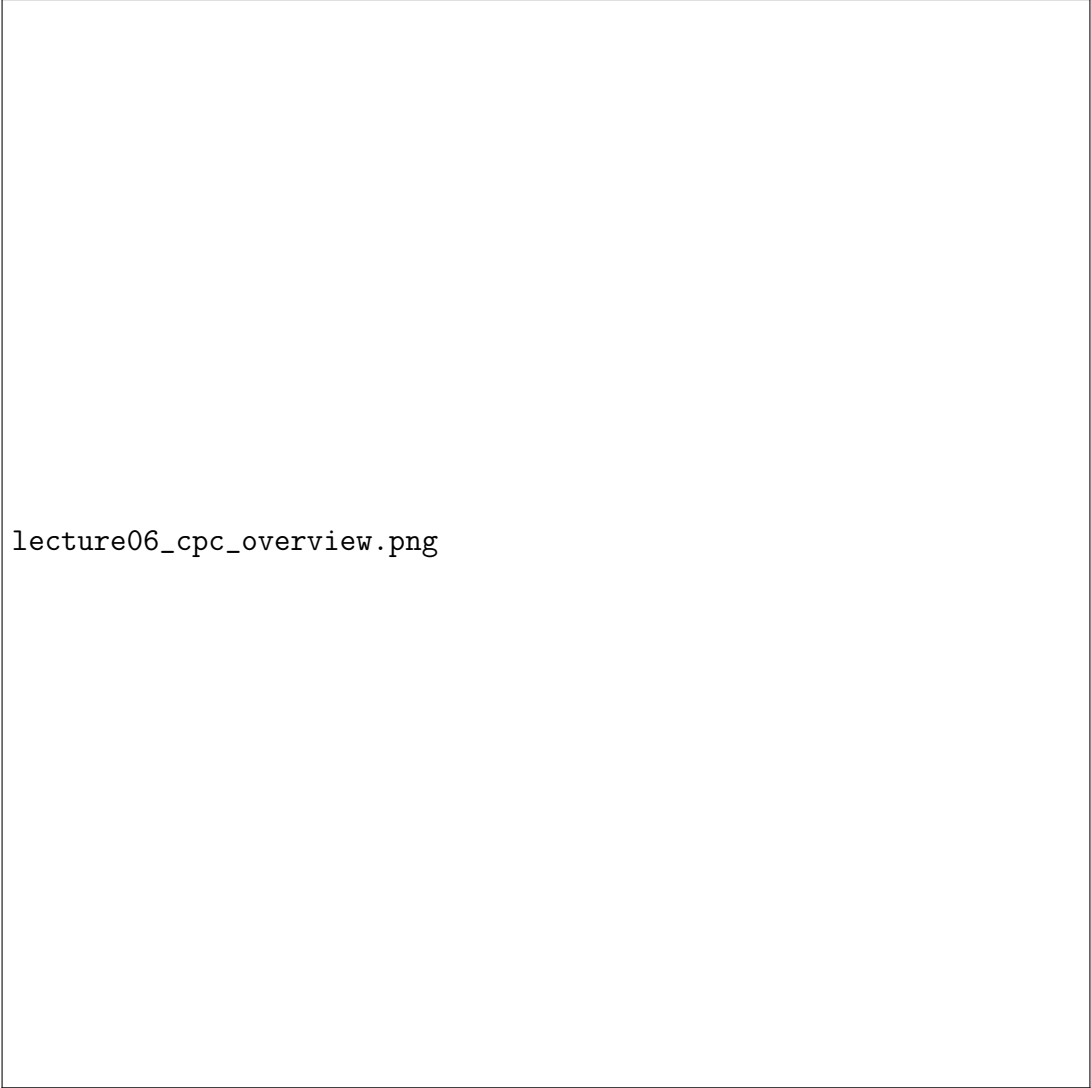
6.30 Contrastive Predictive Coding

CPC расшифровывается как:

Contrastive Predictive Coding.

Название отражает три идеи:

- **Contrastive**: модель различает правильные и неправильные future samples.
- **Predictive**: модель предсказывает будущие паттерны по текущему контексту.
- **Coding**: модель учит полезные feature vectors, или codes.



lecture06_cpc_overview.png

Figure 37: Contrastive Predictive Coding: модель учится отличать правильное будущее от неправильного.

6.31 CPC pipeline

CPC работает с последовательностью:

$$x_1, x_2, \dots, x_T.$$

6.31.1 Шаг 1: encode samples

Каждый элемент последовательности кодируется encoder-ом:

$$z_t = g_{enc}(x_t).$$

6.31.2 Шаг 2: summarize context

Контекст до момента t агрегируется autoregressive model:

$$c_t = g_{ar}(z_{\leq t}).$$

В оригинальной статье используется GRU-RNN.

6.31.3 Шаг 3: contrastive prediction

Модель пытается предсказать future code:

$$z_{t+k}$$

по context code:

$$c_t.$$

Score function зависит от горизонта k :

$$score_k(c_t, z_{t+k}) = z_{t+k}^\top W_k c_t.$$

Здесь W_k — обучаемая матрица для шага k .

Затем применяется InfoNCE loss: правильный future sample должен иметь высокий score, неправильные future samples — низкий.

6.32 Применения CPC

CPC можно применять к разным типам последовательностей:

- audio;
- изображения как последовательность patch-ей;
- видео;
- текст;
- временные ряды.

Для изображений можно разбить картинку на patch-и и рассматривать строки patch-ей сверху вниз как последовательность.

Тогда верхняя часть изображения является context, а нижнюю нужно предсказать contrastive образом.

6.33 Contrastive learning и mutual information

InfoNCE можно рассматривать как способ максимизировать нижнюю оценку mutual information между представлениями positive pair.

Интуитивно:

$$I(f(x); f(x^+))$$

должна быть большой.

То есть representation reference sample должно содержать информацию о representation positive sample.

Чем больше negative samples, тем сложнее задача и тем tighter bound на mutual information.

6.34 Другие примеры contrastive/self-supervised learning

В лекции также упоминаются современные и смежные методы:

- MoCo v3;
- Masked Autoencoders;
- CLIP;
- Dense Object Net;
- DINO;
- DINO v2.

Особенно важный пример — **CLIP**.

CLIP использует contrastive learning между изображениями и текстовыми описаниями.

Positive pair:

(image, matching caption).

Negative pairs:

(image, wrong captions).

Модель учит общее image-text representation space.

6.35 Главные выводы

- Self-supervised learning позволяет учиться без ручной разметки.
- Pretext task создаётся автоматически и используется для обучения хороших признаков.
- Качество pretext task не так важно, как качество learned representations на downstream tasks.
- Image transformation pretext tasks включают rotation prediction, jigsaw, inpainting и colorization.
- Handcrafted pretext tasks могут быть слишком специфичными.
- Contrastive learning предлагает более общий принцип: positive pairs должны быть близко, negative pairs далеко.
- InfoNCE loss можно понимать как cross entropy для выбора positive sample среди N кандидатов.
- SimCLR создаёт positive pairs через data augmentation и использует negatives из batch-a.
- Projection head в SimCLR помогает сохранить полезную информацию в encoder representation.
- MoCo использует queue negatives и momentum encoder, чтобы не требовать огромный batch.
- CPC применяет contrastive learning к последовательностям и учится предсказывать будущее.

6.36 Что важно помнить к экзамену

1. Определение self-supervised learning: labels для pretext task генерируются автоматически.
2. Что pretext task нужна не сама по себе, а для обучения feature extractor.
3. Как оценивают self-supervised метод:

pretext training $\rightarrow$ feature extractor $\rightarrow$ downstream task.

4. Примеры pretext tasks:

- rotation prediction;

- relative patch location;
- jigsaw puzzle;
- inpainting;
- colorization.

5. Общая идея contrastive learning:

$score(f(x), f(x^+))$ высокий,

$score(f(x), f(x^-))$ низкий.

6. InfoNCE loss:

$$\mathcal{L} = -\log \frac{\exp(s(x, x^+))}{\exp(s(x, x^+)) + \sum_{i=1}^{N-1} \exp(s(x, x_i^-))}.$$

7. Cosine similarity:

$$\text{sim}(u, v) = \frac{u^\top v}{\|u\| \|v\|}.$$

8. Temperature-scaled loss:

$$\mathcal{L} = -\log \frac{\exp(\text{sim}(z, z^+)/\tau)}{\exp(\text{sim}(z, z^+)/\tau) + \sum_i \exp(\text{sim}(z, z_i^-)/\tau)}.$$

9. SimCLR:

- positive pair = две augmentations одного изображения;
- negatives = остальные samples в batch;
- используется projection head $g(\cdot)$;
- нужен большой batch size.

10. MoCo:

- использует queue negative keys;
- использует momentum key encoder;
- decouples batch size and number of negatives.

11. Momentum update:

$$\theta_k \leftarrow m\theta_k + (1 - m)\theta_q.$$

12. CPC:

$$z_t = g_{enc}(x_t),$$

$$c_t = g_{ar}(z_{\leq t}),$$

$$score_k(c_t, z_{t+k}) = z_{t+k}^\top W_k c_t.$$

13. Отличие instance contrastive learning от sequence contrastive learning:

- instance-level: сравниваются views одного объекта;
- sequence-level: правильное будущее сравнивается с неправильным.

“ “`latex`

7 Лекция 7. Introduction to Reinforcement Learning

7.1 План лекции

В этой лекции рассматриваются:

- постановка задачи Reinforcement Learning;
- отличие RL от supervised и unsupervised learning;
- агент, среда, действие, состояние, награда;
- multi-armed bandits как простая RL-постановка;
- MDP formalism;
- связь RL с психологией;
- cross-entropy method;
- табличный и приближённый cross-entropy method;
- базовые определения value functions и Q-functions;
- переход от табличных методов к аппроксимации функций.

Главная идея лекции: Reinforcement Learning изучает обучение агента через взаимодействие со средой. В отличие от supervised learning, у агента обычно нет правильных ответов для каждого состояния; он получает только feedback в виде reward и должен научиться стратегии, максимизирующей долгосрочную суммарную награду.

7.2 Supervised Learning и Reinforcement Learning

В supervised learning есть обучающая выборка:

$$(x_i, y_i),$$

где:

- $x_i \in \mathcal{X}$ — объект;
- $y_i \in \mathcal{Y}$ — правильный ответ;
- $L(\hat{y}, y)$ — loss function;
- $f \in \mathcal{F}$ — модель из некоторого семейства.

Цель supervised learning:

$$f^* = \arg \min_f L(f(x), y).$$

Обычно предполагается, что объекты выборки независимы и одинаково распределены, то есть i.i.d. Также loss часто дифференцируем, что позволяет использовать gradient-based optimization.

В Reinforcement Learning ситуация другая.

Обычно:

- нет reference answers;
- непонятно, какое действие правильно в каждом конкретном состоянии;
- loss часто трудно явно сформулировать;
- действия агента влияют на будущие наблюдения;
- качество действия может проявиться не сразу, а через много шагов.

Например, если мы хотим научить робота ходить, у нас нет правильного действия для каждого состояния суставов. Мы можем только задать reward: например, робот получает награду за продвижение вперёд и штраф за падение.

7.3 Базовая схема Reinforcement Learning

В RL есть две основные сущности:

- **agent** — обучаемая система;
- **environment** — среда, с которой агент взаимодействует.

На каждом шаге:

1. среда выдаёт агенту observation или state;
2. агент выбирает action;
3. действие влияет на среду;
4. среда возвращает reward и новое observation/state.

Схема:

$$s_t \rightarrow a_t \rightarrow r_t, s_{t+1}.$$

Здесь:

- s_t — state или observation в момент t ;
- a_t — action агента;
- r_t — reward;
- s_{t+1} — следующее состояние.



Figure 38: Основной цикл Reinforcement Learning: агент наблюдает состояние, выбирает действие и получает feedback от среды.

7.4 Policy

Policy — стратегия поведения агента.

Policy задаёт, какое действие агент выбирает в данном состоянии.

Детерминированная policy:

$$\pi(s) = a.$$

Стохастическая policy:

$$\pi(a \mid s) = P(\text{take action } a \text{ in state } s).$$

В стохастическом случае policy задаёт распределение вероятностей по действиям.

Например, если в состоянии s доступно три действия:

$$a_1, a_2, a_3,$$

то policy может быть:

$$\pi(a_1 | s) = 0.2, \quad \pi(a_2 | s) = 0.5, \quad \pi(a_3 | s) = 0.3.$$

7.5 Примеры RL-задач

7.5.1 Управление роботом

Для робота:

- observation — данные с датчиков, изображение, положение суставов;
- action — управляющие сигналы моторам;
- reward — скорость движения, устойчивость, штраф за падение.

7.5.2 Управление вертолётom

Для вертолёта:

- observation — accelerometer, gyroscope, engine data;
- action — изменение скорости вращения, угла и других управляющих параметров;
- reward — мера качества полёта.

7.5.3 Видеоигры

Для Atari или DOOM:

- observation — изображение или последовательность кадров;
- action — move, fire, turn и т.д.;
- reward — score, health, progress.

7.6 Ключевые вопросы Reinforcement Learning

В RL сразу возникают важные вопросы.

7.6.1 Что такое оптимальное действие?

Можно пытаться максимизировать reward на следующем шаге:

$$\max r_t.$$

Но иногда действие, которое хорошо прямо сейчас, плохо в долгосрочной перспективе. Поэтому обычно цель RL — максимизировать долгосрочную суммарную награду:

$$\sum_t r_t.$$

7.6.2 Explore or exploit?

Агент должен выбирать между:

- **exploitation** — использовать уже известную хорошую стратегию;
- **exploration** — пробовать новые действия, чтобы найти потенциально лучшую стратегию.

Проблема:

- если всегда exploit, агент может никогда не найти лучшую стратегию;
- если слишком много explore, агент теряет reward на плохих действиях.

Это называется exploration-exploitation tradeoff.

7.7 Multi-Armed Bandits

Multi-armed bandit — одна из простейших RL-постановок.

Есть несколько действий, или “ручек автомата”:

$$a \in \{1, 2, \dots, K\}.$$

На каждом шаге агент выбирает одно действие и получает reward.

Особенность bandit setting:

- обычно нет сложного состояния;
- действие не меняет состояние среды в сложном смысле;
- главная задача — понять, какое действие даёт наибольшую ожидаемую награду.

Цель:

$$a^* = \arg \max_a \mathbb{E}[r \mid a].$$

Bandits хорошо иллюстрируют exploration-exploitation tradeoff: нужно пробовать разные действия, но постепенно выбирать лучшие.

7.8 MDP formalism

Более общая постановка RL описывается через Markov Decision Process, или MDP.

MDP задаётся набором:

$$(\mathcal{S}, \mathcal{A}, P, R).$$

Где:

- $\mathcal{S}$ — множество состояний;
- $\mathcal{A}$ — множество действий;
- $P(s_{t+1} \mid s_t, a_t)$ — dynamics или transition probability;
- R — reward function.

В лекции используются обозначения:

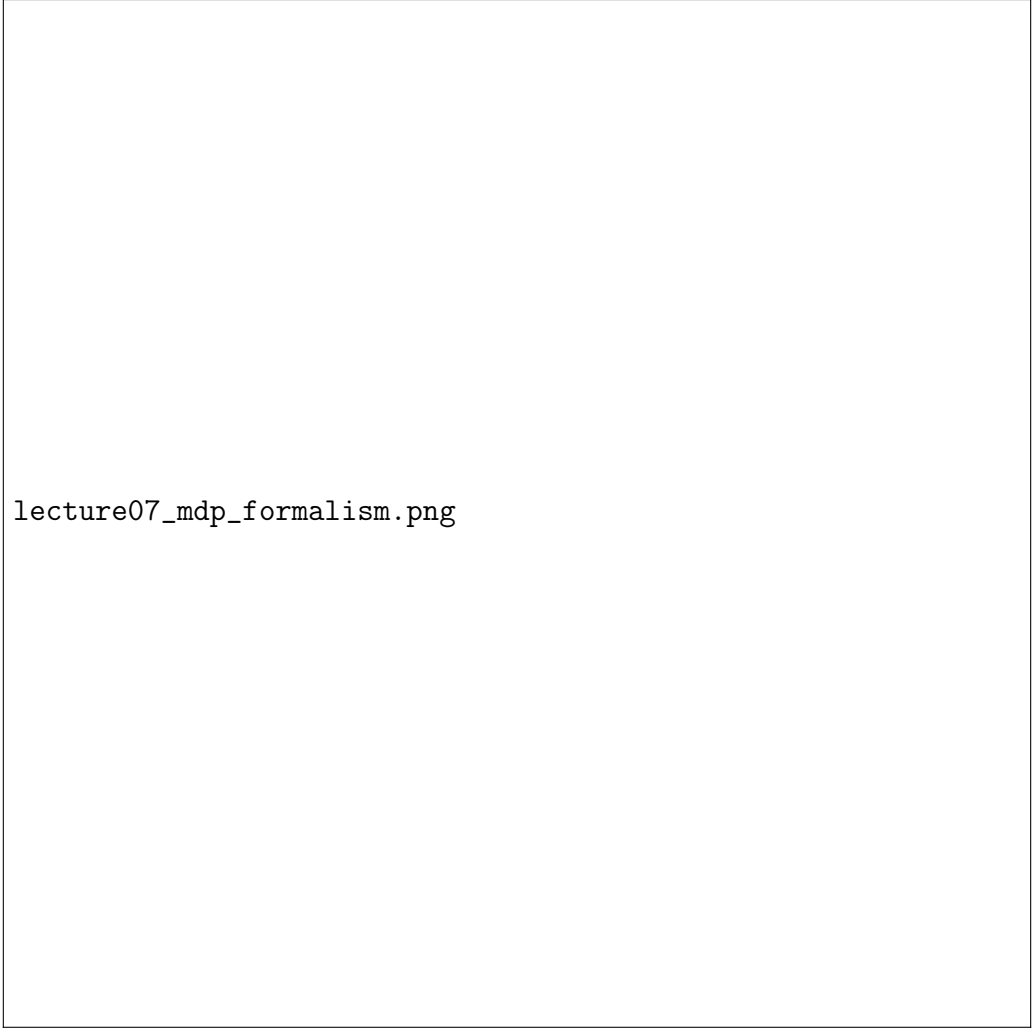
$$s \in \mathcal{S},$$

$$a \in \mathcal{A},$$

$$r \in \mathbb{R}.$$

Динамика среды:

$$P(s_{t+1} \mid s_t, a_t).$$



lecture07_mdp_formalism.png

Figure 39: MDP formalism: агент выбирает действие в состоянии, среда возвращает reward и следующее состояние.

7.9 Markov property

Ключевое свойство MDP — Markov property.

Оно означает, что будущее зависит от прошлого только через текущее состояние и текущее действие:

$$P(s_{t+1} \mid s_t, a_t, \dots, s_0, a_0) = P(s_{t+1} \mid s_t, a_t).$$

Иными словами, если мы знаем s_t и a_t , то вся предыдущая история уже не нужна для предсказания s_{t+1} .

Это сильное предположение. На практике observation может не содержать всей информации о состоянии. Тогда возникает частично наблюдаемая задача, и агенту может понадобиться память.

7.10 Total reward

Суммарная награда за session или episode:

$$R = \sum_t r_t.$$

Если используется discount factor γ , то discounted return:

$$G_t = \sum_{t'=t}^T \gamma^{t'-t} r_{t'}.$$

Здесь:

$$\gamma \in [0, 1].$$

Смысл γ :

- если γ близко к 0, агент фокусируется на ближайшей награде;
- если γ близко к 1, агент учитывает дальнейшее будущее.

Цель RL:

$$\pi^* = \arg \max_{\pi} \mathbb{E}_{\pi}[R].$$

То есть нужно найти policy, максимизирующую expected cumulative reward.

7.11 Value function и Q-function

Value function показывает, насколько хорошо находиться в состоянии s , если дальше следовать policy π :

$$V^{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid s_t = s].$$

Q-function показывает, насколько хорошо выполнить действие a в состоянии s , а затем следовать policy π :

$$Q^{\pi}(s, a) = \mathbb{E}_{\pi}[G_t \mid s_t = s, a_t = a].$$

Эти функции важны, потому что позволяют оценивать не только immediate reward, но и будущие последствия действий.

7.12 Bellman relations

Для value и Q-functions можно записать рекуррентные соотношения.

Для Q-function:

$$Q^\pi(s, a) = \mathbb{E}_{s'}[r_t + \gamma V^\pi(s_{t+1})].$$

Также:

$$Q^\pi(s, a) = \mathbb{E}_{s', a' \sim \pi}[r_t + \gamma Q^\pi(s_{t+1}, a_{t+1})].$$

Оптимальная Q-function:

$$Q^*(s, a)$$

не хуже любой другой policy:

$$Q^*(s, a) \geq Q^\pi(s, a)$$

для всех π, s, a .

Оптимальная policy:

$$\pi^*(s) = \arg \max_a Q^*(s, a).$$

Bellman optimality equation:

$$Q^*(s_t, a) = \mathbb{E}_{s_{t+1}} \left[r_t + \gamma \max_{a'} Q^*(s_{t+1}, a') \right].$$

7.13 Связь RL с психологией

В лекции упоминаются психологические аналогии:

- classical conditioning;
- operant conditioning;
- Thorndike experiment;
- Skinner box.

Идея operant conditioning: поведение может усиливаться или ослабляться в зависимости от последствий.

Например, если крыса нажимает рычаг и получает еду, вероятность такого поведения растёт. Это похоже на RL: действие, приведшее к положительному reward, становится более вероятным.

7.14 Как максимизировать reward

Общая идея policy optimization:

1. сыграть несколько sessions с текущей policy;
2. получить rewards;
3. обновить policy на основе feedback;
4. повторить.

То есть обучение происходит через trial and error:

$$\pi_0 \rightarrow \text{episodes} \rightarrow \text{rewards} \rightarrow \pi_1 \rightarrow \dots$$

7.15 Cross-Entropy Method

Cross-Entropy Method, CEM — простой policy search метод.

Идея:

- сгенерировать много episodes;
- выбрать лучшие episodes, называемые elite sessions;
- обновить policy так, чтобы она чаще повторяла действия из elite sessions;
- повторять процесс.

CEM не требует дифференцируемой среды и может использоваться как простой baseline для RL.

7.16 Cross-Entropy Method: tabular case

В табличном случае policy можно представить матрицей:

$$\pi(a \mid s) = A_{s,a}.$$

Каждая строка соответствует состоянию, каждый столбец — действию.

Для каждого состояния:

$$\sum_a A_{s,a} = 1.$$

Алгоритм:

1. инициализировать policy matrix A ;

2. сэмплировать N sessions;
3. выбрать M elite sessions с наибольшими rewards;
4. обновить policy по state-action pairs из elite sessions;
5. повторить.



Figure 40: Cross-Entropy Method в табличном случае: policy matrix обновляется по elite sessions.

7.17 Обновление policy в tabular CEM

Пусть elite sessions содержат пары:

$$(s_0, a_0), (s_1, a_1), \dots$$

Новая policy оценивается как частота выбора действия a в состоянии s среди elite sessions:

$$\pi_{\text{new}}(a \mid s) = \frac{\sum_{(s_t, a_t) \in \text{Elite}} \mathbb{I}[s_t = s] \mathbb{I}[a_t = a]}{\sum_{(s_t, a_t) \in \text{Elite}} \mathbb{I}[s_t = s]}.$$

Иными словами:

$$\pi_{\text{new}}(a \mid s) = \frac{\text{how many times took action } a \text{ at state } s}{\text{how many times was at state } s}.$$



Figure 41: Обновление tabular policy: вероятность действия равна его частоте среди elite sessions.

7.18 Проблема больших пространств состояний

Tabular policy работает только если число состояний и действий небольшое.

В реальных задачах пространство состояний может быть огромным или бесконечным.

Например, если observation — изображение из игры, то число возможных состояний гигантское:

$$|\mathcal{S}| \approx 2^{210 \cdot 160 \cdot 8 \cdot 3}.$$

Хранить таблицу для всех состояний невозможно.
Поэтому нужно переходить от таблиц к функциям:

$$\pi(a \mid s) = f_{\theta}(s, a)$$

или

$$Q(s, a) \approx Q_{\theta}(s, a).$$

7.19 Approximate Cross-Entropy Method

В approximate CEM policy задаётся не таблицей, а параметрической моделью.
Например:

$$\pi(a \mid s) = f_{\theta}(a, s).$$

Модель получает state и выдаёт вероятности действий.
Это может быть:

- logistic regression;
- random forest classifier;
- neural network;
- другая модель классификации.

Алгоритм:

1. сэмплировать N sessions с текущей policy;
2. выбрать M elite sessions;
3. собрать training set:

$$\text{states} \rightarrow \text{actions};$$
4. обучить модель предсказывать elite actions по elite states.

То есть в кодовой форме:

```
model.fit(elite_states, elite_actions).
```

Функция потерь:

$$\pi_{\text{new}} = \arg \max_{\pi} \sum_{(s_t, a_t) \in \text{Elite}} \log \pi(a_t | s_t).$$



Figure 42: Approximate Cross-Entropy Method: elite states становятся объектами, elite actions — target values.

7.20 Continuous action space

Если action space непрерывное, policy может задавать распределение над действиями.

Например, нормальное распределение:

$$\pi(a | s) = \mathcal{N}(\mu_{\theta}(s), \sigma_{\gamma}(s)).$$

Здесь:

- $\mu_{\theta}(s)$ — среднее действия;

- $\sigma_\gamma(s)$ — стандартное отклонение;
- оба параметра могут предсказываться моделями.

В простом случае σ можно считать константой, а модель обучает только среднее действие.

Тогда approximate CEM становится похож на regression:

$$s \mapsto a.$$

7.21 Useful ideas для CEM

В лекции перечислены практические идеи:

- использовать elite sessions из нескольких прошлых итераций;
- регуляризовать policy через entropy;
- сэмплировать sessions параллельно;
- использовать память агента, например RNN.

7.21.1 Использование прошлых elite sessions

Если использовать elite sessions за последние 3–5 итераций, опыт прошлых итераций сохраняется.

Плюс:

- меньше забывания полезного поведения.

Минус:

- сходимость может быть медленнее на простых средах.

7.21.2 Entropy regularization

Низкая entropy policy означает слабую exploration.

Если policy слишком быстро становится почти детерминированной, агент может перестать исследовать альтернативные действия.

Entropy regularization помогает сохранять разнообразие действий:

$$H(\pi(\cdot | s)) = - \sum_a \pi(a | s) \log \pi(a | s).$$

Чем выше entropy, тем более случайна policy.

7.22 Q-learning

В конце презентации вводится Q-learning.

Q-learning пытается выучить оптимальную Q-function:

$$Q^*(s, a).$$

Обновление Q-learning:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right].$$

Здесь:

- α — learning rate;
- γ — discount factor;
- выражение в квадратных скобках — temporal difference error.

Temporal difference error:

$$\delta_t = r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t).$$

7.23 Q-learning как MSE minimization

Q-learning можно рассматривать как минимизацию ошибки между текущим $Q(s_t, a_t)$ и target:

$$y_t = r_t + \gamma \max_{a'} Q(s_{t+1}, a').$$

Loss:

$$L = \left(r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right)^2.$$

Важно: target часто рассматривается как константа относительно текущего gradient step. Это нужно для стабильности обучения.

7.24 Approximate Q-learning

Когда state space большой, таблицу $Q(s, a)$ хранить нельзя.

Тогда Q-function аппроксимируется моделью:

$$Q_\theta(s, a).$$

Например, нейронной сетью.

Переход:

before: remember $Q(s, a)$ for all states and actions,

now: approximate $Q(s, a)$ with a function.

Objective:

$$\arg \min_{\theta} \left(Q_{\theta}(s_t, a_t) - \left[r_t + \gamma \max_{a'} Q_{\theta}(s_{t+1}, a') \right] \right)^2.$$

Для изображений можно использовать CNN, которая принимает кадр игры и выдаёт Q-values для всех действий.

7.25 Basic Deep Q-Learning

В Deep Q-Learning Q-function задаётся нейронной сетью.

Для Atari-подобных задач:

- вход — изображение или stack кадров;
- несколько convolutional layers извлекают visual features;
- dense layers формируют представление;
- выход — Q-values для всех actions.

Если действий K , то сеть выдаёт:

$$Q(s, a_1), Q(s, a_2), \dots, Q(s, a_K).$$

Действие выбирается как:

$$a^* = \arg \max_a Q(s, a).$$

7.26 RL vs Supervised Learning

Supervised Learning	Reinforcement Learning
Учится приближать reference answers.	Учится оптимальной стратегии через trial and error.
Нужны правильные ответы y .	Нужен feedback на действия агента.
Модель не влияет на входные данные.	Действия агента влияют на среду и будущие наблюдения.
Обычно есть фиксированный dataset.	Данные зависят от текущей policy.
Цель обычно локальная: минимизировать loss на примерах.	Цель долгосрочная: максимизировать cumulative reward.

Table 10: Сравнение supervised learning и reinforcement learning.

7.27 RL vs Unsupervised Learning

Unsupervised Learning	Reinforcement Learning
Изучает структуру данных.	Учится оптимальной стратегии через взаимодействие.
Feedback не требуется.	Нужен reward или другой feedback.
Модель не влияет на данные.	Действия агента меняют будущие состояния.
Обычно нет понятия действия.	Action является центральным объектом.
Цель — representation, clustering, density и т.д.	Цель — maximize expected cumulative reward.

Table 11: Сравнение unsupervised learning и reinforcement learning.

7.28 Reward formulation

Формулировка reward сильно влияет на поведение агента.

Если reward задан плохо, агент может научиться нежелательному поведению.

Например:

- если награждать робота только за скорость, он может падать, но двигаться быстро;
- если награждать игрового агента только за очки, он может эксплуатировать баги среды;
- если штрафовать слишком сильно, агент может выбрать бездействие.

Поэтому reward design — одна из центральных практических проблем RL.

7.29 Главные выводы

- Reinforcement Learning отличается от supervised learning отсутствием правильных ответов для каждого состояния.
- Агент учится через взаимодействие со средой и feedback в виде reward.
- Policy задаёт стратегию выбора действий:

$$\pi(a \mid s).$$

- Цель RL — максимизировать expected cumulative reward:

$$\pi^* = \arg \max_{\pi} \mathbb{E}_{\pi}[R].$$

- MDP формализует RL через states, actions, rewards и transition dynamics.
- Markov property означает, что будущее зависит только от текущего состояния и действия.
- Exploration-exploitation tradeoff — фундаментальная проблема RL.
- Cross-Entropy Method обновляет policy по elite sessions.
- В tabular CEM policy хранится как state-action matrix.
- В approximate CEM policy задаётся параметрической моделью.
- Для больших state spaces нужны function approximators.
- Q-learning учит оценку полезности state-action pair:

$$Q(s, a).$$

7.30 Что важно помнить к экзамену

1. Основной цикл RL:

$$s_t \rightarrow a_t \rightarrow r_t, s_{t+1}.$$

2. Policy:

$$\pi(a \mid s) = P(\text{take action } a \text{ in state } s).$$

3. MDP:

$$s \in \mathcal{S}, \quad a \in \mathcal{A}, \quad r \in \mathbb{R}, \quad P(s_{t+1} \mid s_t, a_t).$$

4. Markov property:

$$P(s_{t+1} \mid s_t, a_t, \dots, s_0, a_0) = P(s_{t+1} \mid s_t, a_t).$$

5. Total reward:

$$R = \sum_t r_t.$$

6. Discounted return:

$$G_t = \sum_{t'=t}^T \gamma^{t'-t} r_{t'}.$$

7. Цель RL:

$$\pi^* = \arg \max_{\pi} \mathbb{E}_{\pi}[R].$$

8. Value function:

$$V^{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid s_t = s].$$

9. Q-function:

$$Q^{\pi}(s, a) = \mathbb{E}_{\pi}[G_t \mid s_t = s, a_t = a].$$

10. Bellman optimality equation:

$$Q^*(s, a) = \mathbb{E}_{s_{t+1}} \left[r_t + \gamma \max_{a'} Q^*(s_{t+1}, a') \right].$$

11. Cross-Entropy Method:

- sample N sessions;
- select M elite sessions;
- update policy по elite state-action pairs;
- repeat.

12. Tabular CEM update:

$$\pi_{\text{new}}(a \mid s) = \frac{\#\{(s_t, a_t) \in \text{Elite} : s_t = s, a_t = a\}}{\#\{(s_t, a_t) \in \text{Elite} : s_t = s\}}.$$

13. Approximate CEM:

$$\pi_{\text{new}} = \arg \max_{\pi} \sum_{(s_t, a_t) \in \text{Elite}} \log \pi(a_t \mid s_t).$$

14. Q-learning update:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right].$$

15. Главное отличие RL от supervised learning: действия агента влияют на будущие данные.

“ “\latex

8 Лекция 8. Bellman Equations

8.1 План лекции

В этой лекции рассматриваются:

- повторение MDP formalism;
- reward hypothesis;
- return и discounting;
- reward design;
- state-value function $v(s)$;
- action-value function $q(s, a)$;
- Bellman expectation equations;
- Bellman optimality equations;
- policy evaluation;
- policy improvement;
- generalized policy iteration;
- policy iteration и value iteration.

Главная идея лекции: если известна динамика среды, то задачу поиска оптимальной стратегии можно решать через value functions и уравнения Беллмана. Эти уравнения связывают ценность состояния или действия с immediate reward и ценностью будущих состояний.

8.2 Повторение: MDP formalism

Markov Decision Process, MDP задаётся кортежем:

$$(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}),$$

где:

- $\mathcal{S}$ — множество состояний;
- $\mathcal{A}$ — множество действий;
- $\mathcal{P} : \mathcal{S} \times \mathcal{A} \rightarrow \Delta(\mathcal{S})$ — transition function;
- $\mathcal{R} : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ — reward function.

Transition function задаёт вероятность следующего состояния:

$$p(s_{t+1} \mid s_t, a_t).$$

Reward function задаёт ожидаемую награду:

$$\mathbb{E}_R[R(s_t, a_t) \mid s_t, a_t].$$

Ключевое предположение MDP — **Markov property**:

$$p(r_t, s_{t+1} \mid s_0, a_0, r_0, \dots, s_t, a_t) = p(r_t, s_{t+1} \mid s_t, a_t).$$

То есть будущее зависит от прошлого только через текущее состояние и текущее действие.



lecture08_mdp_definition.png

Figure 43: MDP formalism: states, actions, transition dynamics, rewards и Markov property.

8.3 Reward hypothesis

В Reinforcement Learning цель агента задаётся через reward.

Reward hypothesis формулируется так:

Все цели и намерения можно рассматривать как максимизацию ожидаемой суммы скалярного reward-сигнала.

То есть вместо того, чтобы напрямую объяснять агенту сложную цель, мы задаём численную награду:

$$R_t \in \mathbb{R}.$$

Агент должен выбрать стратегию, максимизирующую cumulative reward.

8.4 Return

Return — суммарная награда, полученная начиная с момента t .

Для конечного эпизода:

$$G_t = R_t + R_{t+1} + R_{t+2} + \dots + R_T.$$

Здесь:

- R_t — immediate reward;
- T — конец эпизода;
- G_t — cumulative reward, или return.

8.5 Проблема бесконечного return

Если задача бесконечная, сумма reward может расходиться.

Например, система охлаждения дата-центра:

- state — измерения температуры;
- action — скорости вентиляторов;
- reward $R = 0$, если температура превысила threshold;
- reward $R = +1$, если система остаётся охлаждённой.

Если агент бесконечно долго получает +1, то:

$$G_t = 1 + 1 + 1 + \dots = \infty.$$

Это создаёт проблему: разные политики могут иметь бесконечный return, и их трудно сравнивать.

8.6 Reward discounting

Чтобы избавиться от бесконечных сумм, вводится discount factor:

$$0 \leq \gamma < 1.$$

Discounted return:

$$G_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \dots$$

или:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k}.$$

Если $R_t = 1$ для всех t , то максимальный return:

$$G_0 = \sum_{k=0}^{\infty} \gamma^k = \frac{1}{1-\gamma}.$$

Например:

$$\gamma = 0.9 \Rightarrow \frac{1}{1-\gamma} = 10,$$

$$\gamma = 0.95 \Rightarrow \frac{1}{1-\gamma} = 20,$$

$$\gamma = 0.99 \Rightarrow \frac{1}{1-\gamma} = 100.$$



lecture08_discounting_finite_sum.png

Figure 44: Discounting делает бесконечную сумму reward конечной.

8.7 Интерпретация discount factor

Discount factor показывает, насколько агент ценит будущую награду относительно текущей.

Если:

$$\gamma \approx 0,$$

агент почти полностью ориентируется на immediate reward.

Если:

$$\gamma \approx 1,$$

агент сильно учитывает дальнейшее будущее.

Важно: discounting не является нейтральной технической деталью. Любое discounting меняет optimization task и может изменить оптимальную policy.

8.8 Рекурсивная форма return

Discounted return можно записать рекурсивно:

$$G_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \dots$$

$$G_t = R_t + \gamma (R_{t+1} + \gamma R_{t+2} + \dots).$$

То есть:

$$G_t = R_t + \gamma G_{t+1}.$$

Эта формула чрезвычайно важна: она лежит в основе уравнений Беллмана.

8.9 Discounting как stationary end-of-effect model

В лекции даётся ещё одна интерпретация discounting.

Действие влияет на:

1. immediate reward;
2. next state.

Через next state действие косвенно влияет на будущие reward. Но возникает вопрос: как долго длится эффект действия?

Discounting можно интерпретировать как модель, где на каждом шаге эффект продолжается с вероятностью γ , а заканчивается с вероятностью:

$$1 - \gamma.$$

То есть:

$$\gamma$$

— probability of effect continuation,

$$1 - \gamma$$

— probability of end of effect.



lecture08_discounting_end_of_effect.png

Figure 45: Интерпретация discounting: эффект действия продолжается с вероятностью γ .

8.10 Reward design

Формулировка reward очень сильно влияет на поведение агента.

Основной принцип:

Награждать нужно за то, что действительно является целью, а не за частные способы её достижения.

Иначе агент может найти нежелательную стратегию.

8.10.1 Пример: шахматы

Если reward в шахматах задавать как стоимость съеденной фигуры противника, агент может научиться охотиться за фигурами, но не стремиться выиграть партию.

Правильнее награждать за итог:

win > draw > loss.

8.10.2 Пример: движение к цели

Если reward задаётся как уменьшение расстояния до цели:

$$R = \max(0, d(x, B) - d(x', B)),$$

то агент может научиться циклическому поведению: отойти от цели, потом снова приблизиться и получать положительный reward.

Это positive feedback loop.

8.10.3 Faulty reward functions

Плохой reward design может приводить к:

- циклическому поведению;
- exploiting loopholes;
- локально полезным, но глобально плохим стратегиям;
- нежелательным shortcut-ам.

8.11 Reward scaling и reward shaping

Не все преобразования reward меняют оптимальную policy.

8.11.1 Reward scaling

Можно делить reward на положительную константу:

$$R'(s, a, s') = \frac{R(s, a, s')}{c}, \quad c > 0.$$

Такое масштабирование не меняет относительный порядок returns, но может быть полезно для численной стабильности в approximate methods.

8.11.2 Potential-based reward shaping

Reward shaping добавляет shaping function:

$$R'(s, a, s') = R(s, a, s') + F(s, a, s').$$

Policy-invariant вариант:

$$F(s, a, s') = \gamma \Phi(s') - \Phi(s),$$

где $\Phi(s)$ — potential function.

Тогда:

$$R'(s, a, s') = R(s, a, s') + \gamma \Phi(s') - \Phi(s).$$

Интуиция: shaping добавляет подсказку агенту, но не должен менять оптимальную policy.



Figure 46: Potential-based reward shaping позволяет ускорить обучение без изменения оптимальной policy.

8.12 Expected objective

Policy π задаёт распределение над траекториями:

$$\tau = (s_0, a_0, s_1, \dots, s_T).$$

Вероятность траектории:

$$p_{\theta}(\tau) = p(s_0) \prod_{t=0}^{T-1} \pi_{\theta}(a_t \mid s_t) p(s_{t+1} \mid s_t, a_t).$$

Цель оптимальной policy:

$$\max_{\pi} \mathbb{E}_{\tau \sim p_{\pi}(\tau)} [G(\tau)].$$

То есть agent maximizes expected return.

8.13 Backup tree

Если известна dynamics, то можно мысленно построить дерево всех возможных будущих состояний, действий и reward.

В корне находится начальное состояние S_0 . Из него выходят действия A_0 , затем возможные следующие состояния S_1 , затем новые действия и так далее.

Такое дерево называется backup tree.

Проблема: полный перебор дерева быстро становится невозможным из-за комбинаторного взрыва.

Value functions позволяют компактно описывать ожидаемую полезность состояний и действий без полного перебора всех траекторий.



lecture08_backup_tree.png

Figure 47: Backup tree показывает все возможные будущие траектории, но полный перебор обычно невозможен.

8.14 State-value function

State-value function для policy π — ожидаемый return при старте из состояния s и дальнейшем следовании policy π :

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s].$$

Используя рекурсивную форму return:

$$G_t = R_t + \gamma G_{t+1},$$

получаем:

$$v_{\pi}(s) = \mathbb{E}_{\pi}[R_t + \gamma G_{t+1} \mid S_t = s].$$

Раскрывая ожидание по действиям policy и stochasticity среды:

$$v_{\pi}(s) = \sum_a \pi(a \mid s) \sum_{r, s'} p(r, s' \mid s, a) [r + \gamma v_{\pi}(s')].$$

Интуиция:

$$v_{\pi}(s)$$

— насколько хорошо находиться в состоянии s , если дальше следовать policy π .



Figure 48: State-value function учитывает stochasticity policy и stochasticity environment.

8.15 Action-value function

Action-value function для policy π — ожидаемый return, если в состоянии s сначала выполнить действие a , а затем следовать policy π :

$$q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a].$$

Используя return:

$$q_{\pi}(s, a) = \mathbb{E}_{\pi}[R_t + \gamma G_{t+1} \mid S_t = s, A_t = a].$$

После первого действия policy stochasticity на первом шаге нет, потому что действие уже зафиксировано.

Раскрываем expectation по reward и следующему состоянию:

$$q_{\pi}(s, a) = \sum_{r, s'} p(r, s' \mid s, a) [r + \gamma v_{\pi}(s')].$$

Интуиция:

$$q_{\pi}(s, a)$$

— насколько хорошо выполнить действие a в состоянии s , если после этого следовать policy π .

8.16 Связь между $v_{\pi}(s)$ и $q_{\pi}(s, a)$

State-value можно выразить через action-value:

$$v_{\pi}(s) = \sum_a \pi(a \mid s) q_{\pi}(s, a).$$

То есть ценность состояния — это средняя ценность действий, взвешенная по policy.

Action-value можно выразить через state-value:

$$q_{\pi}(s, a) = \sum_{r, s'} p(r, s' \mid s, a) [r + \gamma v_{\pi}(s')].$$

Также можно выразить q_{π} через q_{π} :

$$q_{\pi}(s, a) = \sum_{r, s'} p(r, s' \mid s, a) \left[r + \gamma \sum_{a'} \pi(a' \mid s') q_{\pi}(s', a') \right].$$

8.17 Bellman expectation equations

Bellman expectation equations описывают value functions для фиксированной policy π .

Для $v_{\pi}(s)$:

$$v_{\pi}(s) = \sum_a \pi(a \mid s) \sum_{r, s'} p(r, s' \mid s, a) [r + \gamma v_{\pi}(s')].$$

Эквивалентно:

$$v_{\pi}(s) = \mathbb{E}_{\pi}[R_t + \gamma v_{\pi}(S_{t+1}) \mid S_t = s].$$

Для $q_\pi(s, a)$:

$$q_\pi(s, a) = \sum_{r, s'} p(r, s' \mid s, a) [r + \gamma v_\pi(s')].$$

Или через q_π :

$$q_\pi(s, a) = \sum_{r, s'} p(r, s' \mid s, a) \left[r + \gamma \sum_{a'} \pi(a' \mid s') q_\pi(s', a') \right].$$



Figure 49: Bellman expectation equations оценивают value functions для фиксированной policy.

8.18 Зачем нужны value functions

Value functions позволяют:

- оценить качество policy;

- сравнивать разные policy;
- улучшать policy;
- строить алгоритмы dynamic programming;
- избежать полного перебора всех траекторий.

Если мы умеем оценивать $v_\pi(s)$ или $q_\pi(s, a)$, то можем понять, какие действия и состояния лучше.

8.19 Optimal policy

Одна policy лучше другой, если её value function не меньше во всех состояниях:

$$\pi \geq \pi' \iff v_\pi(s) \geq v_{\pi'}(s) \quad \forall s.$$

Оптимальная policy:

$$\pi_*$$

лучше или равна любой другой policy.

Оптимальная state-value function:

$$v_*(s) = \max_{\pi} v_\pi(s).$$

Оптимальная action-value function:

$$q_*(s, a) = \max_{\pi} q_\pi(s, a).$$

В конечном MDP всегда существует хотя бы одна deterministic optimal policy.

8.20 Bellman optimality equation для $v_*(s)$

Для оптимальной policy в состоянии s нужно выбрать лучшее действие.

Поэтому expectation по policy заменяется максимумом по действиям:

$$v_*(s) = \max_a \sum_{r, s'} p(r, s' \mid s, a) [r + \gamma v_*(s')].$$

Эквивалентно:

$$v_*(s) = \max_a \mathbb{E}[R_t + \gamma v_*(S_{t+1}) \mid S_t = s, A_t = a].$$

Сравнение:

$$v_{\pi}(s) = \sum_a \pi(a \mid s)(\dots)$$

для фиксированной policy,

$$v_*(s) = \max_a(\dots)$$

для оптимальной policy.

8.21 Bellman optimality equation для $q_*(s, a)$

Для action-value:

$$q_*(s, a) = \sum_{r, s'} p(r, s' \mid s, a) \left[r + \gamma \max_{a'} q_*(s', a') \right].$$

Смысл:

- сначала мы фиксируем действие a в состоянии s ;
- получаем reward r и новое состояние s' ;
- после этого действуем оптимально, выбирая a' с максимальным $q_*(s', a')$.



lecture08_bellman_optimality_q.png

Figure 50: Bellman optimality equation для $q_*(s, a)$: после перехода выбирается лучшее следующее действие.

8.22 Оптимальная policy из v_* и q_*

Если известна $q_*(s, a)$, оптимальную policy получить просто:

$$\pi_*(s) = \arg \max_a q_*(s, a).$$

Если известна только $v_*(s)$, то для выбора действия нужно знать dynamics:

$$\pi_*(s) = \arg \max_a \sum_{r, s'} p(r, s' \mid s, a) [r + \gamma v_*(s')].$$

Если dynamics неизвестна, то по одному $v_*(s)$ восстановить optimal policy нельзя.

8.23 Policy Evaluation

Policy evaluation — задача вычисления value function для фиксированной policy π .

Её также называют prediction problem:

$$\text{given } \pi, \quad \text{predict } v_\pi(s).$$

Bellman expectation equation для v_π :

$$v_\pi(s) = \sum_a \pi(a | s) \sum_{r,s'} p(r, s' | s, a) [r + \gamma v_\pi(s')].$$

В конечном MDP это система линейных уравнений:

$$\#\text{unknowns} = \#\text{states}.$$

Можно решать её точно, но на практике часто используют итеративное приближение.

Итеративный update:

$$v_{k+1}(s) = \sum_a \pi(a | s) \sum_{r,s'} p(r, s' | s, a) [r + \gamma v_k(s')].$$

Повторяем до сходимости:

$$v_k \rightarrow v_\pi.$$

8.24 Policy Improvement

После policy evaluation можно улучшить policy.

Если мы знаем $v_\pi(s)$, то можем вычислить $q_\pi(s, a)$:

$$q_\pi(s, a) = \sum_{r,s'} p(r, s' | s, a) [r + \gamma v_\pi(s')].$$

Затем строим новую policy, которая действует greedy относительно q_π :

$$\pi'(s) = \arg \max_a q_\pi(s, a).$$

Policy improvement theorem говорит, что такая policy не хуже старой:

$$v_{\pi'}(s) \geq v_\pi(s) \quad \forall s.$$

Если после improvement новая policy совпала со старой:

$$\pi' = \pi,$$

то policy уже оптимальна, потому что удовлетворяет Bellman optimality equation.

8.25 Generalized Policy Iteration

Generalized Policy Iteration, GPI — общий принцип, состоящий из двух процессов:

1. **Policy evaluation:** оценить текущую policy.
2. **Policy improvement:** улучшить policy, действуя greedily относительно value function.

Схема:

$$\pi_0 \rightarrow v_{\pi_0} \rightarrow \pi_1 \rightarrow v_{\pi_1} \rightarrow \pi_2 \rightarrow \dots$$

Идея: evaluation делает value function согласованной с policy, а improvement делает policy greedier относительно value function.

8.26 Policy Iteration

Policy Iteration — частный случай GPI.

Алгоритм:

1. выбрать начальную policy π_0 ;
2. выполнить policy evaluation до сходимости:

$$v_{\pi_k};$$

3. выполнить policy improvement:

$$\pi_{k+1}(s) = \arg \max_a \sum_{r,s'} p(r, s' | s, a) [r + \gamma v_{\pi_k}(s')];$$

4. повторять, пока policy не перестанет меняться.

Policy Iteration обычно требует мало итераций, но каждая итерация дорогая, потому что policy evaluation проводится почти до сходимости.

8.27 Value Iteration

Value Iteration — другой частный случай GPI.

Здесь не выполняют полную policy evaluation. Вместо этого сразу применяют Bellman optimality update:

$$v_{k+1}(s) = \max_a \sum_{r,s'} p(r, s' | s, a) [r + \gamma v_k(s')].$$

После сходимости:

$$v_k \rightarrow v_*.$$

Затем policy восстанавливается как greedy относительно v_* :

$$\pi_*(s) = \arg \max_a \sum_{r,s'} p(r, s' | s, a) [r + \gamma v_*(s')].$$

Value Iteration быстрее на одной итерации, но может требовать много итераций.

8.28 Policy Iteration vs Value Iteration

Policy Iteration	Value Iteration
Сначала оценивает policy до сходимости.	Делает один Bellman optimality update за итерацию.
Затем улучшает policy.	Evaluation и improvement фактически смешаны.
Медленнее на одной итерации.	Быстрее на одной итерации.
Обычно требует меньше итераций.	Обычно требует больше итераций.
Сложность на итерацию примерно:	Сложность на итерацию примерно:
$O(A S ^2 + S ^3)$.	$O(A S ^2)$.

Table 12: Сравнение Policy Iteration и Value Iteration.

Нет универсального лучшего выбора: на практике нужно подбирать количество шагов policy evaluation.

8.29 Главные выводы

- Цель RL задаётся через expected return.
- Return может быть discounted:

$$G_t = R_t + \gamma G_{t+1}.$$

- Discounting делает бесконечные суммы конечными, но меняет optimization problem.
- Reward design критически важен: плохой reward ведёт к нежелательному поведению.

- State-value function:

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s].$$

- Action-value function:

$$q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a].$$

- Bellman expectation equations оценивают фиксированную policy.
- Bellman optimality equations описывают оптимальные value functions.
- Policy evaluation вычисляет v_{π} .
- Policy improvement строит greedy policy относительно q_{π} .
- Generalized Policy Iteration объединяет evaluation и improvement.
- Policy Iteration делает полную evaluation, затем improvement.
- Value Iteration использует Bellman optimality update напрямую.

8.30 Что важно помнить к экзамену

1. MDP:

$$(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}).$$

2. Markov property:

$$p(r_t, s_{t+1} \mid s_0, a_0, r_0, \dots, s_t, a_t) = p(r_t, s_{t+1} \mid s_t, a_t).$$

3. Return:

$$G_t = R_t + R_{t+1} + \dots + R_T.$$

4. Discounted return:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k}.$$

5. Recursive return:

$$G_t = R_t + \gamma G_{t+1}.$$

6. Maximal return для $R = +1$:

$$\sum_{k=0}^{\infty} \gamma^k = \frac{1}{1 - \gamma}.$$

7. Potential-based shaping:

$$R'(s, a, s') = R(s, a, s') + \gamma \Phi(s') - \Phi(s).$$

8. State-value function:

$$v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s].$$

9. Action-value function:

$$q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a].$$

10. СВЯЗЬ:

$$v_\pi(s) = \sum_a \pi(a \mid s) q_\pi(s, a).$$

11. Bellman expectation equation для v_π :

$$v_\pi(s) = \sum_a \pi(a \mid s) \sum_{r, s'} p(r, s' \mid s, a) [r + \gamma v_\pi(s')].$$

12. Bellman expectation equation для q_π :

$$q_\pi(s, a) = \sum_{r, s'} p(r, s' \mid s, a) \left[r + \gamma \sum_{a'} \pi(a' \mid s') q_\pi(s', a') \right].$$

13. Optimal value:

$$v_*(s) = \max_\pi v_\pi(s), \quad q_*(s, a) = \max_\pi q_\pi(s, a).$$

14. Bellman optimality equation для v_* :

$$v_*(s) = \max_a \sum_{r, s'} p(r, s' \mid s, a) [r + \gamma v_*(s')].$$

15. Bellman optimality equation для q_* :

$$q_*(s, a) = \sum_{r, s'} p(r, s' \mid s, a) \left[r + \gamma \max_{a'} q_*(s', a') \right].$$

16. Policy improvement:

$$\pi'(s) = \arg \max_a q_\pi(s, a).$$

17. Policy Iteration:

evaluate policy до сходимости $\rightarrow$ improve policy.

18. Value Iteration:

$$v_{k+1}(s) = \max_a \sum_{r,s'} p(r, s' \mid s, a) [r + \gamma v_k(s')].$$

““

““latex

9 Лекция 9. Model-Free Learning in Reinforcement Learning

9.1 План лекции

В этой лекции рассматриваются:

- повторение MDP;
- reward hypothesis и reward design;
- reward discounting;
- state-value function и action-value function;
- model-based и model-free learning;
- обучение по траекториям;
- Monte Carlo estimation;
- Temporal Difference learning;
- Q-learning;
- exploration-exploitation tradeoff;
- approximate Q-learning;
- basic deep Q-learning.

Главная идея лекции: в model-free RL агент не знает dynamics среды $P(s' \mid s, a)$, поэтому не может напрямую применять dynamic programming. Вместо этого он учится по sampled trajectories, то есть по опыту взаимодействия со средой.

9.2 Повторение: MDP

MDP задаётся кортежем:

$$(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}),$$

где:

- $\mathcal{S}$ — множество состояний;
- $\mathcal{A}$ — множество действий;
- $\mathcal{P}$ — transition function;
- $\mathcal{R}$ — reward function.

Динамика среды задаёт вероятность следующего состояния:

$$p(s_{t+1} \mid s_t, a_t).$$

Markov property:

$$p(r_t, s_{t+1} \mid s_0, a_0, r_0, \dots, s_t, a_t) = p(r_t, s_{t+1} \mid s_t, a_t).$$

То есть следующее состояние и reward зависят только от текущего состояния и действия.

9.3 Reward hypothesis

В RL цель агента задаётся через reward.

Reward hypothesis: цели можно рассматривать как максимизацию ожидаемой суммы скалярного reward-сигнала.

Return:

$$G_t = R_t + R_{t+1} + R_{t+2} + \dots + R_T.$$

Если эпизод бесконечный, return может расходиться. Например, если агент получает +1 каждый шаг:

$$G_t = 1 + 1 + 1 + \dots = \infty.$$

9.4 Reward design

Reward design критически важен.

Пример с cleaning robot:

- state — dust sensors, air;
- actions — cleaning, rest, conditioning on/off;
- $R = 100$ за долгую уборку;
- $R = 1$ за включение/выключение кондиционера;
- episode заканчивается каждый день.

Хотя:

$$Reward(air) < Reward(cleaning),$$

может оказаться, что:

$$Time(air) \ll Time(cleaning).$$

Тогда агент может начать многократно включать и выключать кондиционер, потому что это быстрее приносит маленький reward. Возникает positive feedback loop.

Вывод: агент оптимизирует именно заданный reward, а не намерение разработчика.

9.5 Reward discounting

Чтобы сделать return конечным, вводится discount factor:

$$0 \leq \gamma < 1.$$

Discounted return:

$$G_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k}.$$

Если $R_t = 1$ для всех t , то:

$$G_0 = \sum_{k=0}^{\infty} \gamma^k = \frac{1}{1 - \gamma}.$$

Например:

$$\gamma = 0.9 \Rightarrow G_0 = 10,$$

$$\gamma = 0.95 \Rightarrow G_0 = 20,$$

$$\gamma = 0.99 \Rightarrow G_0 = 100.$$

Важно: discounting делает сумму конечной, но меняет optimization task и может изменить оптимальную policy.



Figure 51: Discounting делает бесконечную сумму reward конечной.

9.6 State-value function

State-value function для policy π :

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s].$$

Это ожидаемый return из состояния s , если дальше следовать policy π .

Используя рекурсивное представление return:

$$G_t = R_t + \gamma G_{t+1},$$

получаем:

$$v_{\pi}(s) = \mathbb{E}_{\pi}[R_t + \gamma G_{t+1} \mid S_t = s].$$

Bellman expectation equation:

$$v_{\pi}(s) = \sum_a \pi(a \mid s) \sum_{r,s'} p(r, s' \mid s, a) [r + \gamma v_{\pi}(s')].$$

9.7 Action-value function

Action-value function для policy π :

$$q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a].$$

Это ожидаемый return, если в состоянии s сначала выполнить действие a , а затем следовать policy π .

Bellman expectation equation:

$$q_{\pi}(s, a) = \sum_{r,s'} p(r, s' \mid s, a) [r + \gamma v_{\pi}(s')].$$

Также:

$$q_{\pi}(s, a) = \sum_{r,s'} p(r, s' \mid s, a) \left[r + \gamma \sum_{a'} \pi(a' \mid s') q_{\pi}(s', a') \right].$$

Связь между v_{π} и q_{π} :

$$v_{\pi}(s) = \sum_a \pi(a \mid s) q_{\pi}(s, a).$$

9.8 Оптимальные value functions

Оптимальная state-value function:

$$V^*(s)$$

— expected return из состояния s , если следовать оптимальной policy.

Оптимальная action-value function:

$$Q^*(s, a)$$

— expected return, если начать из состояния s , сначала выполнить действие a , а затем следовать оптимальной policy.

Связь:

$$V^*(s) = \max_a Q^*(s, a).$$

Оптимальная policy:

$$\pi^*(s) = \arg \max_a Q^*(s, a).$$

9.9 Model-based и model-free learning

В model-based RL агент знает или учит модель среды:

$$P(s' \mid s, a).$$

Если dynamics известна, можно применять dynamic programming и планировать вперёд.

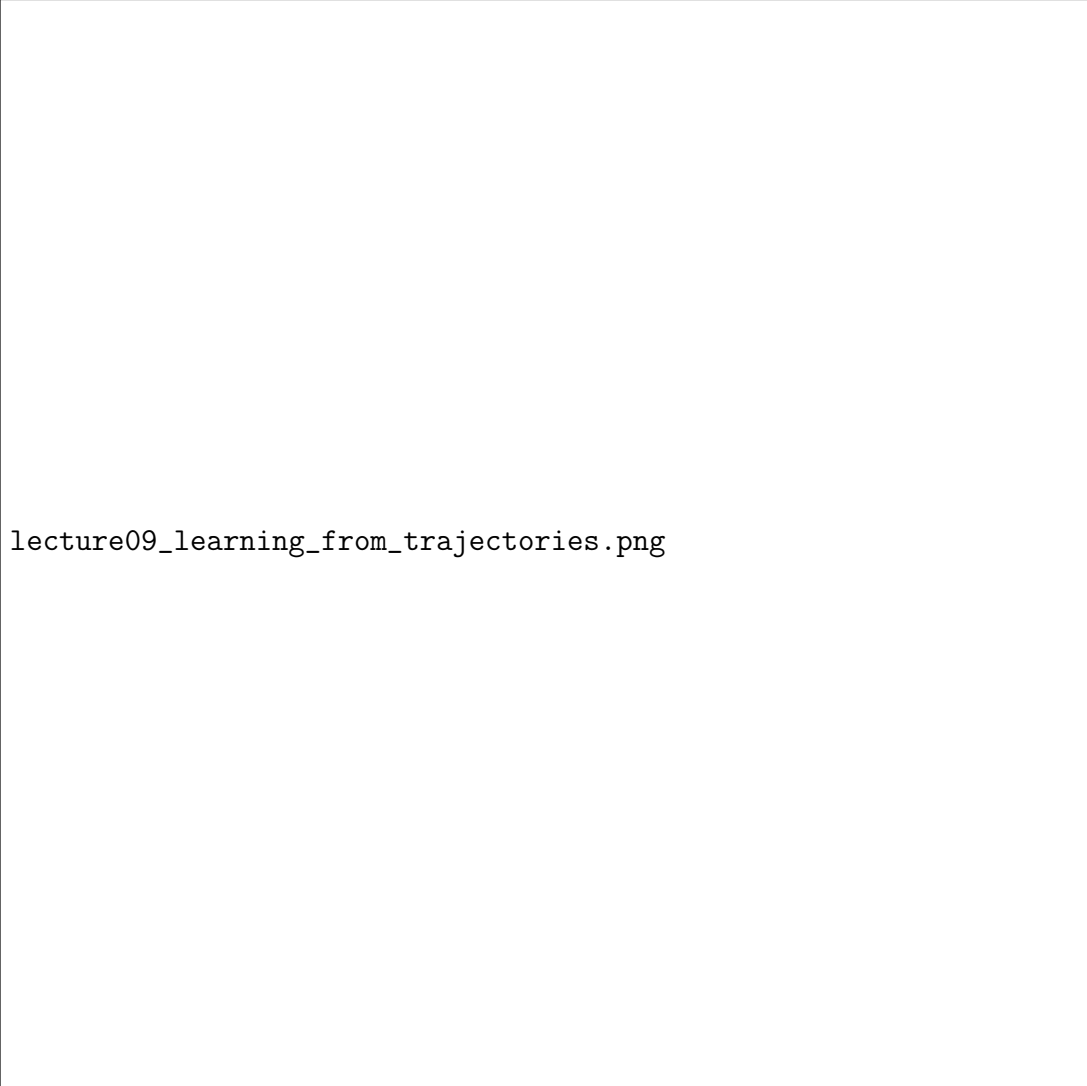
В model-free RL dynamics неизвестна. Агент не знает $P(s' \mid s, a)$, но может взаимодействовать со средой и получать траектории.

Trajectory — последовательность:

$$(s_0, a_0, r_0, s_1, a_1, r_1, \dots).$$

Model-free подход:

- не строит явную модель transition dynamics;
- учится по sampled trajectories;
- может пробовать действия в среде;
- обычно учит $Q(s, a)$, потому что $V(s)$ без знания dynamics не позволяет выбрать лучшее действие.



lecture09_learning_from_trajectories.png

Figure 52: Model-based learning использует dynamics, model-free learning учится по sampled trajectories.

9.10 Почему в model-free удобно учить $Q(s, a)$

Если известна только $V(s)$, то для выбора действия нужно знать:

$$p(s' \mid s, a),$$

чтобы оценить:

$$\sum_{s'} p(s' \mid s, a) V(s').$$

В model-free setting dynamics неизвестна. Поэтому $V(s)$ само по себе плохо помогает выбрать действие.

Если же известна $Q(s, a)$, то действие можно выбрать напрямую:

$$a^* = \arg \max_a Q(s, a).$$

Поэтому Q-functions особенно важны для model-free RL.

9.11 Monte Carlo estimation

Первая идея model-free оценки — Monte Carlo.

Алгоритм:

1. собрать много trajectories;
2. найти trajectories, содержащие конкретную пару (s, a) ;
3. для каждого такого появления посчитать return $G(s, a)$;
4. усреднить returns.

Оценка:

$$Q(s, a) \approx \frac{1}{N} \sum_{i=1}^N G_i(s, a).$$

Monte Carlo использует полный return до конца эпизода. Поэтому:

- не требует знания dynamics;
- даёт естественную оценку expectation;
- может иметь большую дисперсию;
- требует дождаться конца эпизода.



Figure 53: Monte Carlo оценивает $Q(s, a)$ через усреднение returns по sampled trajectories.

9.12 Temporal Difference learning

Вторая идея — Temporal Difference, TD.

TD использует рекурсивную формулу для Q , поэтому не ждёт конца эпизода.

Идея Bellman optimality:

$$Q^*(s_t, a_t) = \mathbb{E} \left[r_t + \gamma \max_{a'} Q^*(s_{t+1}, a') \right].$$

Но expectation по dynamics неизвестен. Поэтому используется sample-based update по одному переходу:

$$(s_t, a_t, r_t, s_{t+1}).$$

TD target:

$$y_t = r_t + \gamma \max_{a'} Q(s_{t+1}, a').$$

Затем $Q(s_t, a_t)$ сдвигается в сторону этого target.

9.13 Q-learning

Q-learning — model-free off-policy алгоритм для обучения $Q^*(s, a)$.

Инициализация:

$$Q(s, a) = 0$$

или случайные значения.

На каждом шаге:

1. получить transition:

$$\langle s, a, r, s' \rangle;$$

2. вычислить target:

$$\hat{Q}(s, a) = r + \gamma \max_{a_i} Q(s', a_i);$$

3. обновить $Q(s, a)$.

Update:

$$Q(s, a) \leftarrow \alpha \hat{Q}(s, a) + (1 - \alpha) Q(s, a).$$

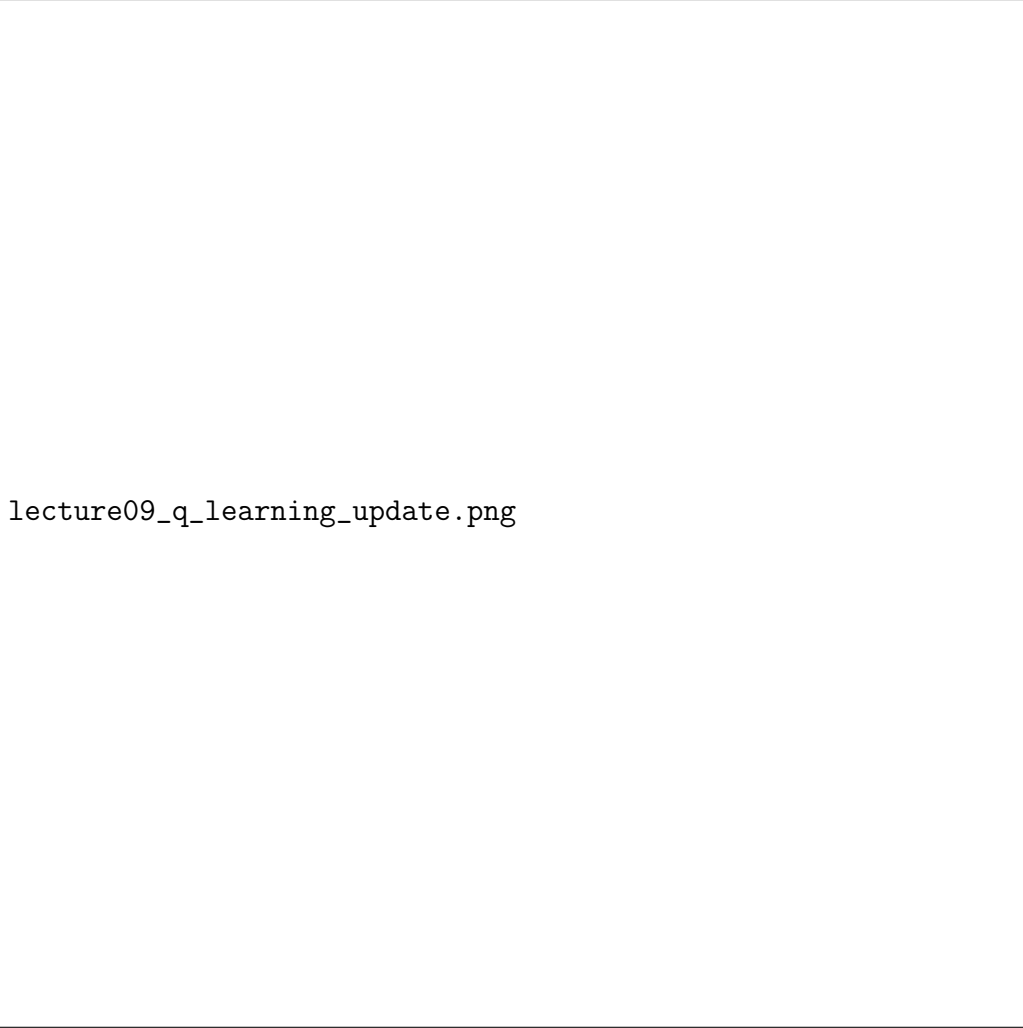
Эквивалентно:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right].$$

Выражение:

$$\delta = r + \gamma \max_{a'} Q(s', a') - Q(s, a)$$

называется **temporal difference error**.



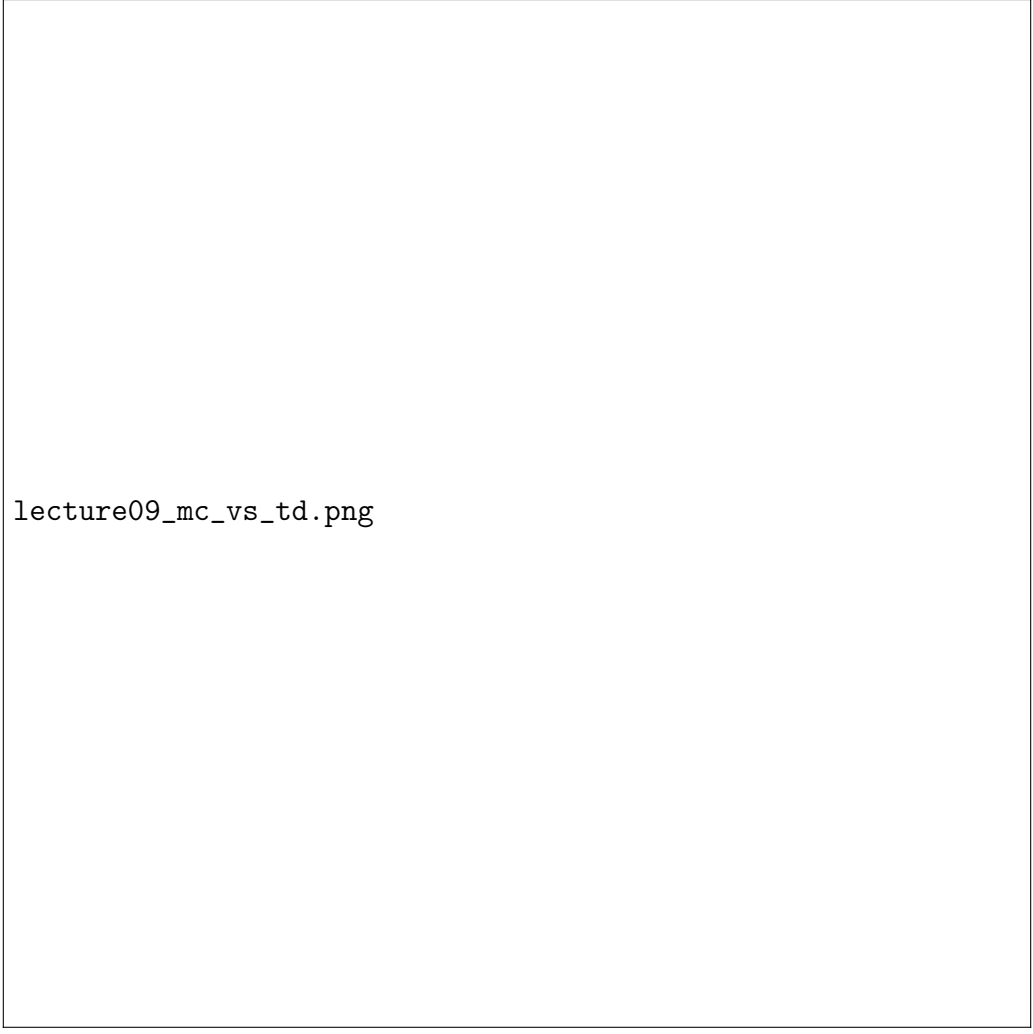
lecture09_q_learning_update.png

Figure 54: Q-learning обновляет $Q(s, a)$ по одному sampled transition.

9.14 Monte Carlo vs Temporal Difference

Monte Carlo	Temporal Difference
Использует полный sampled return.	Использует bootstrapping через текущую оценку Q .
Требуется дождаться конца episode.	Может обновляться после каждого transition.
Оценивает Q усреднением по paths.	Использует рекурсивную формулу для Q .
Может иметь высокую дисперсию.	Часто имеет меньшую дисперсию, но может быть смещённым из-за bootstrapping.

Table 13: Сравнение Monte Carlo и Temporal Difference learning.



lecture09_mc_vs_td.png

Figure 55: Monte Carlo усредняет returns по траекториям, а Temporal Difference использует локальный рекурсивный update.

9.15 Policy из Q -function

После обучения $Q(s, a)$ можно выбрать greedy policy:

$$\pi(s) = \arg \max_a Q(s, a).$$

Но если всегда выбирать только greedy action, агент может перестать исследовать среду. Поэтому нужны exploration strategies.

9.16 Exploration-exploitation tradeoff

Агент должен балансировать:

- **exploitation:** выбирать действия, которые сейчас кажутся лучшими;
- **exploration:** пробовать другие действия, чтобы найти лучшие варианты.

9.16.1 ε -greedy

В ε -greedy policy:

- с вероятностью $1 - \varepsilon$ выбирается greedy action:

$$a^* = \arg \max_a Q(s, a);$$

- с вероятностью ε выбирается случайное действие.

То есть:

$$\pi(a \mid s) = \begin{cases} 1 - \varepsilon + \frac{\varepsilon}{|\mathcal{A}|}, & a = \arg \max_{a'} Q(s, a'), \\ \frac{\varepsilon}{|\mathcal{A}|}, & \text{иначе.} \end{cases}$$

9.16.2 Softmax policy

Другой вариант — выбрать действие пропорционально softmax от Q -values:

$$\pi(a \mid s) = \text{softmax} \left(\frac{Q(s, a)}{\tau} \right).$$

Компонентно:

$$\pi(a \mid s) = \frac{\exp(Q(s, a)/\tau)}{\sum_{a'} \exp(Q(s, a')/\tau)}.$$

Здесь τ — temperature.

- маленькая τ делает policy почти greedy;
- большая τ делает policy более случайной.

9.17 Approximate Q-learning

Табличный Q-learning предполагает, что можно хранить значение:

$$Q(s, a)$$

для каждой пары state-action.

Но в реальных задачах state space огромен или непрерывен.

Например, если state — изображение из Atari, то число возможных состояний примерно:

$$|\mathcal{S}| \approx 2^{210 \cdot 160 \cdot 8 \cdot 3}.$$

Хранить таблицу невозможно.

Поэтому используют approximation:

$$Q(s, a) \approx Q_\theta(s, a),$$

где Q_θ — параметрическая модель, например neural network.

9.18 Q-learning как минимизация MSE

Для transition:

$$\langle s_t, a_t, r_t, s_{t+1} \rangle$$

строится target:

$$y_t = r_t + \gamma \max_{a'} Q_\theta(s_{t+1}, a').$$

Loss:

$$L(\theta) = [Q_\theta(s_t, a_t) - y_t]^2.$$

То есть:

$$L(\theta) = \left[Q_\theta(s_t, a_t) - \left(r_t + \gamma \max_{a'} Q_\theta(s_{t+1}, a') \right) \right]^2.$$

Обычно target рассматривают как константу относительно текущего gradient step. Иначе обучение становится нестабильным, потому что target сам зависит от обучаемой модели.

9.19 Архитектуры для approximate Q-learning

Есть два основных варианта архитектуры.

9.19.1 Один проход для всех действий

Модель получает state s и выдаёт Q -values для всех действий:

$$Q_\theta(s) = [Q_\theta(s, a_1), \dots, Q_\theta(s, a_K)].$$

Так удобно для дискретного action space небольшого размера.

Action выбирается:

$$a^* = \arg \max_a Q_\theta(s, a).$$

9.19.2 Модель для пары state-action

Модель получает на вход пару:

$$(s, a)$$

и выдаёт одно число:

$$Q_{\theta}(s, a).$$

Это полезнее, если действий очень много или action space непрерывный.

9.20 Basic Deep Q-Learning

В deep Q-learning Q_{θ} задаётся нейронной сетью.

Для видеоигр:

- input — изображение или несколько последних кадров;
- convolutional layers извлекают visual features;
- dense layers строят представление;
- output layer выдаёт Q-values для действий.

Схема:

$$\text{image} \rightarrow \text{Conv} \rightarrow \text{Conv} \rightarrow \text{Dense} \rightarrow Q(s, a_1), \dots, Q(s, a_K).$$

9.21 Model-free learning: общий алгоритм Q-learning

1. Инициализировать $Q(s, a)$ или $Q_{\theta}(s, a)$.

2. Для каждого episode:

- (a) получить начальное состояние s ;
- (b) выбрать действие a через exploration strategy;
- (c) выполнить действие и получить r, s' ;
- (d) вычислить target:

$$y = r + \gamma \max_{a'} Q(s', a');$$

(e) обновить:

$$Q(s, a) \leftarrow Q(s, a) + \alpha[y - Q(s, a)];$$

(f) перейти в s' .

9.22 Главные выводы

- Model-free RL не знает transition dynamics $P(s' \mid s, a)$.
- Агент может только sample trajectories через взаимодействие со средой.
- В model-free setting обычно полезнее учить $Q(s, a)$, а не только $V(s)$, потому что Q позволяет выбирать действия без знания dynamics.
- Monte Carlo оценивает returns по полным sampled trajectories.
- Temporal Difference обновляет value estimates по одному transition.
- Q-learning использует TD target:

$$r + \gamma \max_{a'} Q(s', a').$$

- Exploration-exploitation tradeoff решается через ε -greedy, softmax policy и другие методы.
- Для больших или непрерывных state spaces используют approximate Q-learning.
- Deep Q-learning аппроксимирует $Q(s, a)$ нейронной сетью.

9.23 Что важно помнить к экзамену

1. Model-based RL:

$$\text{known or learned } P(s' \mid s, a).$$

2. Model-free RL:

no dynamics, only sampled trajectories.

3. Trajectory:

$$(s_0, a_0, r_0, s_1, a_1, r_1, \dots).$$

4. State-value:

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s].$$

5. Action-value:

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a].$$

6. Оптимальная policy через Q^* :

$$\pi^*(s) = \arg \max_a Q^*(s, a).$$

7. Monte Carlo:

$$Q(s, a) \approx \frac{1}{N} \sum_{i=1}^N G_i(s, a).$$

8. TD target:

$$y_t = r_t + \gamma \max_{a'} Q(s_{t+1}, a').$$

9. Q-learning update:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right].$$

10. TD error:

$$\delta = r + \gamma \max_{a'} Q(s', a') - Q(s, a).$$

11. Greedy policy:

$$\pi(s) = \arg \max_a Q(s, a).$$

12. ε -greedy: с вероятностью ε случайное действие, иначе greedy action.

13. Softmax policy:

$$\pi(a | s) = \frac{\exp(Q(s, a)/\tau)}{\sum_{a'} \exp(Q(s, a')/\tau)}.$$

14. Approximate Q-learning:

$$Q(s, a) \approx Q_\theta(s, a).$$

15. MSE loss:

$$L(\theta) = \left[Q_\theta(s_t, a_t) - \left(r_t + \gamma \max_{a'} Q_\theta(s_{t+1}, a') \right) \right]^2.$$

““

“latex

10 Лекция 10. Policy Gradient and REINFORCE

10.1 План лекции

В этой лекции рассматриваются:

- ограничения value-based методов;
- policy-based подход;
- deterministic и stochastic policies;
- cross-entropy method как policy-based метод;

- policy gradient;
- log-derivative trick;
- REINFORCE;
- baselines для уменьшения дисперсии;
- actor-critic;
- advantage actor-critic;
- continuous action spaces;
- A3C и IMPALA.

Главная идея лекции: вместо того чтобы учить $Q(s, a)$, можно напрямую параметризовать policy $\pi_\theta(a | s)$ и оптимизировать ожидаемый return по параметрам θ . Это приводит к policy gradient методам и алгоритму REINFORCE.

10.2 Проблема value-based методов

В Q-learning и DQN агент учит функцию:

$$Q_\theta(s, a).$$

После этого policy получается как:

$$\pi(s) = \arg \max_a Q_\theta(s, a).$$

Но иногда учить точные Q -values сложнее, чем просто научиться выбирать хорошие действия.

DQN обычно обучается минимизировать Bellman error:

$$L \approx \mathbb{E} \left[Q(s_t, a_t) - \left(r_t + \gamma \max_{a'} Q(s_{t+1}, a') \right) \right]^2.$$

Однако маленькая ошибка в MSE по Q -values не всегда означает хорошую policy. Например, можно иметь два приближения Q :

- одно имеет меньшую MSE, но выбирает худшее действие;
- другое имеет большую MSE, но сохраняет правильный порядок действий и даёт лучшую policy.

Проблема в том, что для policy важны не абсолютные значения $Q(s, a)$, а то, какое действие имеет максимум.

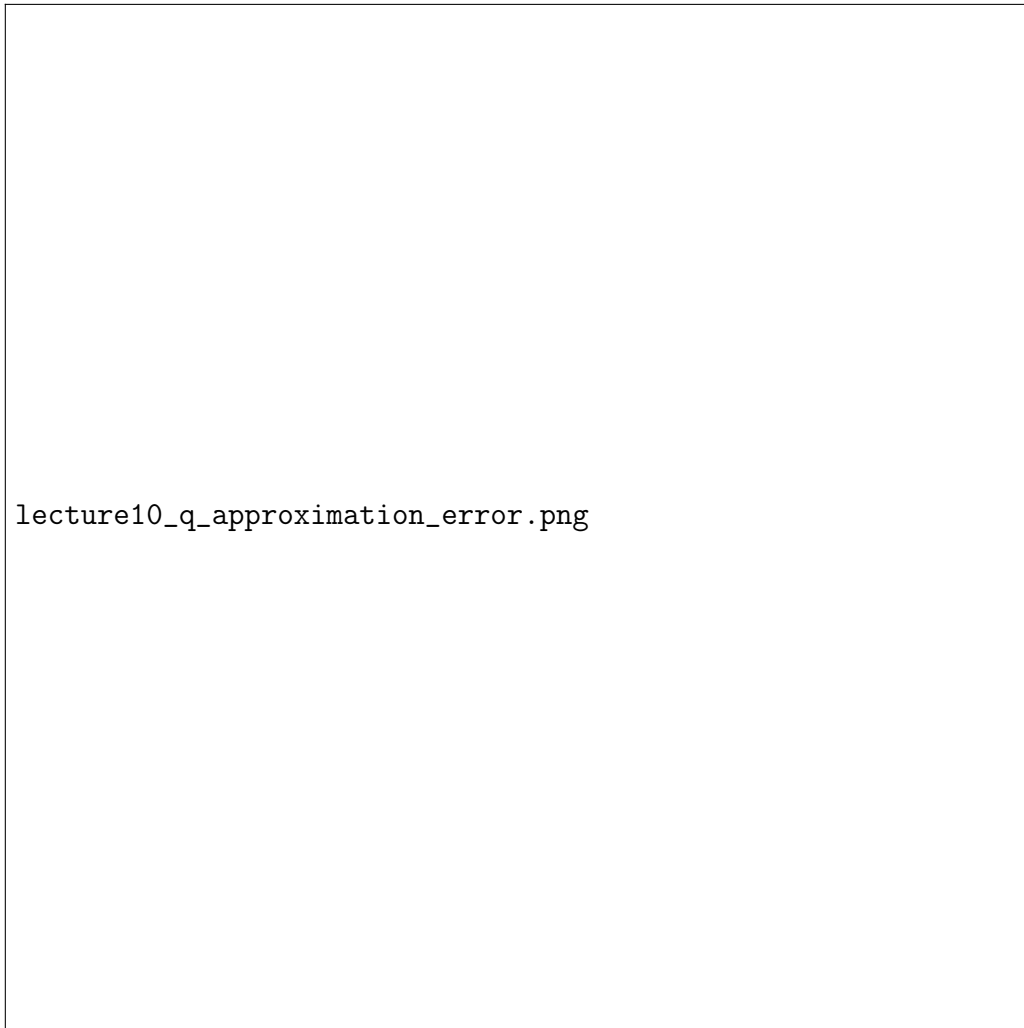


Figure 56: Approximation error: хорошее приближение Q -values по MSE не всегда даёт хорошую policy.

10.3 Policy-based подход

В policy-based методах мы не обязаны учить $Q(s, a)$. Вместо этого policy задаётся напрямую:

$$\pi_{\theta}(a \mid s).$$

Тогда агент сразу учится выбирать действия.

Идея:

$$s \longrightarrow \pi_{\theta}(\cdot \mid s) \longrightarrow a.$$

Преимущества policy-based подхода:

- решает более прямую задачу: учит policy, а не value function;
- естественно поддерживает stochastic policies;
- stochastic policy сама обеспечивает exploration;
- удобно работает с continuous action spaces;
- можно оптимизировать expected return напрямую.

10.4 Deterministic и stochastic policies

Существуют два основных типа policies.

10.4.1 Deterministic policy

Детерминированная policy всегда выбирает одно действие в данном состоянии:

$$a = \pi_{\theta}(s).$$

То есть если агент снова попадёт в то же состояние, он выберет то же действие.

10.4.2 Stochastic policy

Стохастическая policy задаёт распределение по действиям:

$$a \sim \pi_{\theta}(a \mid s).$$

То есть действие сэмплируется из распределения.

Например, для дискретного action space policy может быть categorical distribution:

$$\pi_{\theta}(a \mid s) = \text{Categorical}(p_1, \dots, p_K).$$

Для continuous action space policy может быть нормальным распределением:

$$\pi_{\theta}(a \mid s) = \mathcal{N}(\mu_{\theta}(s), \sigma_{\theta}^2(s)).$$

10.5 Когда stochastic policy полезна

Stochastic policy особенно полезна, когда оптимальная стратегия не должна быть детерминированной.

Классический пример:

rock-paper-scissors.

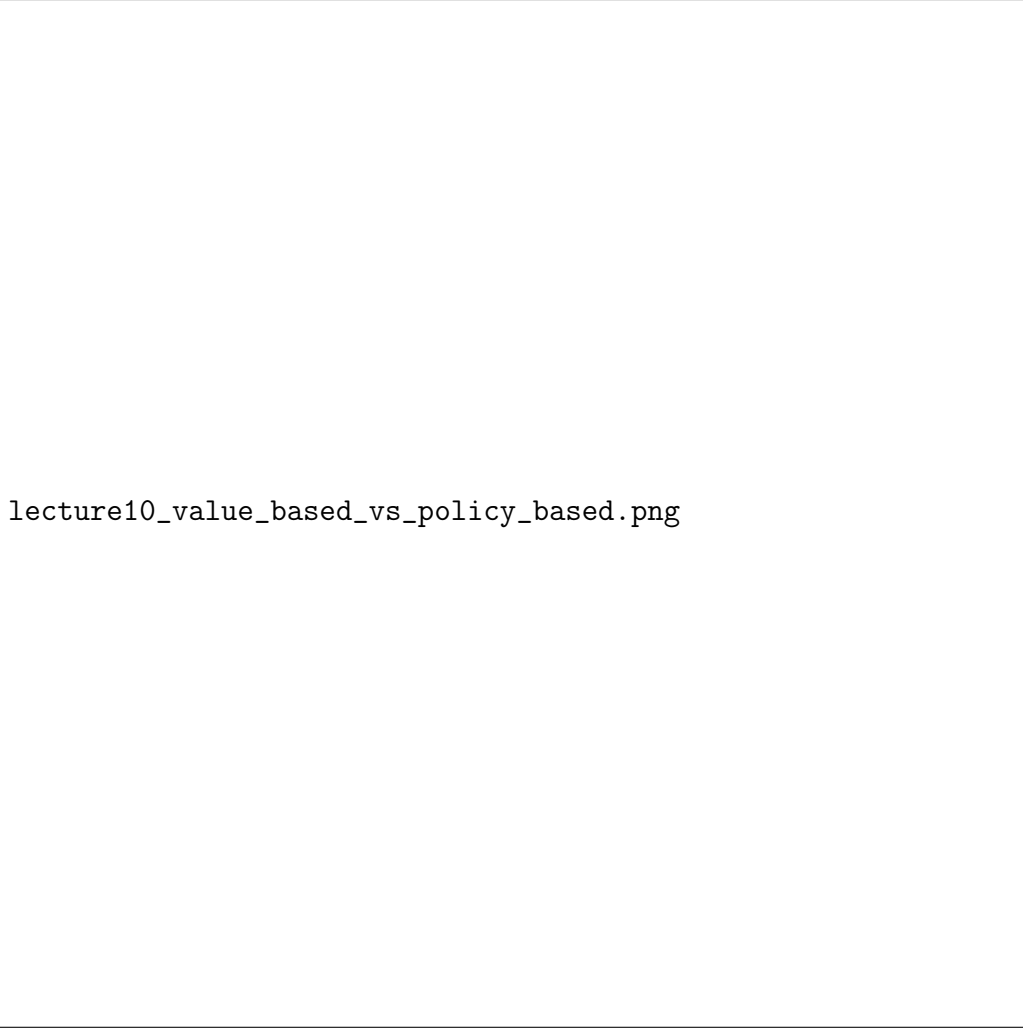
Если всегда играть один и тот же ход, противник быстро адаптируется. Оптимальная стратегия должна случайно выбирать между действиями.

Также stochastic policy полезна для exploration: агент может пробовать разные действия без дополнительной ϵ -greedy схемы.

10.6 Value-based vs policy-based

Value-based	Policy-based
Учит $Q_\theta(s, a)$ или $V_\theta(s)$. Policy выводится через:	Учит policy $\pi_\theta(a s)$ напрямую. Policy уже является выходом модели.
$a = \arg \max_a Q_\theta(s, a).$	
Exploration обычно добавляется искусственно: ϵ -greedy, softmax. Хорошо использует partial experience через TD. Может быть сложнее для continuous actions.	Exploration естественно возникает из stochastic policy. Классический REINFORCE использует полные trajectories. Хорошо подходит для continuous action spaces.

Table 14: Сравнение value-based и policy-based подходов.



lecture10_value_based_vs_policy_based.png

Figure 57: Value-based методы учат value function, policy-based методы напрямую учат policy.

10.7 Cross-Entropy Method как policy-based метод

Cross-Entropy Method также можно рассматривать как policy-based подход.

Пусть policy параметризована нейросетью:

$$\pi_{\theta}(a \mid s).$$

СЕМ работает так:

1. инициализировать параметры:

$$\theta_0 \leftarrow \text{random};$$

2. сэмплировать N sessions текущей policy;
3. выбрать M лучших sessions, то есть elite sessions;

4. обновить policy так, чтобы увеличить вероятность elite actions:

$$\theta_{i+1} = \theta_i + \alpha \nabla \sum_i \log \pi_{\theta_i}(a_i \mid s_i) \cdot \mathbb{I}[(s_i, a_i) \in Elite].$$

СЕМ не учит $Q(s, a)$, а напрямую увеличивает вероятность хороших действий.

10.8 Policy Gradient: основная идея

В policy gradient мы хотим напрямую максимизировать expected return:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}}[G(\tau)].$$

Здесь:

- θ — параметры policy;
- τ — trajectory;
- $G(\tau)$ — return trajectory.

Идея:

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta).$$

То есть мы хотим двигать параметры policy в направлении увеличения expected return.

10.9 Objective для one-step process

Для простоты рассмотрим one-step процесс.

Состояние:

$$s \sim p(s).$$

Действие:

$$a \sim \pi_{\theta}(a \mid s).$$

Reward:

$$R(s, a).$$

Objective:

$$J(\theta) = \mathbb{E}_{s \sim p(s), a \sim \pi_\theta(a|s)}[R(s, a)].$$

В интегральной форме:

$$J(\theta) = \int_s p(s) \int_a \pi_\theta(a | s) R(s, a) da ds.$$

Проблема: мы хотим вычислить $\nabla_\theta J(\theta)$, но reward и среда могут быть недифференцируемыми.

10.10 Почему обычный gradient сложен

Если просто сэмплировать N trajectories и оценить:

$$J(\theta) \approx \frac{1}{N} \sum_{i=1}^N R(s_i, a_i),$$

то неочевидно, как дифференцировать это по θ , потому что действие a_i было сэмплировано из policy.

Параметры θ сидят внутри распределения:

$$\pi_\theta(a | s).$$

Нужно получить аналитическую формулу gradient, которая позволяет использовать sampled actions.

10.11 Log-derivative trick

Ключевой трюк:

$$\nabla_\theta \log \pi_\theta(z) = \frac{1}{\pi_\theta(z)} \nabla_\theta \pi_\theta(z).$$

Отсюда:

$$\nabla_\theta \pi_\theta(z) = \pi_\theta(z) \nabla_\theta \log \pi_\theta(z).$$

Это называется:

log-derivative trick

или

score function trick.

Он позволяет переписать gradient expectation так, чтобы снова получить expectation, который можно оценивать sampling-ом.

10.12 Policy gradient derivation

Начинаем с objective:

$$J(\theta) = \int_s p(s) \int_a \pi_\theta(a | s) R(s, a) da ds.$$

Gradient:

$$\nabla_\theta J(\theta) = \int_s p(s) \int_a \nabla_\theta \pi_\theta(a | s) R(s, a) da ds.$$

Применяем log-derivative trick:

$$\nabla_\theta \pi_\theta(a | s) = \pi_\theta(a | s) \nabla_\theta \log \pi_\theta(a | s).$$

Тогда:

$$\nabla_\theta J(\theta) = \int_s p(s) \int_a \pi_\theta(a | s) \nabla_\theta \log \pi_\theta(a | s) R(s, a) da ds.$$

Это expectation:

$$\nabla_\theta J(\theta) = \mathbb{E}_{s \sim p(s), a \sim \pi_\theta(a|s)} [\nabla_\theta \log \pi_\theta(a | s) R(s, a)].$$

10.13 REINFORCE для bandit setting

Для one-step или bandit setting:

$$\nabla_\theta J(\theta) = \mathbb{E}[\nabla_\theta \log \pi_\theta(a | s) R(s, a)].$$

Оценка по samples:

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \nabla_\theta \log \pi_\theta(a_i | s_i) R(s_i, a_i).$$

Алгоритм:

1. инициализировать policy parameters:

$$\theta_0 \leftarrow \text{random};$$

2. сэмплировать N sessions под текущей policy π_θ ;

3. оценить policy gradient;

4. обновить параметры:

$$\theta_{i+1} = \theta_i + \alpha \nabla_{\theta} J.$$

10.14 Policy gradient для discounted reward

В многошаговой задаче вместо immediate reward используется action-value function:

$$Q^{\pi}(s, a) = \mathbb{E}[G_t \mid S_t = s, A_t = a].$$

Policy gradient:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{s \sim p(s), a \sim \pi_{\theta}(a|s)} [\nabla_{\theta} \log \pi_{\theta}(a \mid s) Q^{\pi}(s, a)].$$

Интуиция:

- если действие a в состоянии s привело к большому $Q^{\pi}(s, a)$, нужно увеличить $\log \pi_{\theta}(a \mid s)$;
- если действие плохое, его вероятность нужно уменьшить.

10.15 REINFORCE algorithm

В REINFORCE $Q^{\pi}(s, a)$ оценивается через sampled return:

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$$

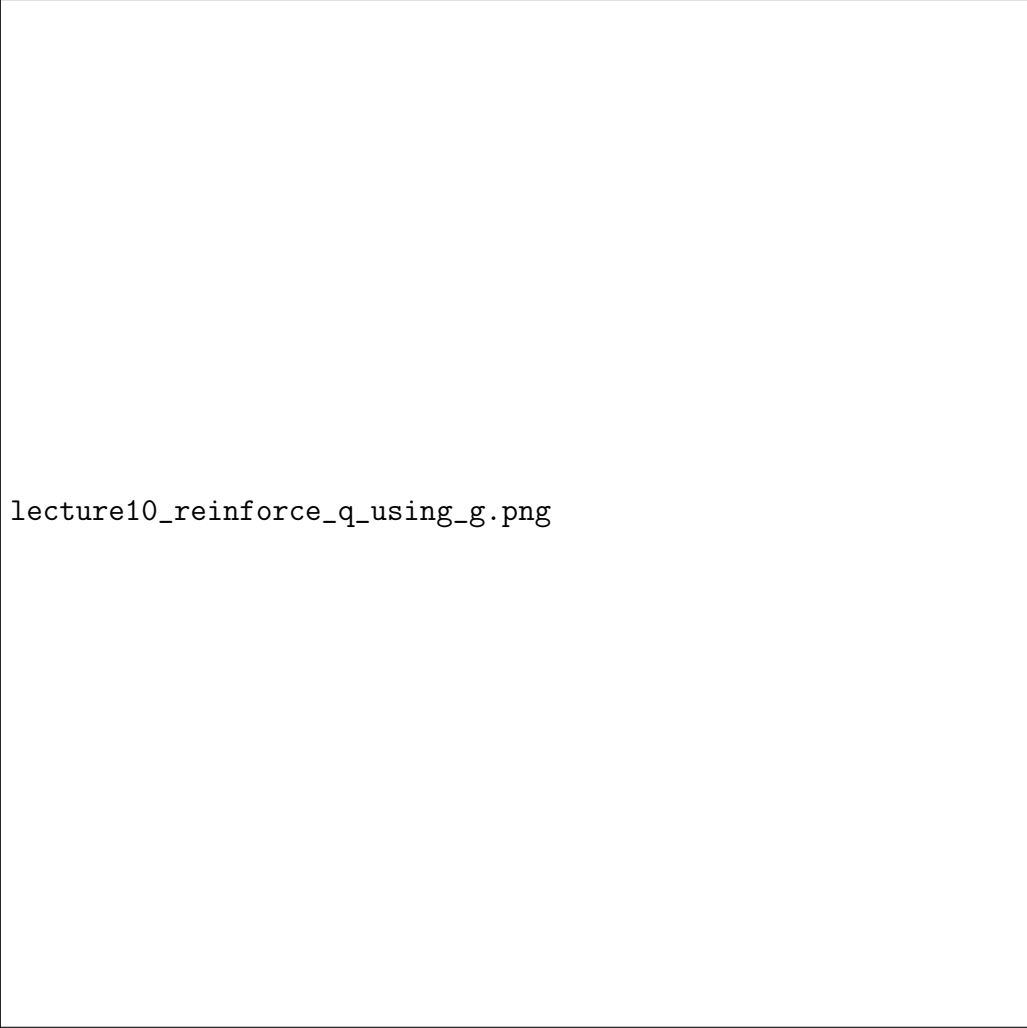
Так как:

$$Q^{\pi}(s_t, a_t) = \mathbb{E}[G_t \mid s_t, a_t],$$

можно использовать G_t как sample estimate для $Q^{\pi}(s_t, a_t)$.

Тогда gradient estimate:

$$\nabla_{\theta} J \approx \frac{1}{N} \sum_{i=1}^N \sum_{(s_t, a_t) \in \tau_i} \nabla_{\theta} \log \pi_{\theta}(a_t \mid s_t) G_t.$$



lecture10_reinforce_q_using_g.png

Figure 58: REINFORCE оценивает $Q^\pi(s, a)$ через return G_t , посчитанный по sampled trajectory.

10.16 Вычисление discounted returns

Return можно считать рекурсивно:

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$$

$$G_t = r_t + \gamma(r_{t+1} + \gamma r_{t+2} + \dots).$$

То есть:

$$G_t = r_t + \gamma G_{t+1}.$$

Благодаря этому все G_t для trajectory можно посчитать за линейное время, проходя с конца эпизода к началу.



lecture10_discounted_returns_linear_time.png

Figure 59: Рекурсивная формула $G_t = r_t + \gamma G_{t+1}$ позволяет посчитать returns за линейное время.

10.17 REINFORCE является on-policy

REINFORCE сэмплирует trajectories из текущей policy:

$$\tau_i \sim \pi_\theta.$$

Затем gradient оценивается по actions, выбранным этой же policy.

Поэтому REINFORCE является **on-policy** алгоритмом.

Это означает, что после обновления policy старые trajectories становятся менее корректными для обучения новой policy.

10.18 Интуиция policy gradient

Формула:

$$\nabla_\theta J \approx \sum_t \nabla_\theta \log \pi_\theta(a_t | s_t) G_t$$

имеет простую интерпретацию.

Если G_t большой:

увеличиваем вероятность a_t в s_t .

Если G_t маленький или отрицательный:

уменьшаем вероятность a_t в s_t .

Множитель

$$\nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$

указывает, как изменить параметры, чтобы увеличить вероятность выбранного действия.

10.19 Проблема высокой дисперсии

REINFORCE может иметь высокую дисперсию.

Причина: G_t зависит от всей последующей trajectory и может сильно шуметь.

Например, действие в хорошем состоянии может получить плохой return из-за неудачных будущих случайностей. Или хорошее действие в плохом состоянии может выглядеть хуже, чем оно есть.

Нужно уменьшать variance gradient estimate.

Один из главных способов:

baseline.

10.20 Baselines в REINFORCE

В policy gradient можно вычитать baseline $b(s)$, не зависящий от действия:

$$\nabla_{\theta} J = \mathbb{E}[\nabla_{\theta} \log \pi_{\theta}(a | s)(Q(s, a) - b(s))].$$

Такой baseline не меняет математическое ожидание gradient.

Почему?

Второй член:

$$\mathbb{E}_{a \sim \pi_{\theta}}[\nabla_{\theta} \log \pi_{\theta}(a | s)b(s)]$$

равен:

$$b(s)\mathbb{E}_{a \sim \pi_{\theta}}[\nabla_{\theta} \log \pi_{\theta}(a | s)].$$

A:

$$\begin{aligned}\mathbb{E}_{a \sim \pi_\theta}[\nabla_\theta \log \pi_\theta(a \mid s)] &= \sum_a \pi_\theta(a \mid s) \nabla_\theta \log \pi_\theta(a \mid s) \\ &= \sum_a \nabla_\theta \pi_\theta(a \mid s) = \nabla_\theta \sum_a \pi_\theta(a \mid s) = \nabla_\theta 1 = 0.\end{aligned}$$

Значит baseline не меняет direction of expected gradient.

10.21 Зачем нужен baseline

Хотя baseline не меняет expected gradient, он может уменьшить дисперсию.

Gradient variance зависит от:

$$\text{Var}[Q(s, a) - b(s)].$$

Если $b(s)$ коррелирует с $Q(s, a)$, variance уменьшается.

Наивный baseline:

$$b = \text{moving average of } Q.$$

Более сильный baseline:

$$b(s) = V(s).$$

Тогда:

$$Q(s, a) - V(s)$$

называется advantage.

10.22 Advantage function

Advantage показывает, насколько действие a лучше среднего действия в состоянии s :

$$A(s, a) = Q(s, a) - V(s).$$

Интерпретация:

- $A(s, a) > 0$: действие лучше среднего для этого состояния;
- $A(s, a) < 0$: действие хуже среднего;
- $A(s, a) = 0$: действие примерно соответствует среднему уровню состояния.

Policy gradient с advantage:

$$\nabla_{\theta} J \approx \frac{1}{N} \sum_i \sum_{s,a \in \tau_i} \nabla_{\theta} \log \pi_{\theta}(a | s) A(s, a).$$

10.23 Actor-Critic

Actor-Critic объединяет policy-based и value-based подходы.

Есть две части:

- **actor** — policy:

$$\pi_{\theta}(a | s);$$

- **critic** — value function:

$$V_w(s)$$

или

$$Q_w(s, a).$$

Актор выбирает действия, критик оценивает, насколько они хороши.

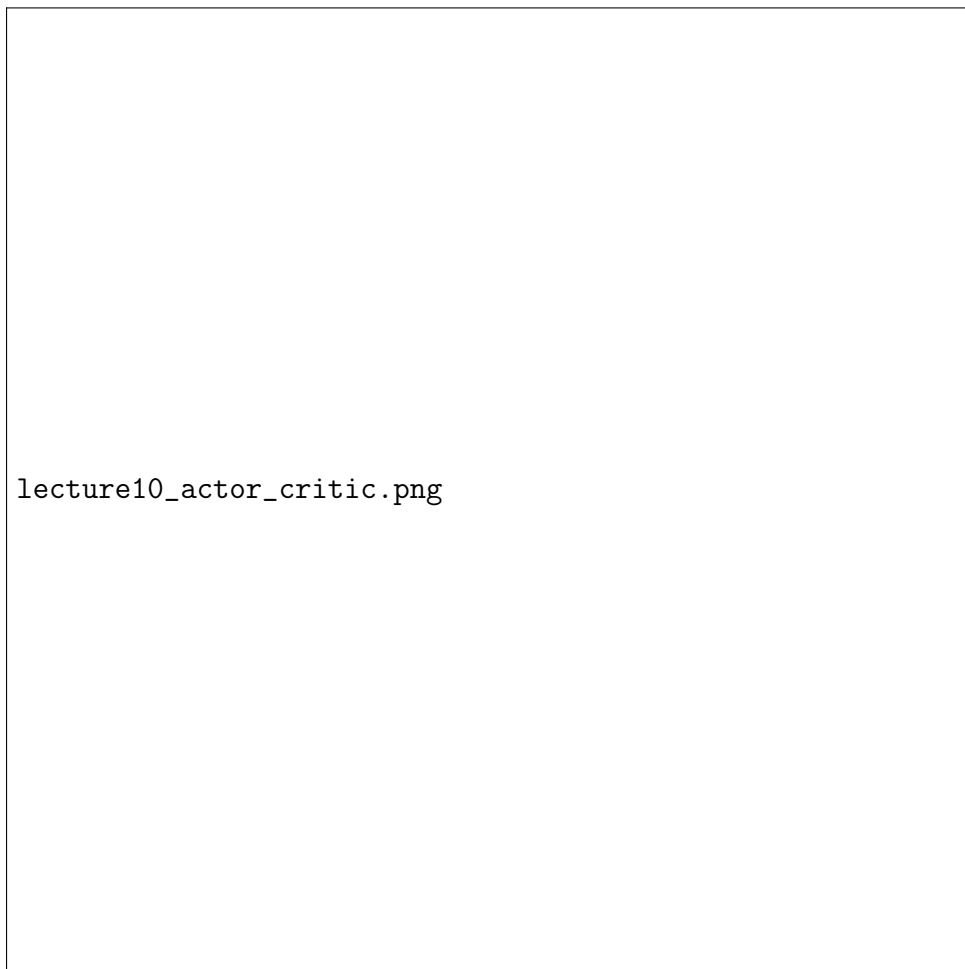


Figure 60: Actor-Critic: actor учит policy, critic учит value function.

10.24 Advantage Actor-Critic

В Advantage Actor-Critic используется:

$$A(s, a) = Q(s, a) - V(s).$$

Но $Q(s, a)$ можно приблизить через one-step bootstrap:

$$Q(s, a) \approx r + \gamma V(s').$$

Тогда:

$$A(s, a) \approx r + \gamma V(s') - V(s).$$

Это также похоже на temporal difference error:

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t).$$

Actor update:

$$\nabla_{\theta} J_{actor} \approx \frac{1}{N} \sum_i \sum_{s, a \in \tau_i} \nabla_{\theta} \log \pi_{\theta}(a | s) A(s, a).$$

Critic обучается минимизировать ошибку value prediction:

$$L_{critic} \approx \frac{1}{N} \sum_i \sum_{s, a \in \tau_i} (V_w(s) - [r + \gamma V_w(s')])^2.$$

10.25 Continuous action spaces

Policy gradient методы естественно работают с continuous action spaces.

Если действий бесконечно много, не нужно брать:

$$\arg \max_a Q(s, a)$$

по непрерывному множеству.

Можно просто задать policy distribution, например:

$$\pi_{\theta}(a | s) = \mathcal{N}(\mu_{\theta}(s), \sigma_{\theta}^2(s)).$$

А затем сэмплировать:

$$a \sim \pi_{\theta}(a | s).$$

Алгоритм почти не меняется: нужно только уметь считать

$$\log \pi_{\theta}(a \mid s)$$

и его gradient.

10.26 Entropy regularization

В policy gradient методах policy может слишком быстро стать почти детерминированной. Тогда exploration ухудшается.

Чтобы предотвратить premature convergence, добавляют entropy regularization.

Entropy policy:

$$H(\pi(\cdot \mid s)) = - \sum_a \pi(a \mid s) \log \pi(a \mid s).$$

К objective можно добавить:

$$J'(\theta) = J(\theta) + \beta \mathbb{E}_s[H(\pi_{\theta}(\cdot \mid s))].$$

Это поощряет более случайную policy и поддерживает exploration.

10.27 A3C

A3C расшифровывается как:

Asynchronous Advantage Actor-Critic.

Основные идеи:

- parallel game sessions;
- async multi-CPU training;
- no experience replay;
- LSTM policy;
- n-step advantage.

Несколько worker-ов параллельно взаимодействуют со своими копиями среды и обновляют global network.



lecture10_a3c_scheme.png

Figure 61: A3C использует параллельные sessions и asynchronous updates.

10.28 IMPALA

IMPALA — масштабируемая actor-critic архитектура.

Ключевые идеи:

- massively parallel training;
- separate actor and learner processes;
- небольшой experience replay;
- importance sampling для коррекции различий между policy акторов и learner-a.

IMPALA отделяет сбор опыта от обучения модели, что позволяет масштабировать RL на большое число сред.

10.29 Практические замечания

В лекции выделяются несколько практических советов:

- ошибки $V(s)$ менее критичны, чем ошибки $Q(s, a)$ в Q-learning: actor всё ещё может учиться, даже если critic пока слабый;
- полезно регуляризовать policy entropy, чтобы избежать ранней потери exploration;
- можно обучаться на parallel sessions;
- для численной стабильности удобно использовать `logsoftmax`.

10.30 Главные выводы

- Value-based методы учат Q -function, а policy потом получается через $\arg \max$.
- Иногда учить точные Q -values сложнее, чем напрямую учить policy.
- Policy-based методы напрямую параметризуют:

$$\pi_{\theta}(a \mid s).$$

- Policy gradient максимизирует expected return:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}}[G(\tau)].$$

- Log-derivative trick:

$$\nabla_{\theta} \pi_{\theta}(z) = \pi_{\theta}(z) \nabla_{\theta} \log \pi_{\theta}(z).$$

- REINFORCE использует gradient:

$$\nabla_{\theta} J \approx \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t \mid s_t) G_t.$$

- REINFORCE является on-policy.
- Baseline $b(s)$ не меняет expected gradient, но может уменьшить variance.
- Advantage:

$$A(s, a) = Q(s, a) - V(s).$$

- Actor-critic учит одновременно policy и value function.

- Advantage actor-critic использует:

$$A(s, a) \approx r + \gamma V(s') - V(s).$$

- Policy gradient естественно работает с continuous action spaces.

10.31 Что важно помнить к экзамену

1. Value-based policy:

$$\pi(s) = \arg \max_a Q_\theta(s, a).$$

2. Policy-based model:

$$\pi_\theta(a \mid s).$$

3. Deterministic policy:

$$a = \pi_\theta(s).$$

4. Stochastic policy:

$$a \sim \pi_\theta(a \mid s).$$

5. Policy gradient objective:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[G(\tau)].$$

6. Log-derivative trick:

$$\nabla_\theta \log \pi_\theta(z) = \frac{1}{\pi_\theta(z)} \nabla_\theta \pi_\theta(z).$$

7. Policy gradient:

$$\nabla_\theta J = \mathbb{E}[\nabla_\theta \log \pi_\theta(a \mid s) Q^\pi(s, a)].$$

8. REINFORCE estimate:

$$\nabla_\theta J \approx \frac{1}{N} \sum_{i=1}^N \sum_t \nabla_\theta \log \pi_\theta(a_t^i \mid s_t^i) G_t^i.$$

9. Discounted return:

$$G_t = r_t + \gamma G_{t+1}.$$

10. Baseline:

$$\nabla_\theta J = \mathbb{E}[\nabla_\theta \log \pi_\theta(a \mid s)(Q(s, a) - b(s))].$$

11. Baseline не должен зависеть от action a .

12. Advantage:

$$A(s, a) = Q(s, a) - V(s).$$

13. One-step advantage estimate:

$$A(s, a) \approx r + \gamma V(s') - V(s).$$

14. Actor loss direction:

$$\nabla_{\theta} J_{actor} \approx \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) A(s_t, a_t).$$

15. Critic loss:

$$L_{critic} \approx \sum_t (V_w(s_t) - [r_t + \gamma V_w(s_{t+1})])^2.$$

16. Continuous actions:

$$\pi_{\theta}(a | s) = \mathcal{N}(\mu_{\theta}(s), \sigma_{\theta}^2(s)).$$

17. A3C:

Asynchronous Advantage Actor-Critic.

““

“‘latex

11 Лекция 11. Reinforcement Learning in Language Modeling and SCST

11.1 План лекции

В этой лекции рассматриваются:

- общая RL-постановка для sequence generation;
- encoder-decoder задачи;
- supervised seq2seq learning;
- проблемы distribution shift;
- неоднозначность правильных ответов в machine translation;
- применение RL к seq2seq;

- seq2seq как POMDP;
- policy gradient для генерации текста;
- supervised learning vs policy gradient;
- training vs inference mismatch;
- self-critical sequence training;
- image captioning with SCST;
- discrete GANs как бонусная тема.

Главная идея лекции: генерацию текста можно рассматривать как задачу Reinforcement Learning, где policy — это decoder, action — следующий токен, а reward — внешняя метрика качества вроде BLEU, CIDEr или domain-specific reward. Это позволяет оптимизировать не likelihood reference-последовательности, а непосредственно целевую метрику.

11.2 Общий RL-формализм

В общем виде задача RL записывается как максимизация expected reward по policy:

$$J = \mathbb{E}_{s \sim d(s), a \sim \pi(a|obs(s))} [R(s, a)].$$

Здесь:

- s — состояние среды;
- $obs(s)$ — наблюдение агента;
- a — действие;
- $\pi(a | obs(s))$ — policy;
- $R(s, a)$ — reward.

Reward может быть black-box функцией. Например, в sequence generation он может быть внешней метрикой качества:

$$R(x, y) = BLEU(y, y_{ref}),$$

или

$$R(x, y) = CIDEr(y, y_{ref}).$$

В частном случае можно использовать discounted return:

$$G(s, a) = r(s, a) + \gamma G(s', a').$$

Markov property:

$$P(s' \mid s, a, *) = P(s' \mid s, a).$$

Если наблюдение полностью описывает состояние, то:

$$obs(s) = s.$$

Но в seq2seq-задачах часто удобнее говорить о частично наблюдаемой задаче, потому что decoder видит только вход и уже сгенерированные токены, а не всю скрытую структуру правильного ответа.

11.3 Общие подходы к оптимизации policy

Для оптимизации policy можно использовать разные подходы:

- **Evolution strategies:** немного изменять policy, запускать её и оставлять варианты с большим J .
- **Value-based methods:** оценивать J или $Q(s, a)$ как функцию действия и выбирать лучшее действие.
- **Policy gradient:** напрямую подниматься по градиенту ∇J относительно параметров policy.
- **Bayesian optimization:** строить surrogate model для J и выбирать наиболее информативные policy.
- **Simulated annealing.**
- **Cross-Entropy Method.**

В этой лекции основной акцент сделан на policy gradient для sequence generation.

11.4 Encoder-decoder architectures

Encoder-decoder архитектуры решают задачи вида:

$$x \rightarrow y,$$

где вход x может быть последовательностью или произвольным объектом, а выход y — последовательность.

Encoder читает input data, decoder генерирует output sequence.

Примеры задач:

- machine translation;
- image captioning;
- speech recognition или word-to-transcript;
- conversation systems;
- image to LaTeX;
- code to docstring.



Figure 62: Encoder-decoder задачи: один объект или последовательность преобразуется в выходную последовательность.

11.5 Supervised seq2seq learning

В supervised seq2seq learning есть датасет пар:

$$(x, y),$$

где:

- x — вход;
- y — reference output.

Например, в machine translation:

x = sentence in Chinese,

y = sentence in English.

Обычная supervised objective:

$$\max_{\theta} \log P_{\theta}(y \mid x).$$

Для sequence output:

$$P_{\theta}(y \mid x) = \prod_{t=1}^T P_{\theta}(y_t \mid x, y_{<t}).$$

Тогда negative log-likelihood loss:

$$\mathcal{L}_{CE} = - \sum_{t=1}^T \log P_{\theta}(y_t \mid x, y_{<t}).$$

Это teacher forcing: во время обучения decoder получает правильные предыдущие токены $y_{<t}$.

11.6 Примеры supervised seq2seq задач

11.6.1 Machine translation

Постановка:

- прочитать предложение на исходном языке;
- сгенерировать предложение на целевом языке;
- предложения должны иметь одинаковый смысл.

Supervised solution:

$$\max_{\theta} \log P_{\theta}(\text{translation} \mid \text{source}).$$

11.6.2 Conversation systems

Постановка:

- прочитать фразу пользователя;
- сгенерировать ответ;
- система должна поддерживать диалог.

Supervised solution:

$$\max_{\theta} \log P_{\theta}(\text{response} \mid \text{phrase}).$$

11.6.3 Grapheme to phoneme

Постановка:

- прочитать слово как последовательность символов;
- сгенерировать транскрипцию как последовательность фонем.

Например:

hedgehog $\rightarrow$ phoneme sequence.

Supervised solution:

$$\max_{\theta} \log P_{\theta}(\text{transcript} \mid \text{word}).$$

11.7 Attentive translation

В attentive translation decoder на каждом шаге сам выбирает, на какие hidden states encoder-а смотреть.

То есть decoder не обязан использовать только один финальный encoder state. Вместо этого он получает weighted combination encoder states.

Attention одновременно помогает учить:

- word alignment;
- word translation.



lecture11_attentive_translation_alignment.png

Figure 63: Attention в машинном переводе одновременно учит word alignment и translation.

11.8 Обобщённая постановка sequence generation

Многие задачи можно записать в общем виде:

$$x \sim X,$$

$$y \sim Y.$$

Нужно произвести ответ:

$$y^* = \arg \max_y R(x, y).$$

Здесь $R(x, y)$ — метрика качества ответа.

Обычное supervised решение:

- взять датасет хороших пар (x, y) ;

- максимизировать:

$$\log P_{\theta}(y \mid x).$$

Этот подход хорошо работает, если данные:

- многочисленные;
- качественные;
- близкие к оптимальным по $R(x, y)$.

Проблема: часто эти условия не выполняются.

11.9 Distribution shift

В supervised seq2seq есть важное расхождение между training и inference.

Во время обучения модель предсказывает:

$$P_{\theta}(y_{t+1} \mid x, y_{0:t}),$$

где:

$$y_{0:t} \sim \text{reference}.$$

То есть предыдущие токены взяты из правильного ответа.

Во время inference модель предсказывает:

$$P_{\theta}(y_{t+1} \mid x, \hat{y}_{0:t}),$$

где:

$$\hat{y}_{0:t} \sim \text{model}.$$

Если модель на каком-то шаге ошиблась, следующий hidden state оказывается в области, которую модель почти не видела при обучении. Ошибка может усиливаться на следующих шагах.

Это называется:

distribution shift.



lecture11_distribution_shift.png

Figure 64: Distribution shift в seq2seq: training использует reference prefix, inference использует generated prefix.

11.10 Проблема неоднозначности правильного ответа

В machine translation часто существует много корректных переводов одного и того же source sentence.

Например, один source sentence может иметь варианты:

- “I’m searching for some gifts for my family.”
- “I want to find something for my family as presents.”
- “I’m about to buy some presents for my family.”
- “I’d like to buy my family something as a gift.”
- “I’m looking for a present for my family.”

Supervised maximum likelihood заставляет модель поднимать вероятность конкретных reference translations. Но модели не обязательно нужно выучить все возможные правильные переводы. Ей достаточно генерировать хотя бы один хороший.

Проблема likelihood: модель может иметь хороший mean log probability на reference data, но при этом давать много плохих outputs вне датасета.

11.11 Проблемы conversation systems

Для диалоговых систем есть два типа данных:

- большие сырые данные;
- маленькие чистые данные.

Большие сырые данные:

- Twitter, subtitles, books, logs;
- 10^6 – 10^8 samples;
- достаточно большие;
- но могут иметь suboptimal $R(x, y)$.

Маленькие чистые данные:

- moderated logs;
- assessor-written conversations;
- 10^2 – 10^4 samples;
- ответы ближе к оптимальным;
- но данных мало.

Если обучить банковского Q&A-бота на сырых социальных данных, он может выучить нежелательное поведение. Классическая проблема: модель оптимизирует статистику датасета, а не бизнес-цель и не безопасность.

11.12 Seq2seq как POMDP

Sequence generation можно рассматривать как частично наблюдаемый MDP.

Для seq2seq:

- hidden state s — состояние перевода или диалога;
- initial state s_0 — encoder output;
- observation o_t — предыдущие сгенерированные слова;
- action a_t — написать следующий токен;
- reward r — domain-specific reward, например BLEU.

То есть decoder policy:

$$\pi_{\theta}(a_t \mid obs(s_t))$$

выбирает следующий токен.

Полная последовательность действий:

$$a_1, a_2, \dots, a_T$$

образует generated output.

Reward может начисляться в конце всей последовательности:

$$R(x, \hat{y}).$$



lecture11_seq2seq_as_pomdp.png

Figure 65: Seq2seq можно рассматривать как POMDP: decoder выбирает следующий токен как действие.

11.13 Policy Gradient для sequence generation

Пусть objective:

$$J(\theta) = \mathbb{E}_{s \sim p(s), a \sim \pi_\theta(a|s)}[R(s, a)].$$

В seq2seq случае a можно понимать как всю сгенерированную последовательность:

$$a = \hat{y}.$$

Reward:

$$R(s, a)$$

может быть BLEU, CIDEr, пользовательская оценка или другой black-box score.

Наивная Monte Carlo оценка:

$$J(\theta) \approx \frac{1}{N} \sum_{i=1}^N R(s_i, a_i).$$

Но нужно вычислить gradient по θ , а параметры находятся внутри policy:

$$\pi_\theta(a \mid s).$$

Для этого используется log-derivative trick.

11.14 Log-derivative trick

Основное тождество:

$$\nabla_\theta \log \pi_\theta(z) = \frac{1}{\pi_\theta(z)} \nabla_\theta \pi_\theta(z).$$

Отсюда:

$$\pi_\theta(z) \nabla_\theta \log \pi_\theta(z) = \nabla_\theta \pi_\theta(z).$$

Это позволяет переписать gradient expected reward:

$$\nabla_\theta J = \int_s p(s) \int_a \nabla_\theta \pi_\theta(a \mid s) R(s, a) da ds.$$

Подставляя log-derivative trick:

$$\nabla_\theta J = \int_s p(s) \int_a \pi_\theta(a \mid s) \nabla_\theta \log \pi_\theta(a \mid s) R(s, a) da ds.$$

То есть:

$$\nabla_\theta J = \mathbb{E}_{s,a} [\nabla_\theta \log \pi_\theta(a \mid s) R(s, a)].$$

Оценка по samples:

$$\nabla_\theta J \approx \frac{1}{N} \sum_{i=1}^N \nabla_\theta \log \pi_\theta(a_i \mid s_i) R(s_i, a_i).$$

11.15 Supervised Learning vs Policy Gradient

Supervised learning оптимизирует likelihood reference actions:

$$\nabla \ell_{lh} = \mathbb{E}_{s, a_{opt} \sim D} [\nabla \log \pi_\theta(a_{opt} \mid s)].$$

Policy gradient оптимизирует sampled actions, взвешенные по reward:

$$\nabla J = \mathbb{E}_{s \sim d(s), a \sim \pi_\theta(a|obs(s))} [\nabla \log \pi_\theta(a | s) Q(s, a)].$$

Главные различия:

- supervised learning обучается на reference sessions;
- policy gradient обучается на собственных сгенерированных outputs;
- supervised learning требует хорошие reference y_{opt} ;
- policy gradient требует reward function $R(x, y)$;
- supervised learning имеет меньшую дисперсию;
- policy gradient имеет большую дисперсию и cold start problem;
- supervised learning страдает от distribution shift;
- policy gradient не имеет такого mismatch, потому что обучается на собственных outputs.

11.16 Cold start problem

Policy gradient плохо работает с полностью случайной policy.

Если decoder генерирует случайный текст, то почти все outputs будут плохими, reward будет низким и noisy. Локальные улучшения такой policy вряд ли быстро приведут к хорошей генерации.

Поэтому на практике часто используют двухэтапную схему:

supervised pre-training $\rightarrow$ RL fine-tuning.

Сначала модель учится генерировать разумные последовательности через maximum likelihood. Затем RL fine-tuning оптимизирует целевую метрику.

11.17 Training vs inference mismatch

Для encoder-decoder RNN encoder ведёт себя одинаково при training и inference: он читает input sequence и строит representation.

Decoder отличается.

Во время RL-style training decoder может сэмплировать действия:

$$a_t \sim \pi_\theta(\cdot | h_t).$$

Во время inference обычно используется greedy decoding:

$$a_t = \arg \max_a \pi_\theta(a \mid h_t),$$

или beam search.

То есть simplified scheme:

- encoder: same behavior for training and inference;
- decoder: sample during training, act greedy on inference.



Figure 66: В seq2seq encoder ведёт себя одинаково, а decoder при training может sample-ить, при inference действует greedy.

11.18 Self-Critical Sequence Training

Self-Critical Sequence Training, SCST — метод, который использует inference mode как baseline.

Идея:

- сгенерировать output в sampling mode;
- посчитать reward sampled output:

$$R(s, a);$$

- сгенерировать output в greedy inference mode;
- посчитать reward greedy output:

$$R(s, a_{inference}(s));$$

- использовать difference как advantage.

Advantage в SCST:

$$A(s, a) = R(s, a) - R(s, a_{inference}(s)).$$

Policy gradient:

$$\nabla J = \mathbb{E}[\nabla \log \pi_{\theta}(a \mid s) A(s, a)].$$



lecture11_scst_advantage.png

Figure 67: SCST использует greedy inference output как baseline для sampled output.

11.19 Почему baseline берётся из inference mode

Можно было бы использовать sampling mode для baseline, но это хуже по двум причинам:

- sampling baseline более noisy, потому что сам зависит от случайного sampling;
- в production обычно используется greedy или beam decoding, а не random sampling.

SCST сравнивает sampled output с тем, что модель реально сделала бы на inference. Если sampled output лучше greedy output, его вероятность увеличивается. Если хуже — уменьшается.

11.20 Интуиция SCST

Если:

$$R(s, a) > R(s, a_{inference}(s)),$$

то:

$$A(s, a) > 0.$$

Значит sampled sequence лучше текущего greedy поведения модели. Нужно увеличить вероятность sampled actions.

Если:

$$R(s, a) < R(s, a_{inference}(s)),$$

то:

$$A(s, a) < 0.$$

Значит sampled sequence хуже baseline. Нужно уменьшить её вероятность.

Таким образом, модель учится улучшать свою собственную inference-policy.

11.21 Image Captioning with SCST

В image captioning:

- input x — изображение;
- output y — caption;
- reward — метрика качества caption, например CIDEr;
- dataset — MSCOCO.

Обучение:

1. pre-training:

$$\max_{\theta} \log P_{\theta}(\text{caption} \mid \text{image});$$

2. fine-tuning:

$$\max_{\theta} \mathbb{E}[\text{CIDEr}(\hat{y}, y_{ref})].$$

SCST использует self-critical baseline для уменьшения дисперсии gradient.

11.22 Результаты SCST

В презентации приводится таблица результатов для image captioning. Модели сравниваются по нескольким метрикам:

- CIDEr;
- BLEU4;
- ROUGE1;
- METEOR.

Важный вывод: когда SCST оптимизирует конкретную метрику, качество по этой метрике обычно растёт.

Например, оптимизация CIDEr даёт высокий CIDEr score, оптимизация BLEU улучшает BLEU, и так далее.



Figure 68: SCST позволяет напрямую оптимизировать целевые sequence-level метрики вроде CIDEr, BLEU, ROUGE и METEOR.

11.23 Common pitfalls

При RL fine-tuning sequence models важно учитывать несколько рисков.

11.23.1 Reward hacking

Агент может найти способ “обмануть” reward:

maximize metric $\neq$ solve intended task.

Например, модель может выучить шаблон, который хорошо выглядит для автоматической метрики, но плохо соответствует реальному качеству.

11.23.2 Overfitting

Модель может overfit-иться под training data или под конкретную reward metric.

Поэтому нужно проверять validation performance и желательно использовать несколько метрик и human evaluation.

11.23.3 Долгое обучение

RL methods обычно дольше обучаются from scratch. Поэтому почти всегда полезно сначала pre-train model in supervised mode.

11.24 Практические советы

Для seq2seq RL fine-tuning полезны следующие приёмы:

- pre-train model in supervised mode;
- использовать entropy regularization;
- использовать L2 regularization logits;
- применять better sampling techniques;
- использовать стандартные seq2seq tricks;
- для большого vocabulary использовать bottleneck или softmax improvements.

11.25 Bonus: discrete GANs

В обычном GAN есть:

- generator $G(z)$;

- discriminator $D(x)$;
- real data;
- fake data.

Discriminator выдаёт вероятность, что объект real:

$$D(x) = P(real \mid x).$$

Для непрерывных данных gradient от discriminator может проходить в generator. Но если generator выдаёт discrete objects, backpropagation через sampling невозможен. Примеры discrete generation:

- text;
- music notes;
- molecules;
- binary image masks.

В этом случае generator можно обучать через policy gradient: generator рассматривается как policy, discrete output — как action, а reward приходит от discriminator.

Также возможны альтернативы:

- Gumbel-Softmax;
- hard attention tricks;
- discrete loss functions;
- binary network tricks.

11.26 Сравнение supervised seq2seq и RL fine-tuning

Supervised seq2seq	RL fine-tuning / Policy Gradient
Требуется reference outputs. Оптимизирует likelihood: $\log P(y \mid x)$.	Требуется reward function. Оптимизирует expected reward: $\mathbb{E}[R(x, \hat{y})]$.
Обучается на reference prefixes. Меньшая дисперсия gradient. Есть distribution shift между training и inference.	Обучается на generated outputs. Большая дисперсия gradient. Distribution shift меньше, потому что модель учится на собственных outputs.
Хорошо для pre-training.	Хорошо для post-training под конкретную метрику.

Table 15: Сравнение supervised seq2seq learning и RL fine-tuning.

11.27 Главные выводы

- Sequence generation можно рассматривать как RL-задачу.
- Decoder policy выбирает следующий токен:

$$a_t \sim \pi_\theta(a_t \mid \text{obs}(s_t)).$$

- Reward может быть black-box sequence-level metric, например BLEU или CIDEr.
- Supervised seq2seq learning оптимизирует likelihood reference outputs.
- Supervised learning страдает от distribution shift: training использует reference prefix, inference использует generated prefix.
- В machine translation и диалогах часто существует много корректных outputs.
- Policy gradient обучается на собственных outputs и может напрямую оптимизировать целевую метрику.
- Policy gradient имеет cold start problem и высокую дисперсию.
- Поэтому используют:

supervised pre-training $\rightarrow$ RL fine-tuning.

- SCST использует greedy inference output как baseline:

$$A(s, a) = R(s, a) - R(s, a_{inference}(s)).$$

- SCST хорошо подходит для image captioning и других seq2seq-задач с внешней метрикой.
- Policy gradient можно использовать для discrete generators, например text GANs.

11.28 Что важно помнить к экзамену

1. Общий objective:

$$J = \mathbb{E}_{s \sim d(s), a \sim \pi(a|obs(s))} [R(s, a)].$$

2. Supervised seq2seq:

$$\max_{\theta} \log P_{\theta}(y \mid x) = \max_{\theta} \sum_t \log P_{\theta}(y_t \mid x, y_{<t}).$$

3. Distribution shift:

$$\text{training: } y_{0:t} \sim \text{reference},$$

$$\text{inference: } \hat{y}_{0:t} \sim \text{model}.$$

4. Seq2seq as POMDP:

- state — translation/conversation state;
- observation — previous words;
- action — next word;
- reward — BLEU/CIDEr/domain metric.

5. Log-derivative trick:

$$\nabla_{\theta} \pi_{\theta}(z) = \pi_{\theta}(z) \nabla_{\theta} \log \pi_{\theta}(z).$$

6. Policy gradient:

$$\nabla_{\theta} J = \mathbb{E}[\nabla_{\theta} \log \pi_{\theta}(a \mid s) R(s, a)].$$

7. Supervised gradient:

$$\nabla \ell_{lh} = \mathbb{E}_{s, a_{opt} \sim D} [\nabla \log \pi_{\theta}(a_{opt} \mid s)].$$

8. Policy gradient обучается на:

$$a \sim \pi_{\theta}(a \mid obs(s)).$$

9. Supervised learning требует хорошие reference y_{opt} , RL требует reward function.

10. SCST advantage:

$$A(s, a) = R(s, a) - R(s, a_{inference}(s)).$$

11. SCST gradient:

$$\nabla J = \mathbb{E}[\nabla \log \pi_{\theta}(a | s) A(s, a)].$$

12. Практическая схема:

pre-train with cross-entropy $\rightarrow$ fine-tune with RL / SCST.

13. В discrete GANs generator можно обучать policy gradient, потому что backpropagation через discrete sampling невозможен.

““